



Specifica Tecnica

Autore	Alessandro Morabito, Alessandro Frison, Lorenzo Grolla, Nicolò Lattanzio, Damiano Berti, Giacomo Nalotto, Giulia Romanato
Verificatore	Alessandro Morabito, Alessandro Frison, Lorenzo Grolla, Nicolò Lattanzio, Damiano Berti, Giacomo Nalotto, Giulia Romanato
Approvazione	Giacomo Nalotto

Registro delle versioni

Versione	Data	Autore	Verificatore	Descrizione delle modifiche
1.0.0	21/04/2026	Giacomo Nalotto	Tutto il gruppo	Approvazione
0.17.0	21/04/2026	Damiano Berti	Giulia Romanato	Aggiunte Auth, User e Membership in backend
0.16.0	21/04/2026	Giacomo Nalotto	Alessandro Morabito	Aggiunta modulo Report
0.15.0	20/04/2026	Alessandro Frison	Nicolò Lattanzio	Aggiunte classi e tipi in scan
0.14.0	20/04/2026	Giacomo Nalotto	Giulia Romanato	Aggiunta modulo Repository
0.13.0	20/04/2026	Nicolò Lattanzio	Alessandro Frison	Aggiunti i tipi all'interno della sezione Container
0.12.0	20/04/2026	Giulia Romanato	Alessandro Frison	Aggiunta modulo WorkspaceManager e WorkspaceUser
0.11.0	19/04/2026	Giacomo Nalotto	Damiano Berti	Aggiunta modulo WorkspaceRepository
0.10.0	19/04/2026	Lorenzo Grolla	Nicolò Lattanzio	Aggiunta sezione Frontend
0.9.0	17/04/2026	Alessandro Morabito	Giacomo Nalotto	Aggiunte sezioni per nodi: sintetizzatore, analisi dei test, GitHub
0.8.0	16/04/2026	Nicolò Lattanzio	Lorenzo Grolla	Aggiunta dei nodi di security, Remediation e Documentazione
0.7.0	14/04/2026	Alessandro Morabito	Damiano Berti	Aggiunta sezioni AppModule, ScanModule, ReporterModule in Container
0.6.0	13/04/2026	Alessandro Frison	Giacomo Nalotto	Aggiunta sezione Scan
0.5.0	12/04/2026	Nicolò Lattanzio	Lorenzo Grolla	Aggiunta sezione Container con Nodo Dependency
0.4.0	10/04/2026	Alessandro Frison	Alessandro Morabito	Aggiunta sezione requisiti
0.3.0	25/03/2026	Alessandro Frison	Giulia Romanato	Aggiunta sezione introduzione

Versione	Data	Autore	Verificatore	Descrizione (continua)
0.2.0	25/03/2026	Lorenzo Grolla	Alessandro Frison	Stesura tecnologie utilizzate
0.1.0	24/03/2026	Alessandro Morabito	Lorenzo Grolla	Creazione documento

Indice

1	Introduzione	9
1.1	Scopo del documento	9
1.2	Glossario	9
1.3	Scopo del prodotto	9
1.4	Riferimenti	9
1.4.1	Riferimenti normativi	9
1.4.2	Riferimenti informativi	10
2	Tecnologie	10
2.1	Linguaggi di programmazione	10
2.1.1	TypeScript	10
2.2	Formato dei dati	10
2.2.1	JSON	10
2.3	Framework	10
2.3.1	NestJS	10
2.3.2	LangGraph	11
2.4	Librerie	11
2.4.1	Librerie backend	11
2.4.1.1	LangChain	11
2.4.1.2	Axios	12
2.4.1.3	dotenv	12
2.4.1.4	isomorphic-git	12
2.4.1.5	Mongoose	12
2.4.1.6	Passport.js	13
2.4.2	Librerie frontend	13
2.4.2.1	Tailwind CSS	13
2.4.2.2	shadcn	13
2.4.2.3	React	14
2.4.2.4	TanStack Router	14
2.4.2.5	react-markdown	14
2.5	Autenticazione	14
2.5.1	Amazon Cognito	14
2.6	Database	15
2.6.1	MongoDB	15
2.7	Testing	15
2.7.1	Jest	15
2.7.2	Vitest	15
2.8	Infrastruttura e deployment	16
2.8.1	Docker	16
2.8.2	AWS Amplify	16
2.8.3	Amazon ECS	16
2.8.4	Amazon EC2	16
3	Architettura di deployment	17

4	Architettura logica	17
4.1	Frontend	17
4.1.1	Introduzione	17
4.1.2	Design pattern	18
4.1.2.1	Repository	18
4.1.2.2	Dependency Injection	18
4.1.3	Infrastruttura condivisa	18
4.1.3.1	apiClient	19
4.1.3.2	ApiError	19
4.1.3.3	Componenti di presentazione condivisi	20
4.1.4	Feature	20
4.1.4.1	Auth	21
4.1.4.1.1	Tipi e DTO	21
4.1.4.1.2	Componenti View	22
4.1.4.1.3	Hook	22
4.1.4.1.4	Repository e Service	23
4.1.4.2	Membership	24
4.1.4.2.1	Tipi e DTO	24
4.1.4.2.2	Componenti View	25
4.1.4.2.3	Hook	25
4.1.4.2.4	Repository	26
4.1.4.3	CreateWorkspace	27
4.1.4.3.1	Tipi e DTO	27
4.1.4.3.2	Componenti View	28
4.1.4.3.3	Hook	28
4.1.4.3.4	Repository	28
4.1.4.4	WorkspaceMembers	29
4.1.4.4.1	Tipi e DTO	29
4.1.4.4.2	Componenti View	29
4.1.4.4.3	Hook	30
4.1.4.4.4	Repository	31
4.1.4.5	WorkspaceRepository	32
4.1.4.5.1	Tipi e DTO	33
4.1.4.5.2	Componenti View	33
4.1.4.5.3	Hook	34
4.1.4.5.4	Repository	34
4.1.4.6	WorkspaceList	36
4.1.4.6.1	Tipi e DTO	36
4.1.4.6.2	Componenti View	36
4.1.4.6.3	Hook	37
4.1.4.6.4	Repository	38
4.1.4.7	Scan	38
4.1.4.7.1	Tipi e DTO	39
4.1.4.7.2	Componenti View	39
4.1.4.7.3	Hook	39
4.1.4.7.4	Repository	40
4.1.4.8	Report	41

	4.1.4.8.1	Tipi e DTO	42
	4.1.4.8.2	Componenti View	44
	4.1.4.8.3	Hook	45
	4.1.4.8.4	Repository	46
4.2	Backend		48
4.2.1	Introduzione		48
4.2.2	Design pattern		48
4.2.2.1	Dependency injection		48
4.2.2.2	Singleton		48
4.2.2.3	Repository pattern		48
4.2.3	Moduli		49
4.2.3.1	Auth		49
	4.2.3.1.1	Tipi	49
	4.2.3.1.2	Classi e Interfacce	49
4.2.3.2	User		51
	4.2.3.2.1	Tipi	51
	4.2.3.2.2	Classi e Interfacce	52
4.2.3.3	Membership		54
	4.2.3.3.1	Tipi	54
	4.2.3.3.2	Classi e Interfacce	56
4.2.3.4	Workspace Manager		60
	4.2.3.4.1	Tipi	60
	4.2.3.4.2	Classi e Interfacce	62
4.2.3.5	Workspace User		65
	4.2.3.5.1	Tipi	66
	4.2.3.5.2	Classi e Interfacce	66
4.2.3.6	Workspace Repository		69
	4.2.3.6.1	Tipi	70
	4.2.3.6.2	Classi e Interfacce	71
4.2.3.7	Repository		73
	4.2.3.7.1	Tipi	73
	4.2.3.7.2	Classi e Interfacce	74
4.2.3.8	Report		77
	4.2.3.8.1	Tipi	78
	4.2.3.8.2	Classi e Interfacce	80
4.2.3.9	Scan		82
	4.2.3.9.1	Modelli e Schemi (Database)	82
	4.2.3.9.2	Classi e Interfacce	83
4.2.3.10	Scan Manager		85
	4.2.3.10.1	Tipi	85
	4.2.3.10.2	Classi e Interfacce	85
4.2.3.11	Scan Status		88
	4.2.3.11.1	Tipi	88
	4.2.3.11.2	Classi e Interfacce	88
4.2.3.12	Scan Auth		90
	4.2.3.12.1	Tipi	90
	4.2.3.12.2	Classi e Interfacce	90

4.3	Container	91
4.3.1	Design pattern	93
4.3.1.1	Dependency Injection	93
4.3.2	AppModule	93
4.3.2.1	AppService	93
4.3.3	ReporterModule	94
4.3.3.1	ReporterService	94
4.3.4	ScanModule	94
4.3.4.1	ScanService	95
4.3.5	OrchestratorModule	95
4.3.5.1	OrchestratorService	95
4.3.5.2	OrchestratorHelper	96
4.3.6	Nodo sintetizzatore	96
4.3.6.1	SynthesizerNodeService	96
4.3.6.2	SynthesizerNodeHelper	96
4.3.7	Nodo analisi dei test	97
4.3.7.1	CoverageNodeService	97
4.3.7.2	CoverageNodeHelper	97
4.3.8	Nodo GitHub	98
4.3.8.1	GithubNodeService	98
4.3.8.2	GithubNodeHelper	98
4.3.9	Nodo analisi della documentazione	99
4.3.9.1	DocsNodeService	99
4.3.9.2	DocsNodeHelper	99
4.3.10	Nodo analisi della sicurezza del codice	100
4.3.10.1	SecurityNodeService	100
4.3.10.2	SecurityNodeHelper	101
4.3.11	Nodo creazione delle remediation	101
4.3.11.1	RemediationNodeService	102
4.3.11.2	RemediationNodeHelper	102
4.3.12	Nodo analisi della dipendenze	103
4.3.12.1	DependencyNodeService	103
4.3.12.2	DependencyNodeHelper	104
4.3.13	Tipi	105
4.3.13.1	DepsReportUnit	105
4.3.13.2	DepsVulnerabilityUnit	105
4.3.13.3	SemgrepResult	105
4.3.13.4	RemediationEntry	106
4.3.13.5	RemediationNodeInput	106
4.3.13.6	VulnerabilityUnit	106
4.3.13.7	Vulnerability	107
4.3.13.8	DepsReport	107
4.3.13.9	DocsReport	107
4.3.13.10	CoverageToolResult	108
4.3.13.11	CoverageNodeResult	108
4.3.13.12	CoverageReport	108
4.3.13.13	TestReport	108

4.3.13.14 FailedTest	109
4.3.13.15 Language	109
4.3.13.16 TechReport	109
4.3.13.17 ReportSummary	109
4.3.13.18 ReportMetadata	110
4.3.13.19 DataReport	110
4.3.13.20 Report	110
4.3.13.21 Target	111
4.3.13.22 ReportedTarget	111
4.3.13.23 WorkflowState	111
4.3.13.24 PartialWorkflowState	112
4.3.13.25 RepositoryConnectionInfo	113
4.3.13.26 SendReportInfo	113
4.3.13.27 SendErrorNotificationInfo	113
5 Stato dei requisiti funzionali	114
5.1 Requisiti Funzionali	114
5.2 Requisiti di Qualità	130
5.3 Requisiti di Vincolo	130

Elenco delle figure

1	UML feature Auth	21
2	UML feature Membership	24
3	UML feature create Workspace	27
4	UML feature workspaceRepository	32
5	UML feature WorkspaceList	36
6	UML feature Scan	38
7	UML feature Report	41
8	UML type reportInfo	42
9	UML Modulo Auth	49
10	UML Modulo User	51
11	UML Modulo Membership	54
12	UML WorkspaceManager	60
13	UML WorkspaceUser	65
14	UML WorkspaceRepository	69
15	UML Repository	73
16	UML Report	77
17	UML Modulo Scan	82
18	UML Container (Scanner)	91
19	UML Tipi Container (Scanner)	92
20	Moduli del container di scansione	93
21	UML AppModule	94
22	UML ReporterModule	94
23	UML ScanModule	95
24	UML OrchestratorModule	95
25	UML SynthesizerModule	96
26	UML CoverageModule	97
27	UML GithubModule	98
28	UML DocsModule	99
29	UML SecurityModule	100
30	UML RemediationModule	102
31	UML DependencyModule	103

1 Introduzione

1.1 Scopo del documento

Il presente documento si pone l'obiettivo di definire in modo esaustivo la **Specifica Tecnica** del progetto, fornendo una guida dettagliata alle scelte architetture e implementative adottate. Esso rappresenta il ponte logico tra i requisiti iniziali e l'effettiva realizzazione del software, garantendo che ogni componente sia integrata in modo coerente e scalabile.

Il documento approfondisce l'evoluzione del sistema rispetto al **Proof of Concept (PoC)** iniziale, delineando una struttura robusta e orientata alla manutenibilità a lungo termine.

Nello specifico, questo documento si propone di:

- **Delineare l'architettura di sistema:** fornendo una panoramica delle macro-componenti e del flusso di dati tra di esse;
- **Dettagliare le scelte tecnologiche:** giustificando l'adozione di specifici framework, librerie e strumenti di sviluppo;
- **Illustrare i design pattern:** descrivendo gli schemi logici e le best practice utilizzate per risolvere problematiche ricorrenti e ottimizzare la struttura del codice;
- **Definire i processi di deployment:** spiegando come le diverse componenti vengono distribuite, configurate e rese operative nell'ambiente di produzione;
- **Fornire una base per la manutenzione:** offrendo la documentazione necessaria affinché futuri sviluppatori possano comprendere ed estendere il sistema con facilità.

1.2 Glossario

Al fine di prevenire ambiguità e garantire una comunicazione uniforme e precisa tra i membri del gruppo e i committenti, è stato redatto un Glossario^G apposito. Per facilitare la lettura del presente documento, i termini tecnici o dotati di un significato specifico all'interno del dominio di progetto sono contrassegnati da una lettera «G» posta in apice (es. Parola^G). La definizione estesa di tali termini è reperibile nel documento citato tra i riferimenti informativi.

1.3 Scopo del prodotto

Il sistema CodeGuardian è progettato per condurre analisi approfondite sulla qualità di repository GitHub^G, garantendo un controllo rigoroso sulla gestione degli accessi e sull'autorizzazione all'avvio delle procedure di scansione.

La piattaforma si propone come uno strumento strategico per team di sviluppo multidisciplinari, consentendo il monitoraggio centralizzato dello stato del software e la generazione di metriche aggregate su molteplici progetti analizzati.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- Norme Di Progetto^G V1.1.0
https://byte-holders.github.io/Documentazione/RTB/Norme_Di_Progetto-V1.1.0.pdf
- Capitolato
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C2.pdf>

1.4.2 Riferimenti informativi

- Glossario V1.0.0

<https://byte-holders.github.io/Documentazione/RTB/Glossario-V1.0.0.pdf>

2 Tecnologie

Questa sezione descrive gli strumenti e le tecnologie adottati per lo sviluppo, il testing e il deployment del software relativo al progetto CodeGuardian. In particolare, vengono illustrati il linguaggio di programmazione, le librerie, i framework e le infrastrutture utilizzate.

2.1 Linguaggi di programmazione

2.1.1 TypeScript

TypeScript è un linguaggio di programmazione open-source sviluppato da Microsoft. Si tratta di un superset di JavaScript, che introduce la tipizzazione statica, consentendo di rilevare errori di programmazione in fase di sviluppo, ridurre il numero di bug e migliorare la qualità del codice.

- **Versione:** 5.7.3
- **Documentazione:** <https://www.typescriptlang.org/docs/> [visitato il 21/04/2026]

Nel contesto del progetto CodeGuardian, TypeScript è il linguaggio principale utilizzato per lo sviluppo sia del frontend che del backend. La tipizzazione statica è particolarmente utile in un progetto che coinvolge strutture dati complesse, come i report di analisi del codice e i grafi di scansione generati dall'agente AI.

Per il frontend si utilizza TSX, un'estensione di TypeScript che permette di definire la struttura dell'interfaccia utente in modo più leggibile e dichiarativo, integrando HTML direttamente nel codice TypeScript.

2.2 Formato dei dati

2.2.1 JSON

JSON (JavaScript Object Notation) è un formato di scambio dati leggero e indipendente dal linguaggio di programmazione. È ampiamente utilizzato nello sviluppo web e nello scambio di dati tra applicazioni. JSON è basato su due strutture di dati:

- **Oggetti:** insiemi di coppie chiave-valore;
- **Array:** liste ordinate di valori.

Nel progetto CodeGuardian, JSON viene utilizzato per la trasmissione dei dati tra frontend e backend (payload delle API REST^G), per la serializzazione dei risultati di analisi, nonché per la configurazione dell'applicazione e la memorizzazione dei dati in MongoDB.

2.3 Framework

2.3.1 NestJS

NestJS è un framework per Node.js^G che offre un'architettura modulare e scalabile per la creazione di applicazioni lato server. Usa Express.js come HTTP adapter di default e fornisce fun-

zionalità aggiuntive come dependency injection^G, middleware e decorator. NestJS consente di organizzare il codice in moduli riutilizzabili e di definire facilmente controller, servizi e provider.

- **Versione:** 11.0.1
- **Documentazione:** <https://docs.nestjs.com/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, NestJS è utilizzato per creare il backend. In particolare, viene impiegato per definire i controller che gestiscono le richieste HTTP provenienti dal frontend, i servizi che implementano la business logic (gestione workspace^G, membership^G, scansioni), e i provider per l'iniezione delle dipendenze. L'architettura del backend è organizzata in sette moduli principali: WorkspaceModule, MembershipModule, RepositoryModule, ScanModule, ReportModule, AnalysisModule e GitHubModule.

2.3.2 LangGraph

LangGraph è un framework open-source per l'orchestrazione di agenti AI^G, sviluppato da LangChain Inc. Consente di modellare workflow complessi come grafi a stati, dove ogni nodo rappresenta un passo di ragionamento o un'azione, e ogni arco il flusso di controllo tra i passi. LangGraph fornisce funzionalità di gestione dello stato, checkpointing, streaming token-by-token e supporto per cicli e ramificazioni condizionali.

- **Versione:** 1.2.6
- **Documentazione:** <https://langchain-ai.github.io/langgraph/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, LangGraph è il componente centrale dell'agente di analisi del codice. Il grafo implementato definisce il flusso di scansione dei repository GitHub^G: dalla clonazione del repository, all'analisi dei file, fino alla generazione del report finale. La struttura a grafo consente di gestire ramificazioni condizionali basate sul tipo di analisi richiesta e permette di integrare verifiche intermedie e retry automatici in caso di errore.

2.4 Librerie

2.4.1 Librerie backend

2.4.1.1 LangChain

LangChain è una libreria open-source che fornisce componenti riutilizzabili per l'integrazione di Large Language Model (LLM)^G in applicazioni software. Offre astrazioni per la gestione dei prompt, l'invocazione dei modelli, l'integrazione con tool esterni e la gestione della memoria conversazionale.

- **Versione:** 1.1.36
- **Documentazione:** <https://js.langchain.com/docs/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, LangChain è utilizzato in combinazione con LangGraph per la costruzione dell'agente di analisi. Fornisce le astrazioni per l'invocazione dei modelli LLM e l'integrazione con i tool di analisi del codice sorgente.

2.4.1.2 Axios

Axios è un client HTTP basato su Promise per Node.js e browser. È isomorfo, cioè può essere eseguito sia nel browser che in Node.js utilizzando lo stesso codice sorgente. Lato server, Axios utilizza il modulo `http` nativo di Node.js, mentre lato client usa `XMLHttpRequest`.

- **Versione:** 1.14
- **Documentazione:** <https://axios-http.com/docs/intro> [visitato il 21/04/2026]

Nel progetto CodeGuardian, Axios viene utilizzato lato frontend per effettuare richieste HTTP al backend (interrogazione delle API REST per workspace, scansioni e report) e lato backend per comunicare con servizi esterni, come le API di GitHub.

2.4.1.3 dotenv

dotenv è una libreria che consente di caricare variabili d'ambiente da un file `.env` nell'oggetto `process.env` di Node.js. Permette di separare la configurazione dal codice sorgente, seguendo le best practice della metodologia Twelve-Factor App.

- **Versione:** 17.3
- **Documentazione:** <https://github.com/motdotla/dotenv> [visitato il 21/04/2026]

Nel progetto CodeGuardian, dotenv è utilizzato per gestire in modo sicuro le credenziali sensibili (chiavi API dei servizi LLM, credenziali MongoDB, configurazioni AWS) sia in ambiente di sviluppo che di testing, evitando di esporre dati riservati nel codice sorgente.

2.4.1.4 isomorphic-git

isomorphic-git è una reimplementazione completa di Git in puro JavaScript, compatibile sia con Node.js che con ambienti browser. Consente di eseguire operazioni Git (clone, checkout, log, status) senza dipendenze da moduli nativi C++, garantendo piena interoperabilità con l'implementazione canonica di Git.

- **Versione:** 1.37
- **Documentazione:** <https://isomorphic-git.org/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, isomorphic-git è una componente fondamentale del modulo `GitHubModule`. Viene utilizzato per clonare i repository GitHub degli utenti direttamente nel backend, consentendo all'agente AI di analizzare il codice sorgente localmente senza dipendere da strumenti CLI esterni. Ciò garantisce portabilità e compatibilità con ambienti containerizzati.

2.4.1.5 Mongoose

Mongoose è una libreria ODM (Object Data Modeling) per MongoDB e Node.js. Fornisce una soluzione basata su schema per modellare i dati dell'applicazione, includendo validazione built-in, casting dei tipi, costruzione di query e hook per la business logic.

- **Versione:** 9.3
- **Documentazione:** <https://mongoosejs.com/docs/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, Mongoose è utilizzato come layer di accesso ai dati per MongoDB. Tutti i modelli del dominio applicativo (workspace, membership, repository, scan, report) sono definiti tramite schema Mongoose, garantendo la validazione dei dati a livello applicativo e la coerenza delle operazioni CRUD.

2.4.1.6 Passport.js

Passport.js è un popolare middleware di autenticazione per Node.js, progettato per essere estremamente flessibile e modulare. Il suo punto di forza principale risiede nell'utilizzo delle cosiddette strategie, che permettono agli sviluppatori di integrare facilmente oltre 500 metodi di accesso diversi all'interno di un'applicazione

- **Versione:** 0.7.0
- **Documentazione:** <https://www.passportjs.org/docs/> [visitato il 21/04/2026]

Nel progetto Passport.js viene utilizzato per gestire l'autenticazione di ogni richiesta http, controllando che l'access token ottenuto facendo il login tramite AWS Cognito sia valido

2.4.2 Librerie frontend

2.4.2.1 Tailwind CSS

Tailwind CSS è un framework CSS utility-first che permette di costruire interfacce utente direttamente nel markup, componendo classi di basso livello invece di scrivere CSS personalizzato. A differenza dei framework tradizionali basati su componenti predefiniti, Tailwind fornisce primitive atomiche altamente componibili che offrono pieno controllo sul design.

- **Versione:** 4.1
- **Documentazione:** <https://tailwindcss.com/docs> [visitato il 21/04/2026]

Nel progetto CodeGuardian, Tailwind CSS è utilizzato come sistema di stile principale per l'intera interfaccia frontend. Ogni componente React è stilizzato tramite classi utility, con l'utilizzo di variabili CSS personalizzate per la gestione dei temi chiaro e scuro, garantendo coerenza visiva e manutenibilità dello stile.

2.4.2.2 shadcn

shadcn/ui è una collezione di componenti UI riutilizzabili costruiti su Radix UI e Tailwind CSS. A differenza delle librerie di componenti tradizionali, shadcn/ui non è un pacchetto npm ma una raccolta di componenti che vengono copiati direttamente nel progetto, offrendo piena libertà di personalizzazione senza dipendenze esterne aggiuntive.

- **Versione:** 1.4
- **Documentazione:** <https://ui.shadcn.com/docs> [visitato il 21/04/2026]

Nel progetto CodeGuardian, shadcn/ui fornisce i componenti base dell'interfaccia utente come dialog, button, input e select. Ogni componente è stato integrato nel sistema di design del progetto tramite le variabili CSS di Tailwind, mantenendo accessibilità e coerenza visiva in tutta l'applicazione.

2.4.2.3 React

React è una libreria JavaScript open-source per la creazione di interfacce utente, sviluppata da Meta. Consente di definire componenti riutilizzabili che rappresentano parti dell'interfaccia, aggiornando automaticamente la vista in base allo stato dell'applicazione. React adotta un approccio dichiarativo per la definizione dell'interfaccia utente, semplificando la gestione dello stato e la sincronizzazione con il DOM.

- **Versione:** 19.2.0
- **Documentazione:** <https://react.dev/learn> [visitato il 21/04/2026]

Nel progetto CodeGuardian, React è il framework di riferimento per l'interfaccia utente. La UI è organizzata in componenti funzionali che gestiscono la visualizzazione dei workspace, la lista dei repository, l'avvio delle scansioni e la consultazione dei report di analisi.

2.4.2.4 TanStack Router

TanStack Router è una libreria open-source di routing type-safe per applicazioni React. A differenza di React Router, offre un sistema di routing completamente tipizzato a livello di TypeScript, con validazione dei parametri di ricerca, caricamento dati integrato e supporto nativo per search params tipizzati.

- **Versione:** 1.168.23
- **Documentazione:** <https://tanstack.com/router/latest> [visitato il 21/04/2026]

Nel progetto CodeGuardian, TanStack Router gestisce la navigazione tra le diverse sezioni dell'applicazione: la dashboard iniziale, la vista dei workspace, le pagine di dettaglio dei repository e le schermate di visualizzazione dei report. L'integrazione type-safe con TypeScript garantisce che tutti i parametri delle rotte siano validati staticamente.

2.4.2.5 react-markdown

react-markdown è una libreria React che consente di renderizzare contenuto Markdown come componenti React. Supporta la sintassi CommonMark e può essere estesa tramite plugin remark e rehype per funzionalità aggiuntive (syntax highlighting, tabelle, GFM).

- **Versione:** 10.1.0
- **Documentazione:** <https://github.com/remarkjs/react-markdown> [visitato il 21/04/2026]

Nel progetto CodeGuardian, react-markdown viene utilizzato per visualizzare i report di analisi generati dall'agente AI, i quali sono prodotti in formato Markdown. La libreria consente di presentare all'utente il report in modo formattato, includendo intestazioni, elenchi, blocchi di codice ed eventuali tabelle riassuntive.

2.5 Autenticazione

2.5.1 Amazon Cognito

Amazon Cognito è un servizio di autenticazione e autorizzazione gestito da Amazon Web Services (AWS)^G. Consente di aggiungere funzionalità di sign-up, sign-in e gestione degli accessi

alle applicazioni web e mobile. Supporta provider di identità federata (come Google e GitHub), autenticazione multi-fattore (MFA) e gestione dei token JWT^G.

- **Documentazione:** <https://docs.aws.amazon.com/cognito/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, Amazon Cognito è il servizio di riferimento per la gestione dell'autenticazione degli utenti. Si occupa della registrazione, dell'accesso (anche tramite provider OAuth come GitHub), del rilascio e della verifica dei token JWT, e della gestione delle sessioni. Il backend NestJS verifica i token Cognito per proteggere le API e implementare il controllo degli accessi basato sui ruoli (owner, admin, member) all'interno dei workspace.

2.6 Database

2.6.1 MongoDB

MongoDB è un database NoSQL^G open-source, orientato ai documenti. A differenza dei database relazionali, MongoDB memorizza i dati in documenti flessibili in formato BSON (Binary JSON), consentendo strutture dati dinamiche che possono variare tra i documenti di una stessa collezione. MongoDB supporta query ricche, indicizzazione, aggregazioni e scalabilità orizzontale tramite sharding.

- **Documentazione:** <https://www.mongodb.com/docs/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, MongoDB è il database principale dell'applicazione. Viene utilizzato per la persistenza di tutte le entità del dominio: workspace, membership, repository, scansioni e report di analisi. La natura document-oriented di MongoDB si adatta alla struttura dei report generati dall'agente AI, che possono variare in formato e complessità. L'accesso ai dati avviene tramite la libreria Mongoose.

2.7 Testing

2.7.1 Jest

Jest è un framework di testing progettato per garantire la correttezza di qualsiasi sorgente JavaScript. Consente di scrivere test con un'API accessibile e ricca di funzionalità, che fornisce risultati rapidamente. Supporta mocking, snapshot testing, code coverage e test paralleli.

- **Versione:** 30.0.0
- **Documentazione:** <https://jestjs.io/docs/getting-started> [visitato il 21/04/2026]

Jest è utilizzato per eseguire i test di unità e di integrazione del backend, verificando che i controller, i servizi e i moduli NestJS siano implementati correttamente. In particolare, viene impiegato per testare la logica di business dei moduli dell'applicazione.

2.7.2 Vitest

Vitest è un framework di testing per Vite. Offre funzionalità avanzate come il supporto nativo per TypeScript, la configurazione tramite file di setup e la possibilità di eseguire test in modalità watch. Questa modalità è particolarmente utile durante lo sviluppo attivo e il debugging, poiché riduce i tempi di attesa per l'esecuzione dei test.

- **Versione:** 3.2.4
- **Documentazione:** <https://vitest.dev/guide/> [visitato il 21/04/2026]

Vitest è utilizzato per eseguire i test di unità e di integrazione del frontend, verificando che i componenti React e le funzionalità di interfaccia siano implementati correttamente.

2.8 Infrastruttura e deployment

2.8.1 Docker

Docker è una piattaforma open-source per lo sviluppo, la distribuzione e l'esecuzione di applicazioni in contenitori. I contenitori Docker sono unità di software che includono il codice sorgente, le dipendenze e le configurazioni necessarie per eseguire un'applicazione, garantendo ambienti consistenti e facilmente riproducibili.

- **Documentazione:** <https://docs.docker.com/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, Docker è utilizzato per containerizzare i singoli servizi dell'applicazione (frontend, backend, agente AI) durante lo sviluppo e il testing locale, garantendo coerenza tra gli ambienti di sviluppo e produzione.

2.8.2 AWS Amplify

AWS Amplify è un servizio di hosting e deployment gestito da Amazon Web Services, progettato per applicazioni frontend e web full-stack. Supporta il deployment continuo da repository Git, la gestione automatica di certificati SSL, CDN globale e preview per le pull request.

- **Documentazione:** <https://docs.amplify.aws/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, AWS Amplify è utilizzato per il deployment e l'hosting del frontend React. Fornisce un ambiente di hosting scalabile con deployment continuo collegato al repository GitHub del progetto.

2.8.3 Amazon ECS

Amazon Elastic Container Service (ECS) è un servizio di orchestrazione di contenitori completamente gestito da AWS. Consente di eseguire, fermare e gestire contenitori Docker su un cluster di istanze EC2 o tramite AWS Fargate (modalità serverless). ECS supporta la scalabilità automatica, il bilanciamento del carico e l'integrazione nativa con altri servizi AWS.

- **Documentazione:** <https://docs.aws.amazon.com/ecs/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, Amazon ECS è utilizzato per il deployment e l'orchestrazione dei container che eseguono il backend NestJS e il servizio dell'agente AI basato su LangGraph. ECS garantisce alta disponibilità e scalabilità dei servizi backend in ambiente di produzione.

2.8.4 Amazon EC2

Amazon Elastic Compute Cloud (EC2) è un servizio di cloud computing che fornisce capacità computazionale ridimensionabile nel cloud AWS. Consente di lanciare istanze di server virtuali con diversi sistemi operativi e configurazioni hardware.

- **Documentazione:** <https://docs.aws.amazon.com/ec2/> [visitato il 21/04/2026]

Nel progetto CodeGuardian, le istanze EC2 costituiscono l'infrastruttura computazionale su cui vengono eseguiti i container gestiti da Amazon ECS. La scelta del tipo di istanza è calibrata in base ai requisiti di memoria e CPU necessari per l'esecuzione delle analisi AI.

3 Architettura di deployment

Per la distribuzione dell'applicazione nell'ambiente di produzione AWS, si è optato per un'architettura monolitica containerizzata. In questo modello, tutte le componenti del backend (gestione utenti, analisi del codice, generazione report) sono incluse in un'unica immagine Docker e distribuite come un singolo servizio su Amazon ECS.

Motivazioni della scelta:

- Proporzionalità rispetto alla scala del progetto: per un sistema delle dimensioni di CodeGuardian, l'adozione di un'architettura a microservizi risulterebbe ingente da implementare e nettamente sovradimensionata. Il passaggio a sistemi distribuiti introdurrebbe una complessità accidentale non giustificata dal carico di lavoro previsto;
- Semplificazione del flusso DevOps: la gestione di un unico artefatto riduce drasticamente l'overhead operativo relativo alla configurazione dei cluster ECS e dei bilanciatori di carico;
- Efficienza nelle operazioni I/O: poiché l'analisi del codice richiede la clonazione locale dei repository, la presenza di tutti i moduli nello stesso ambiente di esecuzione elimina la latenza di rete che si verificherebbe in un'architettura a microservizi;

4 Architettura logica

4.1 Frontend

4.1.1 Introduzione

L'architettura del frontend di *CodeGuardian* adotta il pattern **MVVM** (Model-View-ViewModel), che separa l'applicazione in tre layer con dipendenze strettamente unidirezionali:

View → **ViewModel** → **Model** → `apiClient`

View: componenti React responsabili della presentazione e dell'interazione con l'utente. Si suddividono in *componenti pagina*, che compongono la struttura delle schermate e applicano le guard di autenticazione (`ProtectedRoute`), e *componenti di presentazione*, che ricevono dati tramite props o invocando hook del layer `ViewModel` e si occupano esclusivamente del rendering;

ViewModel: custom hook React che incapsulano la logica di stato e orchestrano le operazioni di lettura e scrittura. Utilizzano `useQuery` e `useMutation` di TanStack Query per gestire rispettivamente il recupero dati con caching automatico e le operazioni di modifica con invalidazione reattiva della cache, eccetto in casi specifici in cui la struttura gerarchica dei provider React non rende disponibile il client TanStack Query, dove si ricorre a `useState/useEffect`;

Model: il layer di dominio è costituito dai DTO tipizzati che rappresentano le entità applicative e dai repository che ne gestiscono il recupero e la persistenza. I repository implementano interfacce segregate per responsabilità secondo il principio ISP: ad esempio, le operazioni sui repository di un workspace sono suddivise in `IGetRepositoriesRepository`, `IAddRepositoryRepository`, `IRemoveRepositoryRepository` e `IUpdateTokenRepository`, quattro interfacce distinte invece di un'unica interfaccia. La comunicazione HTTP con il backend avviene tramite il modulo condiviso `apiClient`, che espone funzioni tipizzate per i verbi HTTP

(apiGet, apiPost, apiPut, apiPatch, apiDelete). O

Nessun layer inferiore ha conoscenza dei layer superiori: i repository non conoscono né gli hook che li consumano né i componenti che renderizzano i dati, ma ricevono parametri primitivi e restituiscono DTO tipizzati. I contratti tra i layer sono definiti da interfacce TypeScript suddivise in due categorie: *interfacce Model*, che dichiarano i metodi esposti dai repository, e *interfacce ViewModel*, che dichiarano le proprietà e le azioni restituite dagli hook alla View. Dove rilevante per la testabilità, gli hook accettano l'implementazione del repository come parametro opzionale con valore di default, consentendo la sostituzione con mock nei test unitari tramite dependency injection; per gli hook basati su TanStack Query (useMutation, useQuery), il mocking avviene invece a livello di modulo.

Il routing è gestito da **TanStack Router** con tipizzazione statica dei parametri di rotta. Le rotte protette sono avvolte dal componente `ProtectedRoute`, che verifica lo stato di autenticazione tramite `useAuthContext` e, in caso di utente non autenticato, reindirizza automaticamente al flusso di login preservando il percorso richiesto per consentire il ritorno alla destinazione originale dopo l'accesso.

4.1.2 Design pattern

4.1.2.1 Repository

Il pattern **Repository** è applicato sistematicamente a ogni accesso al backend: le chiamate HTTP e la serializzazione dei dati sono incapsulate in classi dedicate che espongono ai consumer soltanto metodi di dominio, nascondendo i dettagli del protocollo di trasporto e dei percorsi degli endpoint. Ogni repository concreto implementa una corrispondente interfaccia — ad esempio `WorkspaceListRepository` implementa `IWorkspaceListRepository` — e delega le chiamate HTTP al modulo `ApiClient`. Gli hook del layer `ViewModel` dipendono esclusivamente dall'interfaccia, non dall'implementazione concreta, garantendo un chiaro disaccoppiamento tra logica applicativa e infrastruttura di comunicazione.

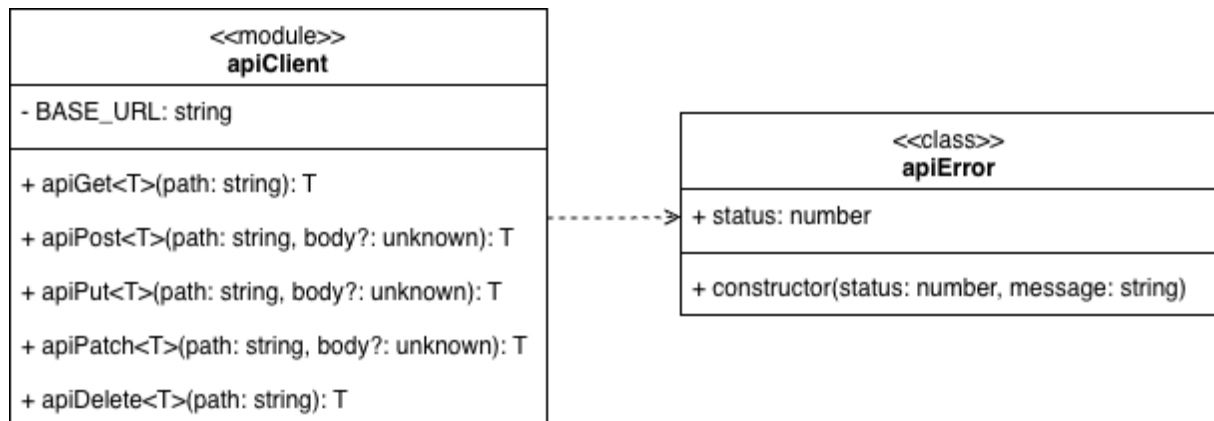
4.1.2.2 Dependency Injection

Il pattern **Dependency Injection** ha l'obiettivo di minimizzare il grado di dipendenza fra le parti di un sistema, fornendo tecniche per passare le dipendenze a una classe in modo che l'accoppiamento fra le parti risulti il più lasco possibile. Nel frontend questo pattern è realizzato negli hook che necessitano di interagire con i repository, tramite un parametro opzionale che accetta l'implementazione del repository e che assume come valore di default il singleton di produzione. In questo modo l'hook resta indipendente rispetto all'implementazione concreta con cui verrà utilizzato e può ricevere dall'esterno una versione alternativa qualora il contesto lo richieda.

4.1.3 Infrastruttura condivisa

Prima di descrivere le singole feature si descrivono i principali componenti trasversali utilizzati dall'intero frontend.

4.1.3.1 apiClient



Il modulo `apiClient` è il punto di accesso unico per tutte le chiamate HTTP verso il backend applicativo. Ogni repository concreto del layer Model delega a questo modulo l'esecuzione delle richieste, evitando la duplicazione della logica di autenticazione e di gestione degli errori. Il modulo espone cinque funzioni tipizzate, una per ciascun verbo HTTP utilizzato dal sistema:

- `apiGet<T>(path: string): Promise<T>` — esegue una richiesta GET all'endpoint specificato;
- `apiPost<T>(path: string, body?: unknown): Promise<T>` — esegue una richiesta POST, serializzando opzionalmente il body come JSON;
- `apiPut<T>(path: string, body?: unknown): Promise<T>` — esegue una richiesta PUT con le stesse semantiche di `apiPost`;
- `apiPatch<T>(path: string, body?: unknown): Promise<T>` — esegue una richiesta PATCH;
- `apiDelete<T>(path: string): Promise<T>` — esegue una richiesta DELETE.

Ogni funzione aggiunge automaticamente un header `Authorization: Bearer <token>` alla richiesta, recuperando l'`accessToken` dalla sessione Amplify corrente tramite `fetchAuthSession`. Questo garantisce che tutte le chiamate al backend siano autenticate senza richiedere ai repository di gestire esplicitamente i token. Fa eccezione la registrazione iniziale dell'utente, gestita da `AuthRepository`, che non utilizza `apiClient` perché richiede l'`idToken` invece dell'`accessToken` per accedere agli *identity claims* necessari al backend.

Al termine di ogni richiesta, il modulo esamina lo stato HTTP della risposta tramite la funzione interna `handleResponse`. Se la risposta indica successo, il corpo viene deserializzato come JSON e restituito; se il corpo è vuoto viene restituito `undefined`. Se la risposta indica un errore, viene sollevata un'istanza di `ApiError` contenente lo status HTTP e un messaggio estratto dal corpo della risposta quando disponibile, o un messaggio di default altrimenti.

4.1.3.2 ApiError

ApiError è la classe di errore utilizzata in tutto il frontend per rappresentare i fallimenti delle chiamate HTTP. Estende la classe nativa `Error` aggiungendo il campo pubblico `readonly status: number`, che espone il codice di stato HTTP della risposta fallita. Questa caratterizzazione consente ai repository di implementare logiche di gestione differenziate in base al tipo

di errore: un esempio è `GetBranchesRepository`, che tratta gli errori 404 e 403 come assenza di branch (restituendo un array vuoto) e propaga invece tutti gli altri errori al chiamante.

4.1.3.3 Componenti di presentazione condivisi

RepositoryCard è il componente di presentazione che visualizza un repository all'interno di una card. Riceve le seguenti props:

- `name: string` — nome del repository;
- `ownerName: string` — username del proprietario;
- `dateScan?: string` — data dell'ultima scansione, opzionale;
- `documentationScore?: number` — punteggio della documentazione, opzionale;
- `codeCoverage?: number` — percentuale di code coverage, opzionale;
- `cvss?: number` — punteggio di sicurezza CVSS, opzionale;
- `onRemove?: () => void` — callback opzionale invocata al click sul pulsante di rimozione;
- `isRemoving?: boolean` — indica se la rimozione è in corso, disabilitando il pulsante.

Il componente mostra il nome del repository, il suo proprietario, le tre metriche principali tramite barre di progresso colorate in base alla soglia raggiunta e la data dell'ultima scansione formattata.

ScoreBar

è il componente di presentazione che visualizza un punteggio numerico su scala da 0 a 10 tramite una barra di progresso colorata. Viene utilizzato dalla feature *Report* all'interno di `DocsSection` e `SecuritySection` per visualizzare i voti delle singole categorie di analisi. Riceve le seguenti props:

- `label: string` — etichetta descrittiva del punteggio;
- `value: number` — valore del punteggio su scala da 0 a 10.

Il colore della barra varia in base al valore: verde per punteggi superiori o uguali a 7, giallo per punteggi compresi tra 5 e 7, rosso per punteggi inferiori a 5.

4.1.4 Feature

Il frontend è organizzato in feature indipendenti, ciascuna corrispondente a un'area funzionale dell'applicazione. Ogni feature replica internamente la struttura a tre layer dell'architettura MV-VM, contenendo i propri componenti View, i propri hook ViewModel, i propri repository Model, le relative interfacce e i tipi di dominio.

4.1.4.1 Auth

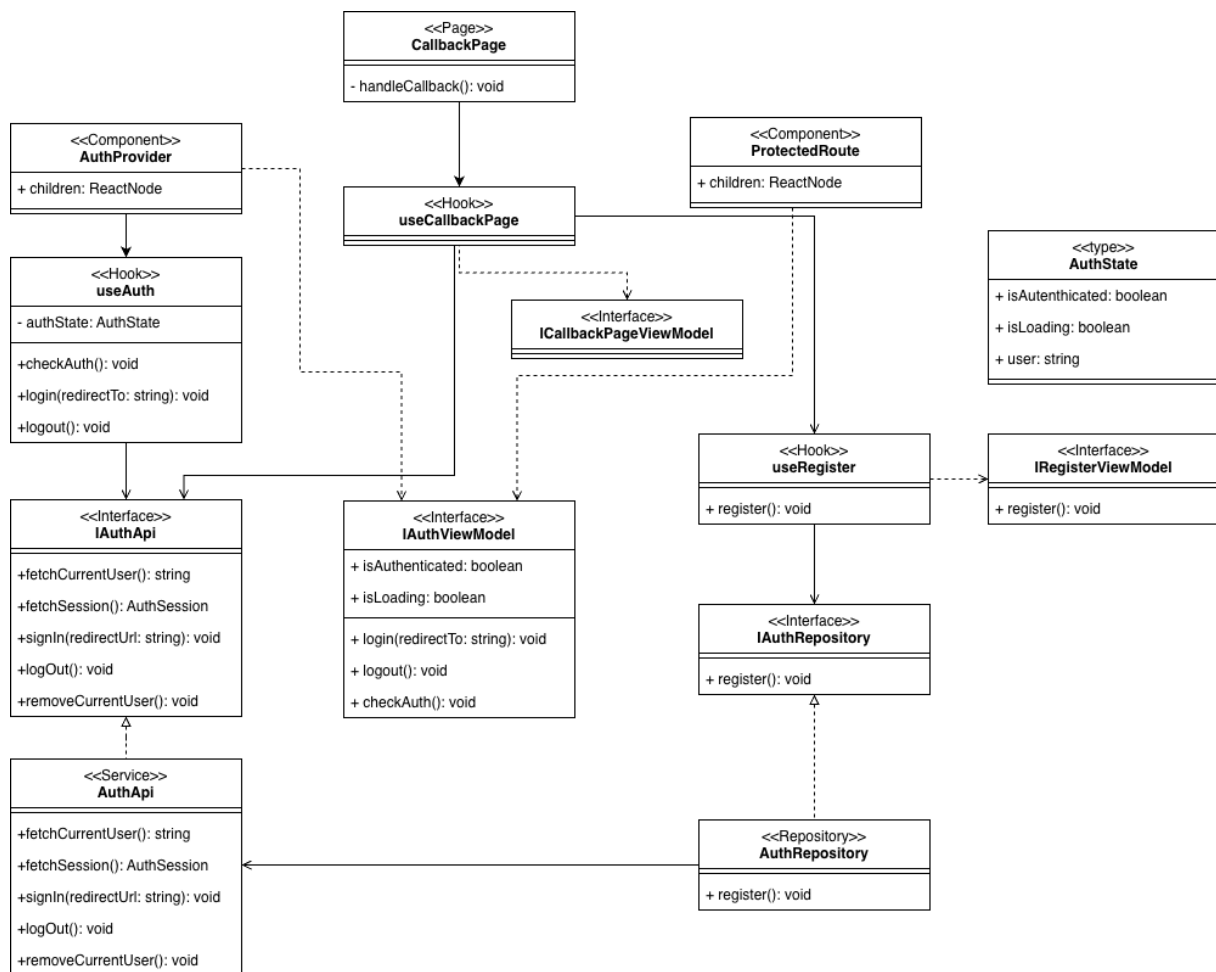


Figura 1: UML feature Auth

La feature *Auth* gestisce l'autenticazione degli utenti tramite Amazon Cognito con il flusso OAuth 2.0. Al primo accesso, l'utente viene reindirizzato alla pagina di login di Cognito; al ritorno, la *CallbackPage* verifica la sessione, registra l'utente nel database applicativo se necessario e lo reindirizza alla destinazione originale. Le route protette vengono sorvegliate dal componente *ProtectedRoute*, che reindirizza automaticamente a Cognito gli utenti non autenticati preservando il percorso richiesto. Lo stato di autenticazione è reso disponibile globalmente tramite il context *AuthContext*, popolato dall'*AuthProvider* e consumato dagli altri componenti tramite all'hook *useAuthContext*.

4.1.4.1.1 Tipi e DTO

AuthState è il tipo che modella lo stato di autenticazione dell'utente. Espone i seguenti campi:

- **isAuthenticated**: boolean — indica se l'utente è attualmente autenticato;

- `isLoading: boolean` — indica se la verifica della sessione è in corso;
- `user: { username: string } | null` — dati dell'utente autenticato, oppure `null` se non autenticato.

4.1.4.1.2 Componenti View

AuthProvider è il componente che inizializza e fornisce il context di autenticazione all'albero dei componenti figli. Riceve la prop `children: ReactNode`, istanzia internamente l'hook `useAuth` e ne passa il risultato come valore del `AuthContext.Provider`. Deve essere posizionato in cima all'albero dei componenti per rendere lo stato di autenticazione disponibile a tutti i discendenti.

ProtectedRoute è il componente che protegge le route che richiedono autenticazione. Riceve la prop `children: ReactNode` e consuma lo stato di autenticazione tramite `useAuthContext`. Se l'utente non è autenticato e il caricamento è terminato, invoca `login` passando il percorso corrente come `redirectTo`, in modo che dopo il login l'utente venga reindirizzato alla destinazione originale. Durante il caricamento mostra un messaggio testuale; se l'utente non è autenticato non renderizza nulla; se autenticato renderizza i componenti figli.

CallbackPage è il componente che gestisce il ritorno dal flusso OAuth di Cognito. Non riceve props e delega interamente la logica all'hook `useCallbackPage`. Mostra un messaggio testuale di attesa durante l'elaborazione del callback.

4.1.4.1.3 Hook

useAuth è il custom hook principale della feature, responsabile della gestione dello stato di autenticazione. Implementa l'interfaccia `IAuthViewModel` e gestisce internamente lo stato tramite `useState` e `useEffect`. Accetta opzionalmente un'implementazione di `IAuthApi` per facilitare il testing tramite dependency injection. Espone i seguenti membri:

- `isAuthenticated: boolean` — indica se l'utente è autenticato;
- `isLoading: boolean` — indica se la verifica della sessione è in corso;
- `user: { username: string } | null` — dati dell'utente autenticato;
- `login(redirectTo?: string): Promise<void>` — salva opzionalmente il percorso di destinazione in `sessionStorage` e avvia il flusso di login con redirect verso Cognito;
- `logout(): Promise<void>` — esegue il logout da Cognito e reindirizza l'utente alla pagina principale tramite `window.location.href`;
- `checkAuth(): Promise<void>` — verifica la sessione corrente invocando `fetchCurrentUser` e `fetchSession`; aggiorna lo stato di autenticazione in base al risultato.

useAuthContext è il custom hook che consente ai componenti di consumare il context di autenticazione. Invoca `useContext` su `AuthContext` e lancia un errore esplicito se utilizzato al di fuori di un `AuthProvider`, garantendo un utilizzo corretto del context.

useCallbackPage è il custom hook che gestisce la logica del callback OAuth. Implementa l'interfaccia `ICallbackPageViewModel` e orchestra il flusso post-login tramite `useEffect`. Al montaggio esegue in sequenza le seguenti operazioni: verifica che l'utente sia presente in Cognito tramite `fetchCurrentUser`; tenta la registrazione nel database applicativo tramite

`useRegister`; in caso di successo legge il percorso salvato in `sessionStorage` e naviga alla destinazione; in caso di fallimento della registrazione rimuove l'utente da Cognito tramite `removeCurrentUser` e naviga alla home con un parametro di errore. Accetta opzionalmente un'implementazione di `IAuthApi` per il testing.

useRegister è il custom hook responsabile della registrazione dell'utente nel database. Implementa l'interfaccia `IRegisterViewModel` e delega l'operazione all'implementazione di `IAuthRepository` ricevuta per dependency injection. Espone il membro:

- `register(): Promise<void>` — invoca il metodo `register` del repository per registrare l'utente autenticato nel backend.

IAuthViewModel è l'interfaccia che definisce il contratto esposto da `useAuth` verso il layer View. Estende `AuthState` aggiungendo i membri `login`, `logout` e `checkAuth`, con le stesse semantiche descritte per l'hook.

ICallbackPageViewModel è l'interfaccia che definisce il contratto esposto da `useCallbackPage`. Nel codice attuale è un'interfaccia vuota, in quanto il componente `CallbackPage` non necessita di dati esposti dall'hook.

IRegisterViewModel è l'interfaccia che definisce il contratto esposto da `useRegister`. Dichiarare il solo membro `register(): Promise<void>`.

4.1.4.1.4 Repository e Service

IAuthApi è l'interfaccia che definisce il contratto del service di autenticazione verso Amazon Cognito tramite AWS Amplify. Dichiarare i seguenti metodi:

- `fetchCurrentUser(): Promise<{ username: string }>` — recupera l'utente attualmente autenticato dalla sessione Amplify;
- `fetchSession(): Promise<AuthSession>` — recupera la sessione corrente, inclusi i token JWT;
- `signIn(redirectUrl?: string): Promise<void>` — salva opzionalmente il percorso di destinazione e avvia il flusso OAuth con redirect verso Cognito;
- `logout(): Promise<void>` — esegue il logout da Amplify e reindirizza l'utente all'endpoint di logout di Cognito;
- `removeCurrentUser(): Promise<void>` — elimina l'utente corrente da Cognito, utilizzato in caso di fallimento della registrazione nel database applicativo.

AuthApi è il service concreto che implementa `IAuthApi` wrappando le funzioni di AWS Amplify (`getCurrentUser`, `fetchAuthSession`, `signInWithRedirect`, `signOut`, `deleteUser`). Il metodo `logout` esegue prima il logout da Amplify, poi reindirizza all'endpoint di logout del dominio Cognito configurato per invalidare la sessione lato provider.

IAuthRepository è l'interfaccia che definisce il contratto per la registrazione dell'utente nel database applicativo. Dichiarare il metodo:

- `register(): Promise<void>` — registra l'utente autenticato nel backend applicativo tramite una richiesta HTTP autenticata con `idToken`.

AuthRepository è la classe concreta che implementa `IAuthRepository`. Recupera l'`idToken` dalla sessione Amplify corrente e invia una richiesta POST all'endpoint `/api/users`. Una risposta HTTP 409 viene trattata come caso non eccezionale, in quanto indica che l'utente è già registrato nel sistema.

4.1.4.2 Membership

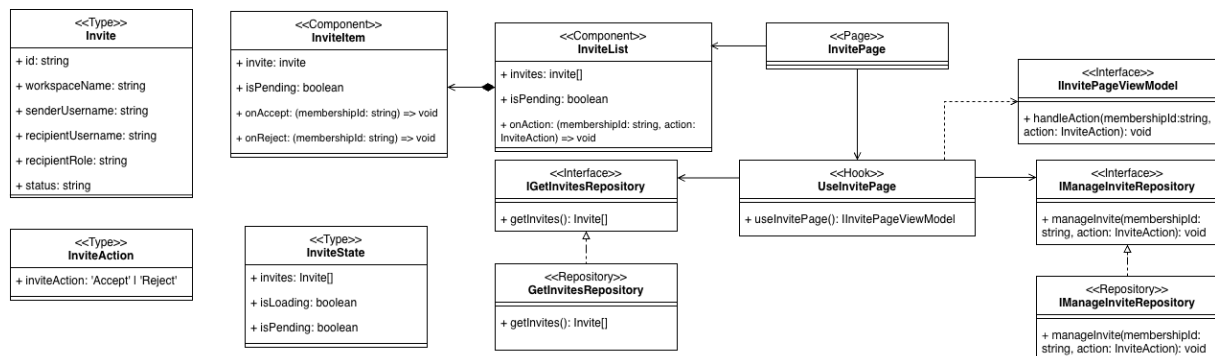


Figura 2: UML feature Membership

La feature *Membership* consente all'utente autenticato di visualizzare e gestire gli inviti ai workspace ricevuti da altri utenti. Dalla pagina dedicata, l'utente può accettare o rifiutare ciascun invito pendente. La lista degli inviti viene recuperata dal backend al caricamento della pagina e aggiornata automaticamente dopo ogni azione, garantendo la coerenza dei dati visualizzati.

4.1.4.2.1 Tipi e DTO

Invite è il tipo che modella un invito ricevuto dall'utente. Espone i seguenti campi:

- `id: string` — identificativo univoco dell'invito;
- `workspaceName: string` — nome del workspace al quale l'utente è stato invitato;
- `senderUsername: string` — username dell'utente che ha inviato l'invito;
- `recipientUsername: string` — username del destinatario dell'invito;
- `recipientRole: string` — ruolo assegnato al destinatario nel workspace;
- `status: string` — stato corrente dell'invito; può assumere i valori `PENDING`, `ACCEPTED` o `REJECTED`.

InviteAction è il tipo union che rappresenta le azioni che l'utente può compiere su un invito. Ammette i valori `'Accept'` e `'Reject'`.

InviteState è il tipo che raccoglie lo stato condiviso della pagina degli inviti. Espone i seguenti campi:

- `invites: Invite[]` — lista degli inviti pendenti dell'utente;
- `isLoading: boolean` — indica se il caricamento iniziale degli inviti è in corso;
- `isPending: boolean` — indica se è in corso una mutazione, ovvero un'operazione di accettazione o rifiuto di un invito.

4.1.4.2.2 Componenti View

InvitePage è il componente React che costituisce la pagina principale della feature. Recupera lo stato e i handler dall'hook `useInvitePage` e li passa al componente `InviteList`. Gestisce i casi di caricamento mostrando un messaggio testuale e, a caricamento completato, delega la resa della lista al componente figlio.

InviteList è il componente che si occupa di renderizzare la collezione di inviti. Riceve le seguenti props:

- `invites: Invite[]` — lista degli inviti da visualizzare;
- `isPending: boolean` — se `true`, disabilita i pulsanti di azione su tutti gli item per prevenire operazioni concorrenti;
- `onAction: (membershipId: string, action: InviteAction) => void` — callback invocata quando l'utente accetta o rifiuta un invito.

In assenza di inviti, renderizza un messaggio di stato vuoto. In presenza di inviti, renderizza un `InviteItem` per ciascuno, traducendo le callback `onAccept` e `onReject` nell'unica callback `onAction` con il valore di `InviteAction` appropriato.

InviteItem è il componente che rappresenta il singolo invito nella lista. Riceve le seguenti props:

- `invite: Invite` — i dati dell'invito da visualizzare;
- `isPending: boolean` — se `true`, disabilita i pulsanti di azione;
- `onAccept: (membershipId: string) => void` — callback invocata al click sul pulsante *Accetta*;
- `onReject: (membershipId: string) => void` — callback invocata al click sul pulsante *Rifiuta*.

Mostra il nome del workspace, le sue iniziali come avatar, lo username del mittente e il ruolo assegnato al destinatario. I pulsanti di azione vengono disabilitati quando `isPending` è `true`.

4.1.4.2.3 Hook

useInvitePage è il custom hook che costituisce il layer `ViewModel` della feature. Implementa l'interfaccia `IInvitePageViewModel` ed è responsabile del recupero degli inviti e della gestione delle azioni dell'utente. Internamente utilizza `useQuery` per il fetch degli inviti e `useMutation` per le operazioni di accettazione e rifiuto, invalidando la query `['invites']` al successo di ogni mutazione per mantenere la lista aggiornata. Espone i seguenti membri:

- `useInvitePage(): IInvitePageViewModel` — funzione costruttrice dell'hook, non richiede parametri in ingresso.

InvitePageViewModel è l'interfaccia che definisce il contratto esposto dall'hook verso il layer `View`. Estende `InviteState` aggiungendo il seguente membro:

- `handleAction(membershipId: string, action: InviteAction): void` — metodo invocato dalla `View` per avviare la mutazione di accettazione o rifiuto dell'invito identificato da `membershipId`.

4.1.4.2.4 Repository

IGetInvitesRepository è l'interfaccia che definisce il contratto per il recupero degli inviti dal backend. Dichiarare il metodo:

- `getInvites(): Promise<Invite[]>` — recupera la lista degli inviti pendenti dell'utente autenticato.

GetInvitesRepository è la classe concreta che implementa `IGetInvitesRepository`. Utilizza la funzione `apiGet` del client HTTP condiviso per eseguire una richiesta GET all'endpoint `/api/invitations`. Espone il metodo:

- `getInvites(): Promise<Invite[]>` — effettua la chiamata HTTP e restituisce la lista degli inviti deserializzata.

IManageInviteRepository è l'interfaccia che definisce il contratto per la gestione di un invito. Dichiarare il metodo:

- `manageInvite(membershipId: string, action: InviteAction): Promise<void>` — invia al backend l'azione scelta dall'utente per l'invito identificato da `membershipId`.

ManageInviteRepository è la classe concreta che implementa `IManageInviteRepository`. Utilizza la funzione `apiPatch` del client HTTP condiviso per eseguire una richiesta PATCH all'endpoint `/api/invitations/:membershipId`, passando come corpo un oggetto contenente il campo `action`. Espone il metodo:

- `manageInvite(membershipId: string, action: InviteAction): Promise<void>` — effettua la chiamata HTTP per aggiornare lo stato dell'invito.

4.1.4.3 CreateWorkspace

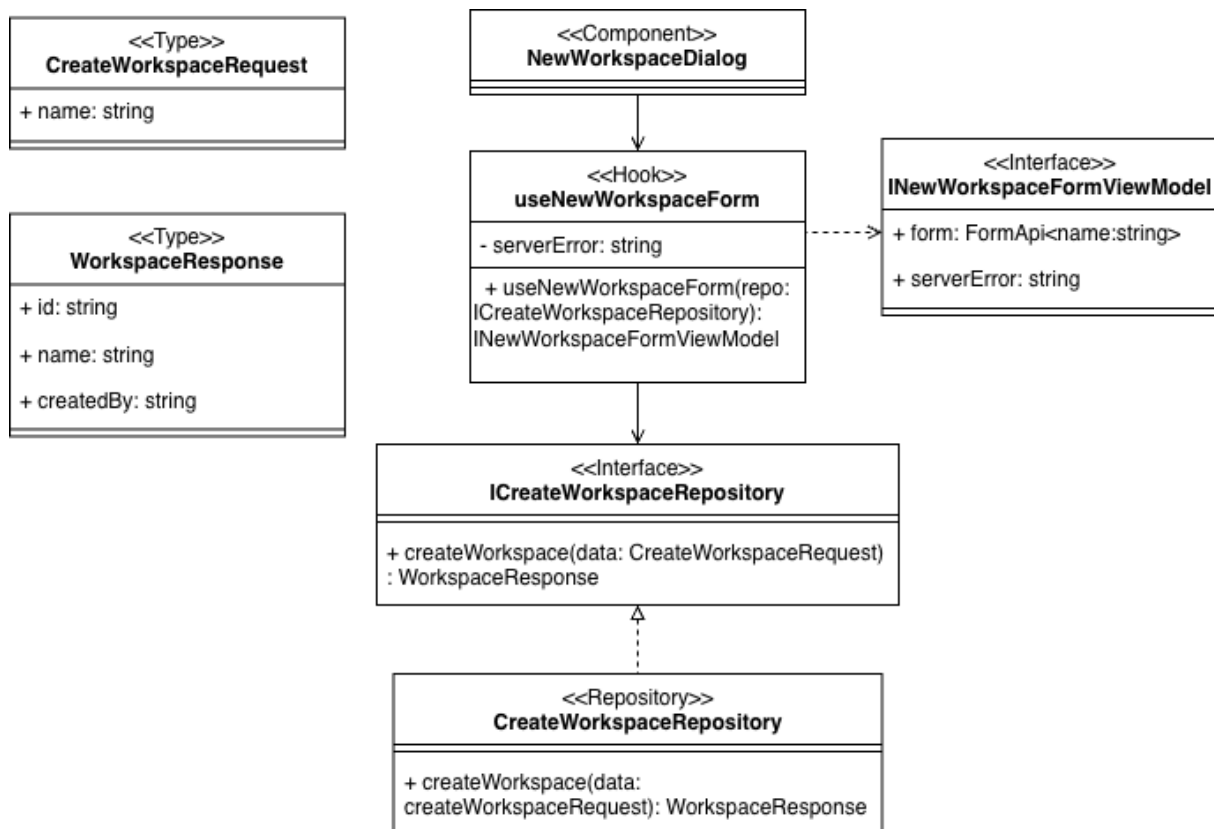


Figura 3: UML feature create Workspace

La feature *CreateWorkspace* consente all'utente autenticato di creare un nuovo workspace tramite un dialog modale. L'utente inserisce il nome del workspace, il quale viene validato lato client prima di essere inviato al backend. In caso di errore restituito dal server, questo viene mostrato direttamente nel form. Al successo della creazione, l'utente viene reindirizzato automaticamente alla pagina dei repository del workspace appena creato.

4.1.4.3.1 Tipi e DTO

CreateWorkspaceRequest è il tipo che rappresenta il corpo della richiesta di creazione workspace inviata al backend. Espone il campo `name: string`, contenente il nome scelto dall'utente. Le informazioni sull'owner vengono ricavate autonomamente dal backend a partire dal token JWT presente nella richiesta, senza che il client debba fornirle esplicitamente.

WorkspaceResponse è il tipo che modella la risposta del backend a seguito della creazione. Espone i seguenti campi:

- `id: string` — identificativo univoco del workspace appena creato;
- `name: string` — nome del workspace;
- `createdBy: string` — username dell'utente che ha creato il workspace.

4.1.4.3.2 Componenti View

NewWorkspaceDialog è il componente React che realizza il layer di presentazione della feature. Renderizza un pulsante *+ New Workspace* che, al click, apre un dialog modale contenente un form per l'inserimento del nome del workspace. Il componente delega interamente la logica di stato e di submission all'hook `useNewWorkspaceForm`, dal quale riceve l'istanza `form` e l'eventuale `serverError`. Il campo nome è soggetto a validazione: l'errore di validazione viene mostrato solo dopo che il campo è stato toccato (*touched*). In caso di errore restituito dal server, questo viene visualizzato sotto il campo. I pulsanti *Cancel* e *Create Workspace* consentono rispettivamente di annullare o confermare l'operazione.

4.1.4.3.3 Hook

useNewWorkspaceForm è il custom hook che costituisce il layer ViewModel della feature. Implementa l'interfaccia `INewWorkspaceFormViewModel` ed è responsabile della gestione dello stato del form e della logica di submission. Internamente utilizza `useForm` di TanStack Form con validazione tramite schema Zod (`newWorkspaceSchema`), che impone che il nome del workspace abbia una lunghezza compresa tra 2 e 30 caratteri. Espone i seguenti membri:

- `serverError: string` — messaggio di errore restituito dal backend in caso di fallimento della creazione; è null in assenza di errori;
- `useNewWorkspaceForm(repo: ICreateWorkspaceRepository): INewWorkspaceFormViewModel` — funzione costruttrice dell'hook; accetta opzionalmente un'implementazione del repository per facilitare il testing tramite dependency injection; al successo della creazione invalida la cache TanStack Query e naviga alla pagina dei repository del nuovo workspace.

INewWorkspaceFormViewModel è l'interfaccia che definisce il contratto esposto dall'hook verso il layer View. Dichiara i seguenti membri:

- `form: FormApi<{name: string}>` — istanza del form gestita da TanStack Form, contenente stato, validazione e handler degli eventi;
- `serverError: string` — eventuale messaggio di errore proveniente dal backend.

4.1.4.3.4 Repository

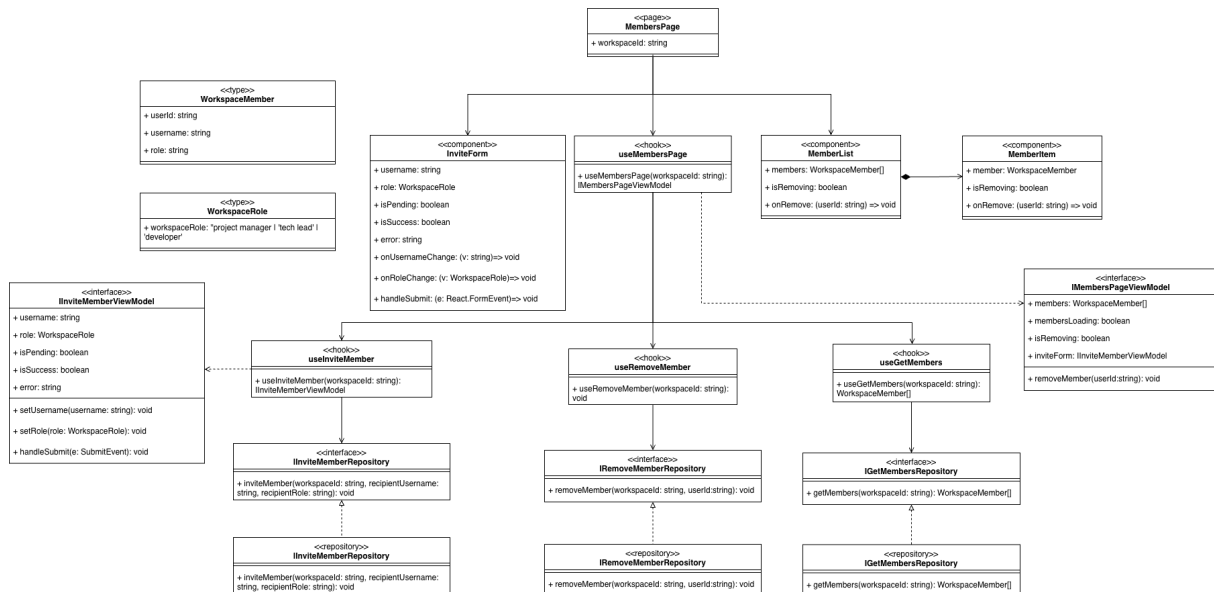
ICreateWorkspaceRepository è l'interfaccia che definisce il contratto del layer di accesso ai dati per la creazione del workspace. Dichiara il metodo:

- `createWorkspace(data: CreateWorkspaceRequest): Promise<WorkspaceResponse>` — invia la richiesta di creazione workspace al backend e restituisce i dati del workspace creato.

CreateWorkspaceRepository è la classe concreta che implementa `ICreateWorkspaceRepository`. Utilizza la funzione `apiPost` del client HTTP condiviso per eseguire una richiesta POST all'endpoint `/api/workspaces/`, passando come corpo il DTO `CreateWorkspaceRequest`. Espone il metodo:

- `createWorkspace(data: CreateWorkspaceRequest): Promise<WorkspaceResponse>` — effettua la chiamata HTTP e restituisce la risposta deserializzata come `WorkspaceResponse`.

4.1.4.4 WorkspaceMembers



La feature *WorkspaceMembers* consente all'utente autenticato di gestire i membri di un workspace. Dalla pagina dedicata è possibile visualizzare la lista dei membri correnti, invitare nuovi utenti specificandone username e ruolo, e rimuovere membri esistenti. Tutte le operazioni di modifica invalidano automaticamente la cache della lista membri per garantire la coerenza dei dati visualizzati.

4.1.4.4.1 Tipi e DTO

WorkspaceMember è il tipo che modella un membro del workspace. Espone i seguenti campi:

- `userId: string` — identificativo univoco dell'utente;
- `username: string` — nome utente visualizzato;
- `role: string` — ruolo del membro all'interno del workspace.

WorkspaceRole è il tipo union che rappresenta i ruoli assegnabili a un membro del workspace. Ammette i valori `'project manager'`, `'tech lead'` e `'developer'`.

4.1.4.4.2 Componenti View

MembersPage è il componente che costituisce la pagina principale della feature. Riceve la prop `workspaceId: string` e recupera tutto lo stato necessario dall'hook aggregatore `useMembersPage`. Compose al suo interno i componenti `InviteForm` e `MemberList`, gestendo il caso di caricamento della lista con un messaggio testuale.

InviteForm è il componente che renderizza il form di invito di un nuovo membro. Riceve le seguenti props:

- `username: string` — valore corrente del campo username;
- `role: WorkspaceRole` — ruolo selezionato per il nuovo membro;
- `isPending: boolean` — se `true`, disabilita il pulsante di invio durante la mutazione;

- `isSuccess: boolean` — se `true`, mostra un messaggio di conferma invio;
- `error: string` — eventuale messaggio di errore restituito dal backend;
- `onUsernameChange: (v: string) => void` — callback invocata al cambio del campo `username`;
- `onRoleChange: (v: WorkspaceRole) => void` — callback invocata al cambio del ruolo selezionato;
- `onSubmit: (e: React.FormEvent) => void` — callback invocata alla submission del form.

Il pulsante di invio è disabilitato anche quando il campo `username` è vuoto.

MemberList è il componente che renderizza la lista dei membri del workspace. Riceve le seguenti props:

- `members: WorkspaceMember[]` — lista dei membri da visualizzare;
- `isRemoving: boolean` — se `true`, disabilita i pulsanti di rimozione su tutti gli item per prevenire operazioni concorrenti;
- `onRemove: (userId: string) => void` — callback invocata quando l'utente richiede la rimozione di un membro.

In assenza di membri mostra un messaggio di stato vuoto. In presenza di membri renderizza un `MemberItem` per ciascuno.

MemberItem è il componente che rappresenta il singolo membro nella lista. Riceve le seguenti props:

- `member: WorkspaceMember` — i dati del membro da visualizzare;
- `isRemoving: boolean` — se `true`, disabilita il pulsante di rimozione;
- `onRemove: (userId: string) => void` — callback invocata al click sul pulsante *Rimuovi*.

Mostra le iniziali dello `username` come avatar, il nome utente e il ruolo del membro.

4.1.4.4.3 Hook

useMembersPage è il custom hook aggregatore che costituisce il layer `ViewModel` della feature. Implementa l'interfaccia `IMembersPageViewModel` componendo internamente i tre hook specializzati `useGetMembers`, `useRemoveMember` e `useInviteMember`. Accetta il parametro `workspaceId: string` ed espone i seguenti membri:

- `members: WorkspaceMember[]` — lista dei membri correnti del workspace;
- `membersLoading: boolean` — indica se il caricamento della lista è in corso;
- `isRemoving: boolean` — indica se una rimozione è in corso;
- `removeMember(userId: string): void` — avvia la mutazione di rimozione del membro identificato da `userId`;
- `inviteForm: IInviteMemberViewModel` — oggetto che espone lo stato e i handler del form di invito.

useGetMembers è il custom hook responsabile del recupero della lista dei membri. Utilizza `useQuery` di TanStack Query con query key `['members', workspaceId]`, abilitata solo se `workspaceId` è definito. Accetta il parametro `workspaceId: string` e restituisce i dati della query inclusi `data` e `isLoading`.

useInviteMember è il custom hook responsabile della logica di invito di un nuovo membro. Implementa l'interfaccia `IInviteMemberViewModel` e utilizza `useMutation` per invocare `InviteMemberRepository.inviteMember`. Al successo della mutazione invalida la query `['members', workspaceId]` e reimposta i campi del form. Espone i seguenti membri:

- `username: string` — valore corrente del campo username;
- `setUsername(username: string): void` — aggiorna il valore del campo username;
- `role: WorkspaceRole` — ruolo correntemente selezionato;
- `setRole(role: WorkspaceRole): void` — aggiorna il ruolo selezionato;
- `isPending: boolean` — indica se la mutazione di invito è in corso;
- `isSuccess: boolean` — indica se l'invito è stato inviato con successo;
- `error: Error` — eventuale errore restituito dal backend;
- `handleSubmit(e: SubmitEvent): void` — gestisce la submission del form, prevenendo l'invio se lo username è vuoto.

useRemoveMember è il custom hook responsabile della rimozione di un membro. Utilizza `useMutation` per invocare `RemoveMemberRepository.removeMember` e al successo invalida la query `['members', workspaceId]`. Accetta il parametro `workspaceId: string` e restituisce direttamente il risultato di `useMutation`, esponendo tra gli altri `mutate` e `isPending`.

IInviteMemberViewModel è l'interfaccia che definisce il contratto esposto da `useInviteMember` verso il layer View. Dichiarare i membri `username`, `setUsername`, `role`, `setRole`, `isPending`, `isSuccess`, `error` e `handleSubmit`, con le stesse semantiche descritte per l'hook.

IMembersPageViewModel è l'interfaccia che definisce il contratto esposto da `useMembersPage` verso il layer View. Dichiarare i membri `members`, `membersLoading`, `isRemoving`, `removeMember` e `inviteForm`, con le stesse semantiche descritte per l'hook.

4.1.4.4 Repository

IGetMembersRepository è l'interfaccia che definisce il contratto per il recupero dei membri. Dichiarare il metodo:

- `getMembers(workspaceId: string): Promise<WorkspaceMember[]>` — recupera la lista dei membri del workspace identificato da `workspaceId`.

GetMembersRepository è la classe concreta che implementa `IGetMembersRepository`. Utilizza `apiGet` per eseguire una richiesta GET all'endpoint `/api/workspaces/:workspaceId/users`. Espone il metodo:

- `getMembers(workspaceId: string): Promise<WorkspaceMember[]>` — effettua la chiamata HTTP e restituisce la lista dei membri deserializzata.

IInviteMemberRepository è l'interfaccia che definisce il contratto per l'invito di un nuovo membro. Dichiarare il metodo:

- `inviteMember(workspaceId: string, recipientUsername: string, recipientRole: WorkspaceRole): Promise<void>` — invia al backend la richiesta di invito per l'utente specificato con il ruolo indicato.

InviteMemberRepository è la classe concreta che implementa `IInviteMemberRepository`. Utilizza `apiPost` per eseguire una richiesta POST all'endpoint `/api/invitations`, passando come corpo un oggetto contenente `workspaceId`, `recipientUsername` e `recipientRole`. Espone il metodo:

- `inviteMember(workspaceId: string, recipientUsername: string, recipientRole: WorkspaceRole): Promise<void>` — effettua la chiamata HTTP di invito.

IRemoveMemberRepository è l'interfaccia che definisce il contratto per la rimozione di un membro. Dichiarare il metodo:

- `removeMember(workspaceId: string, userId: string): Promise<void>` — invia al backend la richiesta di rimozione del membro identificato da `userId` dal workspace identificato da `workspaceId`.

RemoveMemberRepository è la classe concreta che implementa `IRemoveMemberRepository`. Utilizza `apiDelete` per eseguire una richiesta DELETE all'endpoint `/api/workspaces/:workspaceId/users/:userId`. Espone il metodo:

- `removeMember(workspaceId: string, userId: string): Promise<void>` — effettua la chiamata HTTP di rimozione.

4.1.4.5 WorkspaceRepository

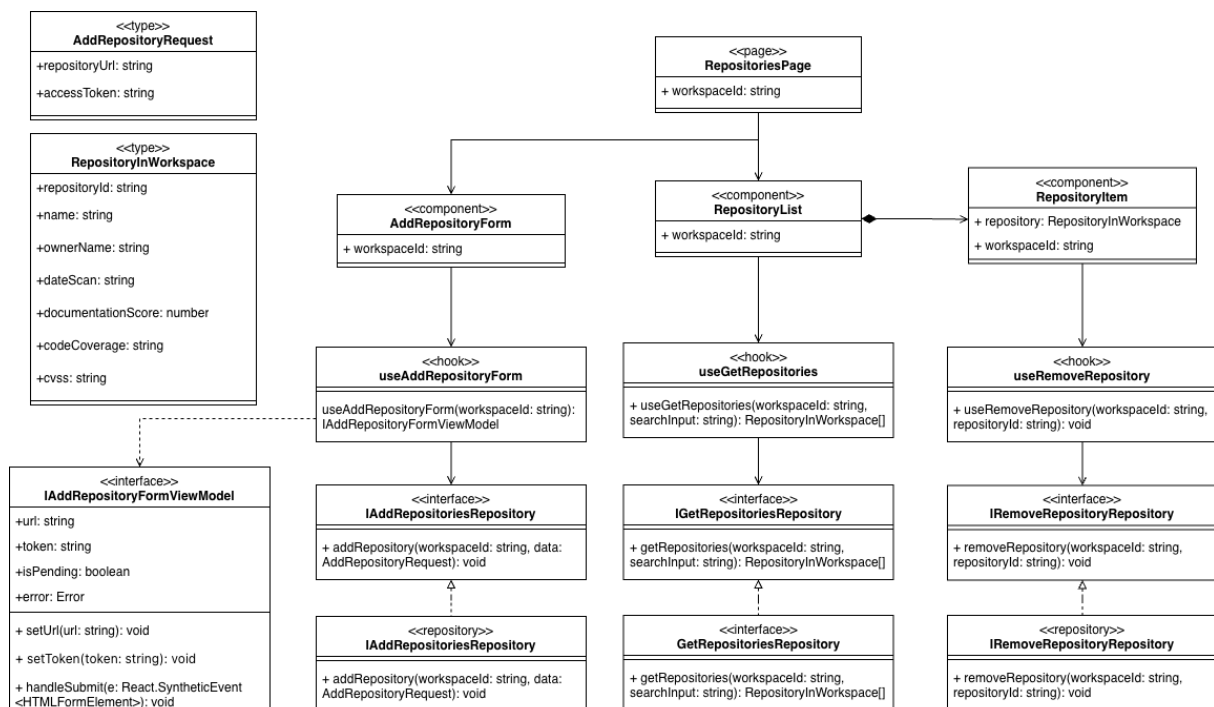


Figura 4: UML feature workspaceRepository

La feature *WorkspaceRepository* consente all'utente autenticato di gestire i repository GitHub associati a un workspace. Dalla pagina dedicata è possibile visualizzare la lista dei repository collegati, aggiungerne di nuovi tramite URL con token di accesso opzionale per repository privati, rimuovere repository esistenti e aggiornare il token di accesso di un repository già collegato. La lista supporta inoltre la ricerca per nome tramite un campo di testo con debounce implicito sulla query key di TanStack Query.

4.1.4.5.1 Tipi e DTO

RepositoryInWorkspace è il tipo che modella un repository collegato a un workspace. Espone i seguenti campi:

- `repositoryId`: `string` — identificativo univoco del repository;
- `name`: `string` — nome del repository;
- `ownerName`: `string` — username del proprietario del repository su GitHub;
- `dateScan?`: `string` — data dell'ultima scansione eseguita, opzionale;
- `documentationScore?`: `number` — punteggio della documentazione dell'ultima scansione, opzionale;
- `codeCoverage?`: `number` — percentuale di code coverage dell'ultima scansione, opzionale;
- `cvss?`: `number` — punteggio di sicurezza CVSS dell'ultima scansione, opzionale.

AddRepositoryRequest è il tipo che rappresenta il corpo della richiesta di aggiunta di un repository. Espone i seguenti campi:

- `repositoryUrl`: `string` — URL del repository GitHub da collegare;
- `accessToken?`: `string` — token di accesso GitHub opzionale, necessario per repository privati.

4.1.4.5.2 Componenti View

RepositoriesPage è il componente che costituisce la pagina principale della feature. Riceve la prop `workspaceId`: `string`, è protetta da `ProtectedRoute` e contiene il componente `RepositoryList`.

RepositoryList è il componente che si occupa di renderizzare la lista dei repository del workspace. Riceve la prop `workspaceId`: `string` e gestisce internamente lo stato del campo di ricerca tramite `useState`. Recupera i dati dall'hook `useGetRepositories` passando il termine di ricerca corrente come parametro opzionale. Compone al suo interno il componente `AddRepositoryForm` per l'aggiunta di nuovi repository e renderizza un `RepositoryItem` per ciascun repository ricevuto. Gestisce i casi di caricamento, errore e lista vuota con messaggi testuali dedicati.

AddRepositoryForm è il componente che renderizza il form di aggiunta di un nuovo repository. Riceve la prop `workspaceId`: `string` e delega tutta la logica all'hook `useAddRepositoryForm`. Espone un campo per l'URL del repository e un campo opzionale per il token di accesso GitHub. Il pulsante di invio è disabilitato durante la mutazione e gli eventuali errori del backend vengono mostrati sotto il form.

RepositoryItem è il componente che rappresenta il singolo repository nella lista. Riceve le seguenti props:

- `repository`: `RepositoryInWorkspace` — i dati del repository da visualizzare;
- `workspaceId`: `string` — identificativo del workspace, necessario per la navigazione e le operazioni di rimozione e aggiornamento token.

Renderizza internamente il componente `RepositoryCard` con i dati del repository e sovrappone un pulsante *Rimuovi* in posizione assoluta. Avvolge la card in un `Link` che naviga alla pagina del report del repository. Utilizza l'hook `useRemoveRepository` per gestire la rimozione.

4.1.4.5.3 Hook

useAddRepositoryForm è il custom hook responsabile della logica del form di aggiunta repository. Implementa l'interfaccia `IAddRepositoryFormViewModel` e utilizza `useMutation` per invocare `AddRepositoryRepository.addRepository`. Al successo invalida la query `['repositories', workspaceId]` e reimposta i campi del form. Espone i seguenti membri:

- `url`: `string` — valore corrente del campo URL;
- `setUrl(url: string): void` — aggiorna il valore del campo URL;
- `token`: `string` — valore corrente del campo token di accesso;
- `setToken(token: string): void` — aggiorna il valore del campo token;
- `isPending`: `boolean` — indica se la mutazione di aggiunta è in corso;
- `error`: `Error` — eventuale errore restituito dal backend;
- `handleSubmit(e: React.SyntheticEvent<HTMLFormElement>): void` — gestisce la submission del form, prevenendo l'invio se il campo URL è vuoto.

useGetRepositories è il custom hook responsabile del recupero della lista di repository. Utilizza `useQuery` con query key `['repositories', workspaceId, searchInput]`, abilitata solo se `workspaceId` è definito. Accetta i seguenti parametri:

- `workspaceId`: `string` — identificativo del workspace;
- `searchInput?: string` — termine di ricerca opzionale per filtrare i repository per nome.

Restituisce i dati della query inclusi `data`, `isLoading` e `error`.

useRemoveRepository è il custom hook responsabile della rimozione di un repository. Utilizza `useMutation` per invocare `RemoveRepositoryRepository.removeRepository` e al successo invalida la query `['repositories', workspaceId]`. Accetta il parametro `workspaceId: string` e restituisce direttamente il risultato di `useMutation`, esponendo tra gli altri `mutate` e `isPending`.

IAddRepositoryFormViewModel è l'interfaccia che definisce il contratto esposto da `useAddRepositoryForm` verso il layer View. Dichiarare i membri `url`, `setUrl`, `token`, `setToken`, `isPending`, `error` e `handleSubmit`, con le stesse semantiche descritte per l'hook.

4.1.4.5.4 Repository

IAddRepositoriesRepository è l'interfaccia che definisce il contratto per l'aggiunta di un repository. Dichiarare il metodo:

- `addRepository(workspaceId: string, data: AddRepositoryRequest): Promise<void>`
— invia al backend la richiesta di collegamento del repository specificato al workspace.

AddRepositoryRepository è la classe concreta che implementa `IAddRepositoriesRepository`.

Utilizza `apiPost` per eseguire una richiesta POST all'endpoint `/api/workspaces/:workspaceId/repositories`.

Espone il metodo:

- `addRepository(workspaceId: string, data: AddRepositoryRequest): Promise<void>`
— effettua la chiamata HTTP di aggiunta repository.

IGetRepositoriesRepository è l'interfaccia che definisce il contratto per il recupero della lista di repository. Dichiarare il metodo:

- `getRepositories(workspaceId: string, searchInput?: string): Promise<RepositoryInWorkspace>`
— recupera la lista dei repository del workspace, con filtraggio opzionale per nome.

GetRepositoriesRepository è la classe concreta che implementa `IGetRepositoriesRepository`.

Utilizza `apiGet` per eseguire una richiesta GET all'endpoint `/api/workspaces/:workspaceId/repositories`, aggiungendo il parametro query `searchInput` codificato tramite `encodeURIComponent` se fornito.

Espone il metodo:

- `getRepositories(workspaceId: string, searchInput?: string): Promise<RepositoryInWorkspace>`
— effettua la chiamata HTTP e restituisce la lista dei repository deserializzata.

IRemoveRepositoryRepository è l'interfaccia che definisce il contratto per la rimozione di un repository. Dichiarare il metodo:

- `removeRepository(workspaceId: string, repositoryId: string): Promise<void>`
— invia al backend la richiesta di rimozione del repository dal workspace.

RemoveRepositoryRepository è la classe concreta che implementa `IRemoveRepositoryRepository`.

Utilizza `apiDelete` per eseguire una richiesta DELETE all'endpoint `/api/workspaces/:workspaceId/repositories/:repositoryId`.

Espone il metodo:

- `removeRepository(workspaceId: string, repositoryId: string): Promise<void>`
— effettua la chiamata HTTP di rimozione.

4.1.4.6 WorkspaceList

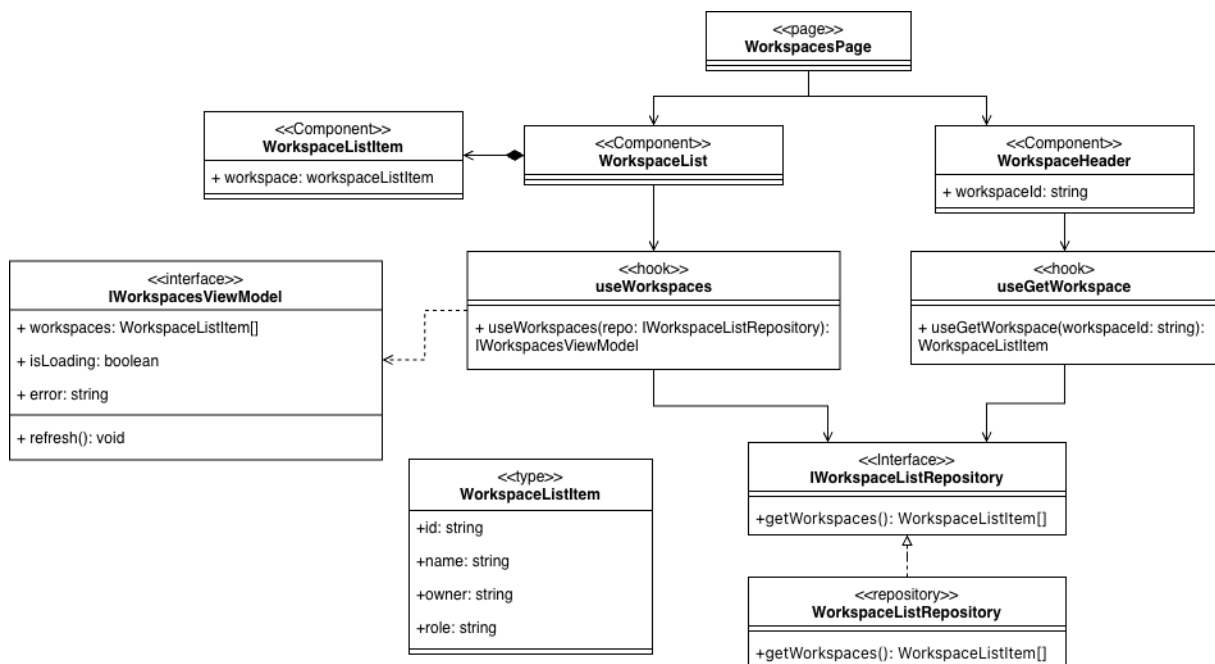


Figura 5: UML feature WorkspaceList

La feature *WorkspaceList* consente all'utente autenticato di visualizzare la lista dei workspace a cui appartiene e di accedere al dettaglio di ciascuno. La pagina principale mostra l'elenco dei workspace recuperati dal backend, con gestione esplicita degli stati di caricamento e di errore. È presente inoltre un componente header riutilizzabile che mostra le informazioni di riepilogo del workspace corrente, tra cui nome, ruolo dell'utente, numero di membri e numero di repository.

4.1.4.6.1 Tipi e DTO

WorkspaceListItem è il tipo che modella un workspace nella lista. Espone i seguenti campi:

- `id: string` — identificativo univoco del workspace;
- `name: string` — nome del workspace;
- `owner: string` — username del proprietario del workspace;
- `role: string` — ruolo dell'utente autenticato all'interno del workspace.

4.1.4.6.2 Componenti View

WorkspacesPage è il componente che costituisce la pagina principale della feature. È protetta da `ProtectedRoute` e contiene i componenti `WorkspaceList` e `WorkspaceHeader`. Fornisce il layout generale della pagina, includendo il titolo e la descrizione introduttiva.

WorkspaceList è il componente che si occupa di renderizzare la collezione di workspace dell'utente. Recupera lo stato dall'hook `useWorkspaces` e gestisce i seguenti casi: caricamento in corso, errore nella chiamata API, lista vuota e lista popolata. In quest'ultimo caso renderizza un `WorkspaceListItem` per ciascun workspace ricevuto.

WorkspaceListItem è il componente che rappresenta il singolo workspace nella lista. Riceve la seguente prop:

- `workspace`: `WorkspaceListItem` — i dati del workspace da visualizzare.

Mostra il nome del workspace, il suo owner, il ruolo dell'utente e la prima lettera del nome come avatar. È interamente avvolto in un `Link` di TanStack Router che naviga alla pagina dei repository del workspace selezionato.

WorkspaceHeader è il componente riutilizzabile che mostra le informazioni di riepilogo del workspace corrente, posizionato in cima alle pagine interne al workspace. Riceve la seguente prop:

- `workspaceId`: `string` — identificativo del workspace di cui mostrare le informazioni.

Internamente utilizza `useGetWorkspace` per recuperare i dati del workspace, `useGetMembers` per il conteggio dei membri e `useGetRepositories` per il conteggio dei repository. Mostra il nome del workspace, le sue iniziali come avatar, il badge del ruolo dell'utente con colore differenziato per ruolo, e le statistiche di membri e repository quando disponibili. Mostra il pulsante di eliminazione workspace solo se l'utente autenticato è il proprietario. Durante il caricamento mostra uno skeleton animato; se il workspace non viene trovato non renderizza nulla.

4.1.4.6.3 Hook

useWorkspaces è il custom hook responsabile del recupero della lista di workspace. Implementa l'interfaccia `IWorkspacesViewModel` e gestisce internamente lo stato tramite `useState` e `useEffect`, senza ricorrere a TanStack Query. Accetta opzionalmente un'implementazione di `IWorkspaceListRepository` per facilitare il testing tramite dependency injection. Espone i seguenti membri:

- `workspaces`: `WorkspaceListItem[]` — lista dei workspace dell'utente; è un array vuoto durante il caricamento o in caso di errore;
- `isLoading`: `boolean` — indica se il caricamento è in corso;
- `error`: `string` — messaggio di errore in caso di fallimento della chiamata API; è `null` in assenza di errori;
- `refresh()`: `void` — forza un nuovo fetch della lista dei workspace, reimpostando `isLoading` a `true`.

useGetWorkspace è il custom hook responsabile del recupero di un singolo workspace tramite il suo identificativo. Utilizza internamente `useQuery` di TanStack Query con la stessa query key `['workspaces']` di `useWorkspaces`, sfruttando l'opzione `select` per filtrare il workspace corrispondente dalla lista già in cache, evitando così una chiamata HTTP aggiuntiva. Accetta il seguente parametro:

- `workspaceId`: `string` — identificativo del workspace da recuperare.

Restituisce i dati del workspace corrispondente oppure `undefined` se non trovato, insieme allo stato di caricamento della query.

IWorkspacesViewModel è l'interfaccia che definisce il contratto esposto da useWorkspaces verso il layer View. Dichiarare i membri workspaces, isLoading, error e refresh, con le stesse semantiche descritte per l'hook.

4.1.4.6.4 Repository

IWorkspaceListRepository è l'interfaccia che definisce il contratto del layer di accesso ai dati per il recupero dei workspace. Dichiarare il metodo:

- `getWorkspaces(): Promise<WorkspaceListItem[]>` — recupera la lista dei workspace a cui appartiene l'utente autenticato.

WorkspaceListRepository è la classe concreta che implementa **IWorkspaceListRepository**. Utilizza la funzione `apiGet` del client HTTP condiviso per eseguire una richiesta GET all'endpoint `/api/workspaces`. Espone il metodo:

- `getWorkspaces(): Promise<WorkspaceListItem[]>` — effettua la chiamata HTTP e restituisce la lista dei workspace deserializzata.

4.1.4.7 Scan

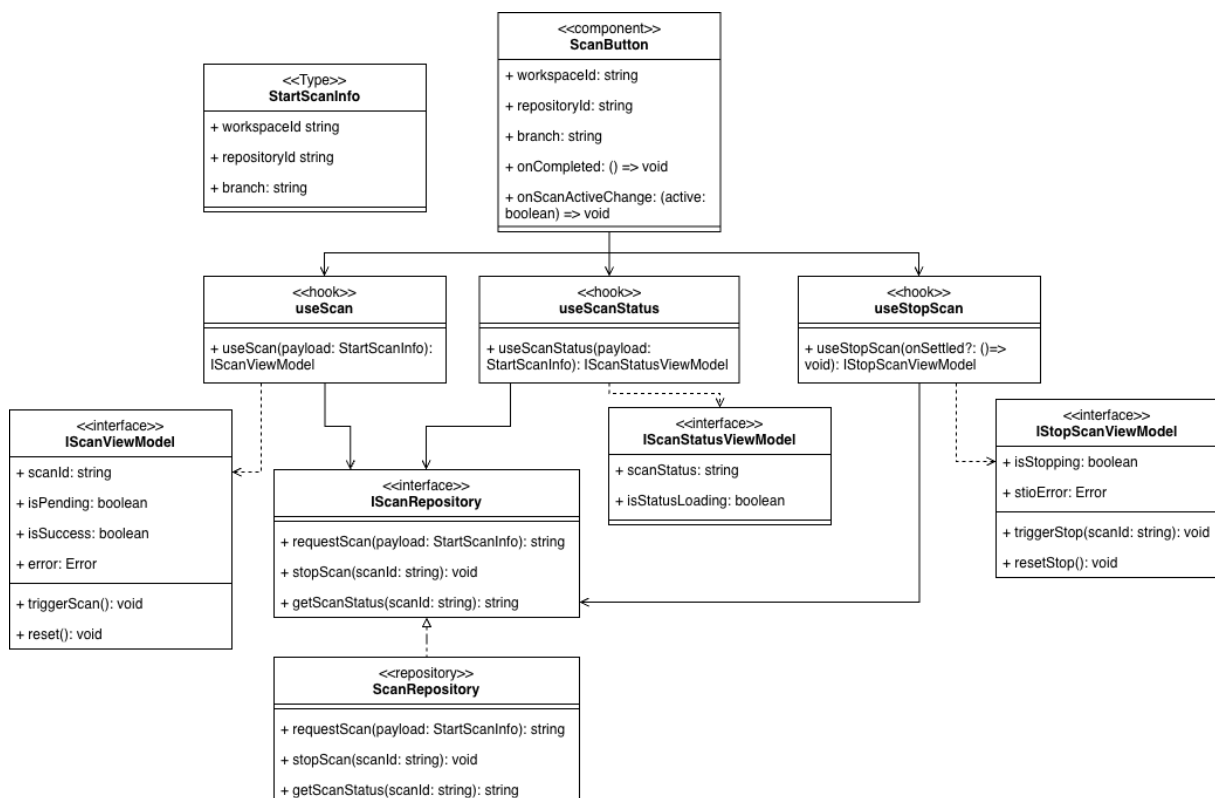


Figura 6: UML feature Scan

La feature *Scan* consente all'utente di avviare e interrompere una scansione su un repository GitHub per un determinato branch. Il componente dedicato mostra un pulsante di avvio che,

una volta premuto, dà inizio alla scansione e si trasforma in un pulsante di interruzione. Lo stato della scansione viene monitorato tramite polling periodico al backend. Al raggiungimento di uno stato terminale, la cache del report viene invalidata automaticamente per aggiornare i risultati visualizzati. È presente inoltre un meccanismo di fallback che, trascorso un timeout, interroga direttamente il report per verificare il completamento della scansione in caso di mancata ricezione dello stato terminale.

4.1.4.7.1 Tipi e DTO

StartScanInfo è il tipo che raccoglie le informazioni necessarie per avviare una scansione. Espone i seguenti campi:

- `workspaceId: string` — identificativo del workspace a cui appartiene il repository;
- `repositoryId: string` — identificativo del repository da scansionare;
- `branch: string` — nome del branch su cui eseguire la scansione.

4.1.4.7.2 Componenti View

ScanButton è il componente React che costituisce l'unico elemento di presentazione della feature. Viene utilizzato all'interno della `ReportPage` e coordina internamente i tre hook della feature. Riceve le seguenti props:

- `workspaceId: string` — identificativo del workspace;
- `repositoryId: string` — identificativo del repository;
- `branch: string` — branch selezionato; se vuoto, il pulsante di avvio viene disabilitato;
- `onCompleted: () => void` — callback opzionale invocata al completamento con successo della scansione, utilizzata dalla `ReportPage` per invalidare la cache del report;
- `onScanActiveChange: (active: boolean) => void` — callback opzionale invocata ogni volta che lo stato attivo della scansione cambia, utilizzata per sopprimere lo skeleton di caricamento del report durante la scansione.

Il componente distingue due stati visivi principali: quando nessuna scansione è attiva mostra il pulsante *Lancia scansione*; quando una scansione è in corso mostra il pulsante *Ferma scansione*. Gli eventuali errori di avvio o di interruzione vengono mostrati sotto il pulsante. Internamente gestisce un meccanismo di fallback basato su `setTimeout` e polling del report tramite `useQueryClient` per garantire il completamento anche in assenza di aggiornamenti di stato dal backend entro 90 secondi.

4.1.4.7.3 Hook

useScan è il custom hook responsabile dell'avvio della scansione. Implementa l'interfaccia `IScanViewModel` e utilizza internamente `useMutation` di TanStack Query per invocare `ScanRepository.request`. Accetta in ingresso un payload di tipo `StartScanInfo` e restituisce i seguenti membri:

- `triggerScan(): void` — avvia la mutazione di richiesta scansione;
- `scanId: string` — identificativo della scansione restituito dal backend al successo della mutazione; è `undefined` prima del successo;

- `isPending`: `boolean` — indica se la richiesta di avvio è in corso;
- `isSuccess`: `boolean` — indica se la scansione è stata avviata con successo;
- `error`: `Error` — eventuale errore restituito dal backend;
- `reset()`: `void` — reimposta lo stato della mutazione al valore iniziale.

useScanStatus è il custom hook responsabile del monitoraggio dello stato di una scansione in corso. Implementa l'interfaccia `IScanStatusViewModel` e utilizza internamente `useQuery` con `refetch` automatico ogni 3 secondi, che si interrompe automaticamente al raggiungimento di uno stato terminale (`completed`, `stopped`, `error`). Accetta in ingresso il `scanId` e la query è abilitata solo se questo è definito. Espone i seguenti membri:

- `scanStatus`: `string` — stato corrente della scansione restituito dal backend;
- `isStatusLoading`: `boolean` — indica se il primo caricamento dello stato è in corso.

useStopScan è il custom hook responsabile dell'interruzione di una scansione in corso. Implementa l'interfaccia `IStopScanViewModel` e utilizza internamente `useMutation` per invocare `ScanRepository.stopScan`. Accetta opzionalmente una callback `onSettled` invocata al termine della mutazione, indipendentemente dal suo esito, utilizzata dal `ScanButton` per rimuovere la query di stato e resettare la mutazione di avvio. Espone i seguenti membri:

- `triggerStop(scanId: string): void` — avvia la mutazione di interruzione della scansione identificata da `scanId`;
- `isStopping`: `boolean` — indica se la richiesta di interruzione è in corso;
- `stopError`: `Error` — eventuale errore restituito dal backend durante l'interruzione;
- `resetStop(): void` — reimposta lo stato della mutazione al valore iniziale.

IScanViewModel è l'interfaccia che definisce il contratto esposto da `useScan` verso il layer View. Dichiarare i membri `triggerScan`, `scanId`, `isPending`, `isSuccess`, `error` e `reset`, con le stesse semantiche descritte per l'hook.

IScanStatusViewModel è l'interfaccia che definisce il contratto esposto da `useScanStatus`. Dichiarare i membri `scanStatus` e `isStatusLoading`.

IStopScanViewModel è l'interfaccia che definisce il contratto esposto da `useStopScan`. Dichiarare i membri `triggerStop`, `isStopping`, `stopError` e `resetStop`.

4.1.4.7.4 Repository

IScanRepository è l'interfaccia che definisce il contratto del layer di accesso ai dati per la feature. Dichiarare i seguenti metodi:

- `requestScan(payload: StartScanInfo): Promise<string>` — invia la richiesta di avvio scansione al backend e restituisce l'identificativo della scansione creata;
- `stopScan(scanId: string): Promise<void>` — invia la richiesta di interruzione della scansione identificata da `scanId`;
- `getScanStatus(scanId: string): Promise<string>` — recupera lo stato corrente della scansione identificata da `scanId`.

ScanRepository è la classe concreta che implementa `IScanRepository`. Utilizza le funzioni `apiPost`, `apiPatch` e `apiGet` del client HTTP condiviso. Espone i seguenti metodi:

- `requestScan(payload: StartScanInfo): Promise<string>` — esegue una richiesta POST all'endpoint `/api/scan` con il payload fornito e restituisce il campo `scanId` dalla risposta;
- `stopScan(scanId: string): Promise<void>` — esegue una richiesta PATCH all'endpoint `/api/scan/:scanId` per interrompere la scansione;
- `getScanStatus(scanId: string): Promise<string>` — esegue una richiesta GET all'endpoint `/api/scan/:scanId/status` e restituisce lo stato corrente come stringa.

4.1.4.8 Report

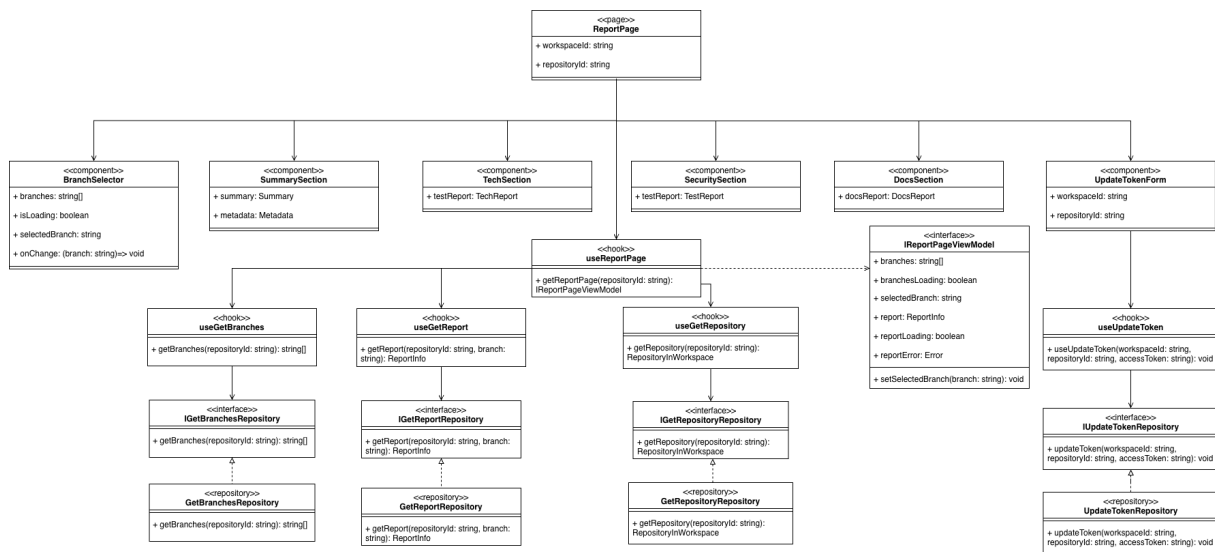


Figura 7: UML feature Report

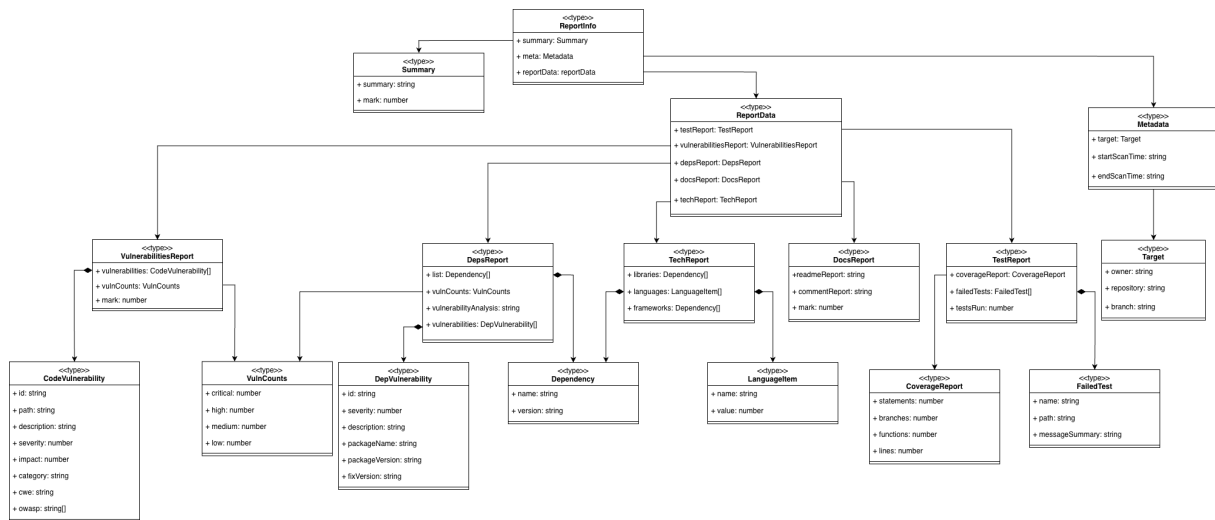


Figura 8: UML type reportInfo

La feature *Report* consente all'utente autenticato di visualizzare i risultati di analisi di un repository per un branch selezionato. La pagina principale recupera la lista dei branch disponibili, seleziona automaticamente il branch `develop` se presente o il primo disponibile, e carica il report corrispondente. Il report è suddiviso in sezioni tematiche dedicate a riepilogo globale, tecnologie rilevate, test e coverage, sicurezza e documentazione. È presente una barra di navigazione laterale per lo scorrimento rapido tra le sezioni. La pagina include inoltre il componente `UpdateTokenForm` per aggiornare il token di accesso del repository e il componente `ScanButton` per avviare nuove scansioni.

4.1.4.8.1 Tipi e DTO

ReportInfo è il tipo radice che modella l'intero report restituito dal backend. Espone i seguenti campi:

- `summary?: Summary` — riepilogo globale del report con voto e testo descrittivo, opzionale in quanto non sempre presente;
- `data: ReportData` — oggetto contenente i risultati delle analisi suddivisi per categoria;
- `metadata?: Metadata` — metadati della scansione, opzionali.

Summary è il tipo che modella il riepilogo globale del report. Espone i seguenti campi:

- `summary: string` — testo descrittivo del riepilogo in formato Markdown;
- `mark: number` — voto globale del report su scala da 0 a 10.

Metadata è il tipo che raccoglie i metadati della scansione. Espone i seguenti campi:

- `target: Target` — informazioni sul repository scansionato;
- `startScanTime: string` — timestamp di inizio scansione in formato ISO 8601;
- `endScanTime: string` — timestamp di fine scansione in formato ISO 8601.

Target è il tipo che identifica il repository scansionato. Espone i campi `owner: string`, `repository: string` e `branch: string`.

ReportData è il tipo che raccoglie i risultati delle analisi suddivisi per categoria. Espone i seguenti campi:

- `testReport: TestReport` — risultati dell'analisi dei test;
- `vulnerabilitiesReport: VulnerabilitiesReport` — risultati dell'analisi delle vulnerabilità nel codice;
- `depsReport: DepsReport` — risultati dell'analisi delle dipendenze vulnerabili;
- `docsReport: DocsReport` — risultati dell'analisi della documentazione;
- `techReport: TechReport` — risultati dell'analisi tecnologica.

TestReport è il tipo che modella i risultati dell'analisi dei test. Espone i seguenti campi:

- `coverageReport: CoverageReport` — percentuali di coverage per categoria;
- `failedTests: FailedTest[]` — lista dei test falliti;
- `testsRun: number` — numero totale di test eseguiti.

CoverageReport è il tipo che modella le percentuali di coverage. Espone i campi `statements: number`, `branches: number`, `functions: number` e `lines: number`.

FailedTest è il tipo che modella un test fallito. Espone i campi `name: string`, `path: string` e `messageSummary: string`.

VulnerabilitiesReport è il tipo che modella i risultati dell'analisi delle vulnerabilità nel codice sorgente. Espone i seguenti campi:

- `vulnerabilities: CodeVulnerability[]` — lista delle vulnerabilità rilevate;
- `vulnCounts?: VulnCounts` — conteggio delle vulnerabilità per livello di severità, opzionale;
- `mark: number` — voto della sicurezza del codice su scala da 0 a 10.

CodeVulnerability è il tipo che modella una singola vulnerabilità nel codice. Espone i campi `id: string`, `path: string`, `description: string`, `severity: number`, `impact: string`, `category: string`, `cwe?: string` e `owasp?: string[]`.

DepsReport è il tipo che modella i risultati dell'analisi delle dipendenze vulnerabili. Espone i seguenti campi:

- `list?: Dependency[]` — lista completa delle dipendenze rilevate, opzionale;
- `vulnerabilities: DepVulnerability[]` — lista delle dipendenze vulnerabili;
- `vulnCounts?: VulnCounts` — conteggio delle dipendenze vulnerabili per livello di severità, opzionale;
- `vulnerabilityAnalysis: string` — testo descrittivo dell'analisi in formato Markdown.

DepVulnerability è il tipo che modella una singola dipendenza vulnerabile. Espone i campi `id: string`, `severity: string`, `description?: string`, `packageName: string`, `packageVersion: string` e `fixVersion?: string`.

VulnCounts è il tipo che aggrega il conteggio delle vulnerabilità per livello di severità. Espone i campi `critical: number`, `high: number`, `medium: number` e `low: number`.

Dependency è il tipo che modella una singola dipendenza rilevata. Espone i campi `name: string` e `version: string`.

DocsReport è il tipo che modella i risultati dell'analisi della documentazione. Espone i campi `readmeReport: string`, `commentReport: string` e `mark: number`.

TechReport è il tipo che modella i risultati dell'analisi tecnologica. Espone i seguenti campi:

- `libraries: Dependency[]` — lista delle librerie rilevate con nome e versione;
- `frameworks: Dependency[]` — lista dei framework rilevati con nome e versione;
- `languages: LanguageItem[]` — lista dei linguaggi rilevati con percentuale di utilizzo.

LanguageItem è il tipo che modella un linguaggio rilevato nel repository. Espone i campi `name: string` e `value: number`, dove `value` rappresenta la percentuale di utilizzo del linguaggio sul totale del codice sorgente.

4.1.4.8.2 Componenti View

ReportPage è il componente che costituisce la pagina principale della feature. Riceve le props `workspaceId: string` e `repositoryId: string`. Recupera lo stato dall'hook `useReportPage` e le informazioni del repository da `useGetRepository`. Presenta un header fisso con il nome del repository, il selettore di branch, il pulsante di scansione e il form di aggiornamento token. Il contenuto principale è organizzato in un layout a due colonne con una barra di navigazione laterale sticky e le sezioni del report a destra. Gestisce i casi di caricamento con skeleton animati, errore con un messaggio dedicato e assenza del campo `summary` omettendo la relativa sezione.

BranchSelector è il componente che renderizza il selettore del branch. Riceve le seguenti props:

- `branches: string[]` — lista dei branch disponibili;
- `isLoading: boolean` — se `true`, mostra uno skeleton animato al posto del selettore;
- `selectedBranch: string` — branch attualmente selezionato;
- `onChange: (branch: string) => void` — callback invocata al cambio del branch selezionato.

SummarySection è il componente che renderizza il riepilogo globale del report. Riceve le props `summary: Summary` e `metadata?: Metadata`. Mostra il voto globale tramite un grafico SVG ad anello colorato in base alla soglia, il testo del riepilogo in formato Markdown e, se presenti i metadati, le informazioni di inizio, fine e durata della scansione.

TechSection è il componente che renderizza i risultati dell'analisi tecnologica. Riceve le props `techReport: TechReport` e `allDeps?: Dependency[]`. Mostra un grafico a torta dei linguaggi con percentuale di utilizzo, la lista dei framework e delle librerie rilevati e, se fornita, la lista completa delle dipendenze in una tabella espandibile. I linguaggi con utilizzo inferiore all'1% vengono mostrati come chip separati.

TestSection è il componente che renderizza i risultati dell'analisi dei test. Riceve la prop `testReport: TestReport`. Mostra la coverage media, le percentuali per categoria tramite un grafico a barre colorato in base alla soglia, il numero totale di test eseguiti e, se presenti, la lista dei test falliti con nome, percorso e messaggio di errore.

SecuritySection è il componente che renderizza i risultati dell'analisi della sicurezza. Riceve le props `vulnerabilitiesReport: VulnerabilitiesReport` e `depsReport: DepsReport`.

Mostra il voto di sicurezza, una panoramica numerica delle vulnerabilità per categoria, grafici a barre delle vulnerabilità per severità sia per il codice che per le dipendenze, la lista espandibile delle vulnerabilità nel codice con dettaglio di rimedio, CWE e OWASP, e la tabella delle dipendenze vulnerabili raggruppate per pacchetto.

DocsSection è il componente che renderizza i risultati dell'analisi della documentazione. Riceve la prop `docsReport: DocsReport`. Mostra il voto della documentazione tramite una barra di punteggio e il contenuto testuale dell'analisi del README e dei commenti nel codice, entrambi renderizzati in formato Markdown.

UpdateTokenForm è il componente che consente l'aggiornamento del token di accesso GitHub di un repository privato direttamente dalla pagina del report. Riceve le props `workspaceId: string` e `repositoryId: string`. Mostra un pulsante icona che, al click, espande inline un campo di testo per l'inserimento del token e un pulsante di salvataggio. Gli errori di validazione lato client e gli errori del backend vengono mostrati sotto il campo.

4.1.4.8.3 Hook

useReportPage è il custom hook aggregatore che costituisce il layer ViewModel della feature. Implementa l'interfaccia `IReportPageViewModel` componendo internamente `useGetBranches` e `useGetReport`. Gestisce la selezione automatica del branch tramite `useEffect`: al caricamento dei branch seleziona `develop` se presente, altrimenti il primo disponibile; la selezione automatica non sovrascrive una selezione effettuata manualmente dall'utente, tracciata tramite un flag interno `userSelected`. Accetta il parametro `repositoryId: string` ed espone i seguenti membri tramite `IReportPageViewModel`:

- `branches: string[]` — lista dei branch disponibili per il repository;
- `branchesLoading: boolean` — indica se il caricamento dei branch è in corso;
- `selectedBranch: string` — branch attualmente selezionato;
- `setSelectedBranch(branch: string): void` — aggiorna il branch selezionato marcando la selezione come manuale;
- `report: ReportInfo` — dati del report per il branch selezionato;
- `reportLoading: boolean` — indica se il caricamento del report è in corso;
- `reportError: Error` — eventuale errore nel caricamento del report.

useGetBranches è il custom hook responsabile del recupero dei branch disponibili per un repository. Utilizza `useQuery` con query key `['branches', repositoryId]`, abilitata solo se `repositoryId` è definito. Accetta il parametro `repositoryId: string` e restituisce i dati della query.

useGetReport è il custom hook responsabile del recupero del report per un branch specifico. Utilizza `useQuery` con query key `['report', repositoryId, branch]`, abilitata solo se `branch` è definito, e con `retry: false` per non ripetere la richiesta in caso di errore. Accetta i parametri `repositoryId: string` e `branch: string | undefined` e restituisce i dati della query.

useGetRepository è il custom hook responsabile del recupero dei metadati del repository visualizzato. Utilizza `useQuery` con query key `['repository', repositoryId]`, abilitata solo se `repositoryId` è definito. Accetta il parametro `repositoryId: string` e restituisce i dati del repository corrispondente.

useUpdateToken è il custom hook responsabile dell'aggiornamento del token di accesso di un repository privato. Utilizza `useMutation` per invocare `UpdateTokenRepository.updateToken`. Prima di inviare la richiesta esegue una validazione lato client del token tramite espressione regolare, accettando token nel formato `ghp_*` o `github_pat_*`. Accetta i parametri `workspaceId: string`, `repositoryId: string` e la callback opzionale `onSaved` invocata al successo della mutazione. Espone i seguenti membri:

- `token: string` — valore corrente del campo token;
- `setToken(token: string): void` — aggiorna il valore del campo token;
- `isPending: boolean` — indica se la mutazione è in corso;
- `clientError: string` — eventuale messaggio di errore di validazione lato client;
- `serverError: Error` — eventuale errore restituito dal backend;
- `handleSubmit(e: React.SyntheticEvent<HTMLFormElement>): void` — gestisce la submission eseguendo prima la validazione del formato del token.

IReportPageViewModel è l'interfaccia che definisce il contratto esposto da `useReportPage` verso il layer View. Dichiarare i membri `branches`, `branchesLoading`, `selectedBranch`, `setSelectedBranch`, `report`, `reportLoading` e `reportError`, con le stesse semantiche descritte per l'hook.

4.1.4.8.4 Repository

IGetBranchesRepository è l'interfaccia che definisce il contratto per il recupero dei branch di un repository. Dichiarare il metodo:

- `getBranches(repositoryId: string): Promise<string[]>` — recupera la lista dei branch disponibili per il repository identificato da `repositoryId`.

GetBranchesRepository è la classe concreta che implementa `IGetBranchesRepository`. Utilizza `apiGet` per eseguire una richiesta GET all'endpoint `/api/repositories/:repositoryId/branches`. Gestisce internamente gli errori HTTP con codice 404 e 403 restituendo un array vuoto, propagando invece tutti gli altri errori. Espone il metodo:

- `getBranches(repositoryId: string): Promise<string[]>` — effettua la chiamata HTTP e restituisce la lista dei branch.

IGetReportRepository è l'interfaccia che definisce il contratto per il recupero del report. Dichiarare il metodo:

- `getReport(repositoryId: string, branch: string): Promise<ReportInfo>` — recupera il report del repository per il branch specificato.

GetReportRepository è la classe concreta che implementa `IGetReportRepository`. Utilizza `apiGet` per eseguire una richiesta GET all'endpoint `/api/repositories/:repositoryId/branches/:branch`, codificando il nome del branch tramite `encodeURIComponent` per gestire correttamente branch con caratteri speciali come lo slash. Espone il metodo:

- `getReport(repositoryId: string, branch: string): Promise<ReportInfo>` — effettua la chiamata HTTP e restituisce il report deserializzato.

IGetRepositoryRepository è l'interfaccia che definisce il contratto per il recupero dei metadati di un singolo repository. Dichiarare il metodo:

- `getRepository(repositoryId: string): Promise<RepositoryInWorkspace>` — recupera i dati del repository identificato da `repositoryId`.

GetRepositoryRepository è la classe concreta che implementa `IGetRepositoryRepository`. Utilizza `apiGet` per eseguire una richiesta GET all'endpoint `/api/repositories/:repositoryId`. Espone il metodo:

- `getRepository(repositoryId: string): Promise<RepositoryInWorkspace>` — effettua la chiamata HTTP e restituisce i dati del repository deserializzati.

IUpdateTokenRepository è l'interfaccia che definisce il contratto per l'aggiornamento del token di accesso di un repository. Dichiarare il metodo:

- `updateToken(workspaceId: string, repositoryId: string, accessToken: string): Promise<void>` — invia al backend il nuovo token di accesso per il repository specificato.

UpdateTokenRepository è la classe concreta che implementa `IUpdateTokenRepository`. Utilizza `apiPatch` per eseguire una richiesta PATCH all'endpoint `/api/workspaces/:workspaceId/repositories` passando come corpo un oggetto contenente il campo `accessToken`. Espone il metodo:

- `updateToken(workspaceId: string, repositoryId: string, accessToken: string): Promise<void>` — effettua la chiamata HTTP di aggiornamento del token.

4.2 Backend

4.2.1 Introduzione

Il backend sfrutta l'approccio modulare offerto dal framework NestJS e adotta un'architettura layered, facendo in modo che i vari moduli siano organizzati in strati logici distinti, per garantire una chiara separazione delle responsabilità.

L'architettura layered si articola sui seguenti strati:

- **Presentation Layer:** funge da punto di ingresso per le richieste HTTP provenienti dal frontend, si occupa esclusivamente della ricezione degli input e della restituzione delle risposte.
- **Business Logic Layer:** in questo livello viene implementata la logica di business.
- **Persistence Layer:** gestisce l'interazione con il database, questo strato isola le operazioni di persistenza dal resto dell'applicazione.

4.2.2 Design pattern

4.2.2.1 Dependency injection

In NestJS, la Dependency Injection (DI) è il meccanismo per coordinare le interazioni tra classi. Attraverso il decoratore `@Injectable()`, i servizi vengono forniti a controller o altri provider in modo automatico. Tale architettura minimizza la dipendenza tra parti del codice e ne semplifica il testing, sollevando l'utente dal compito di dover istanziare manualmente gli oggetti.

4.2.2.2 Singleton

In NestJS, il Singleton è predefinito per gestione dei provider. Una volta inizializzato all'avvio, un servizio mantiene un'unica istanza condivisa per tutto il ciclo di vita dell'applicazione. Questa strategia ottimizza le prestazioni e riduce il consumo di memoria, garantendo che non si debba ricreare continuamente gli oggetti.

4.2.2.3 Repository pattern

Agisce da ponte per separare le regole di business dai dettagli di persistenza. Centralizzando le operazioni verso il database. In questo modo viene nascosta la complessità delle query.

4.2.3 Moduli

4.2.3.1 Auth

Il modulo Auth fornisce le funzionalità per trattare l'autenticazione di ogni richiesta http recuperando i dati di colui che le esegue grazie a Passport.js e apposite guardie

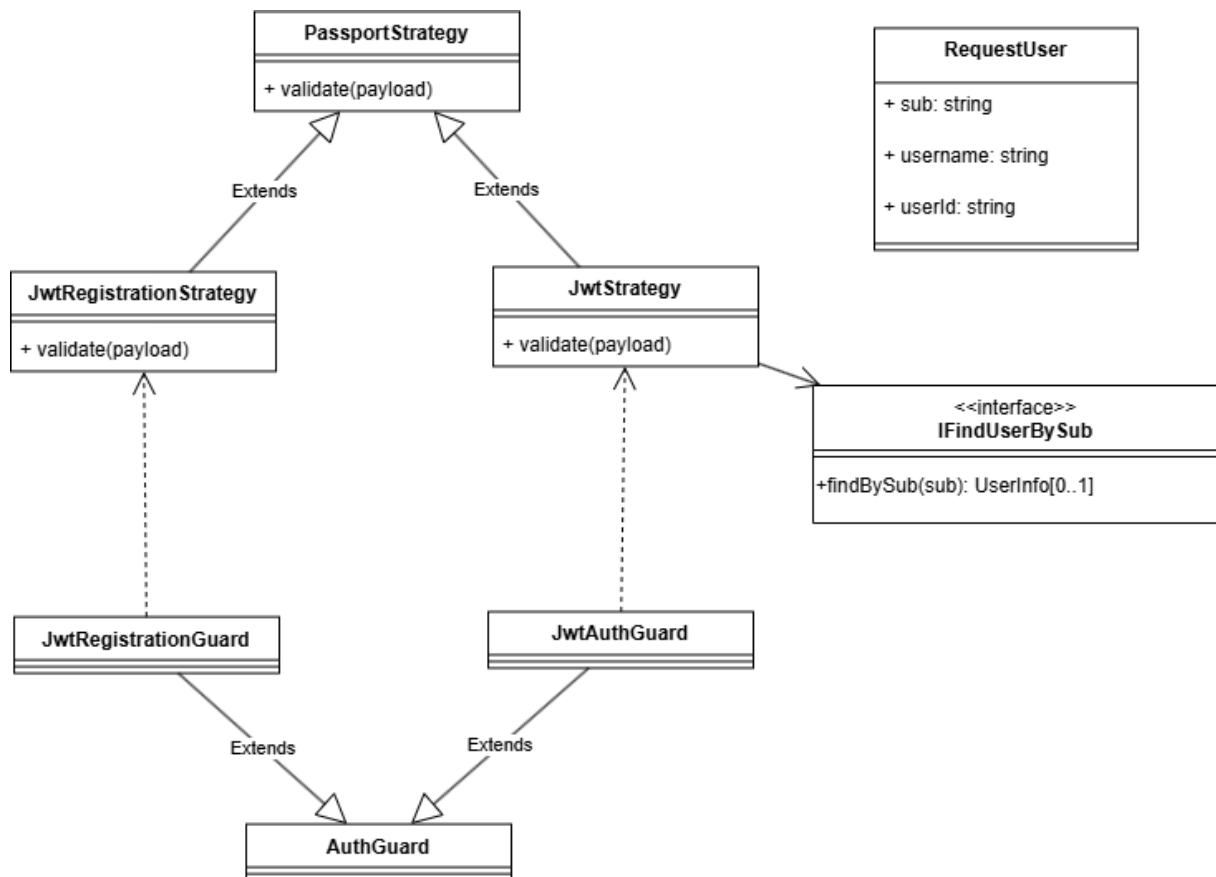


Figura 9: UML Modulo Auth

4.2.3.1.1 Tipi

- **Application logic**

- **RequestUser** rappresenta i dati dell'utente estratti dall'access token JWT utilizzato per l'autenticazione della richieste http con AWS Cognito:
 - * `sub: string`
 - * `username: string`
 - * `userId: string`

4.2.3.1.2 Classi e Interfacce

- **Interfacce**

- IfindUserBySub
 - * +IfindBySub(sub: string): UserInfo[0..1]

• Classi

- JwtRegistrationStrategy
 - * Operazioni
 - +validate(payload) estrae le informazioni dell'utente dall'ID token JWT fornito da Amazon Cognito, e ritorna le informazioni sub, username e email che saranno aggiunte alla richiesta http
- JwtAuthStrategy
 - * Proprietà:
 - -findUserBySub: IfindUserBySub, istanza del servizio per la gestione degli utenti, iniettata tramite dependency injection
 - * Operazioni
 - +validate(payload) estrae le informazioni dell'utente dall'Access token JWT fornito da Amazon Cognito, recupera l'id dell'utente tramite findUserBySub e restituisce un oggetto, e ritorna le informazioni sub, username e email che saranno aggiunte alla richiesta http nel campo user
- JwtAuthGuard Guardia che utilizza la strategia di autenticazione JwtAuthStrategy per proteggere gli endpoint che richiedono autenticazione, verificando la validità del token JWT e autorizzando l'accesso solo agli utenti autenticati
- JwtRegistrationGuard Guardia che utilizza la strategia di autenticazione JwtRegistrationStrategy per proteggere l'endpoint di registrazione di registrazione, verificando la validità dell'ID token JWT e autorizzando l'accesso solo agli utenti che stanno completando il processo di registrazione

4.2.3.2 User

Il modulo User è responsabile della gestione degli utenti; fornisce le funzionalità per registrare l'utente sul database, dopo che esso si è registrato attraverso cognito e per ottenere un utente attraverso sub o username

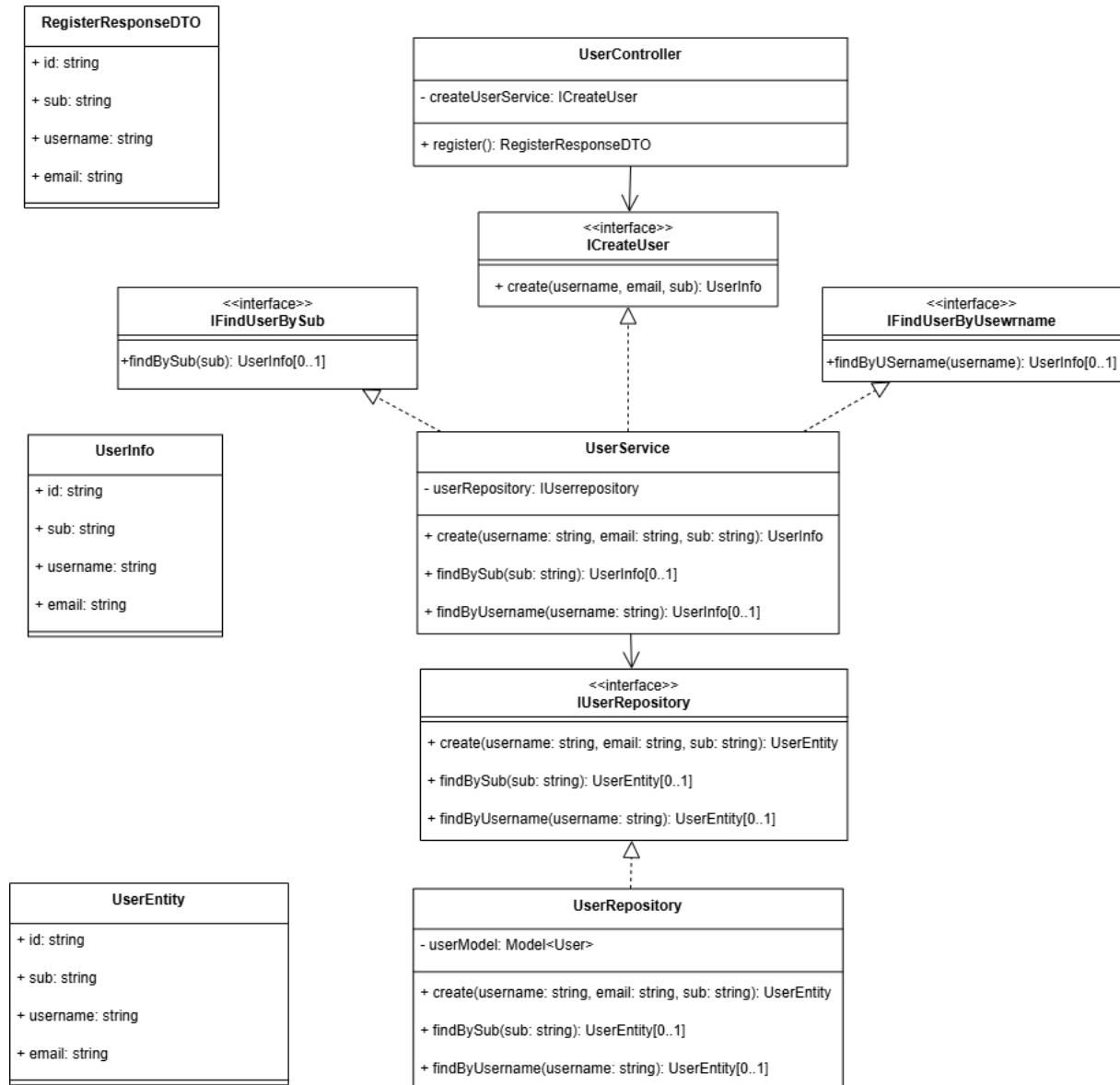


Figura 10: UML Modulo User

4.2.3.2.1 Tipi

- **Application logic**

- `RegisterResponseDTO`, rappresenta il dto di risposta dopo la registrazione di un utente:

- * id: string
- * sub: string
- * username: string
- * email: string

- **Business logic**

- UserInfo, rappresenta il modello di un utente:

- * id: string
- * sub: string
- * username: string
- * email: string

- **Persistence logic**

- UserEntity, rappresenta i dati necessari alla persistenza di un utente:

- * id: string
- * sub: string
- * username: string
- * email: string

4.2.3.2.2 Classi e Interfacce

- **Interfacce**

- IfindUserBySub
 - * +IfindBySub(sub: string): UserInfo[0..1]
- ICreateUser
 - * +createUser(sub, username, email): UserInfo
- IfindUserByUsername
 - * +findByUsername(username: string): UserInfo[0..1]
- IUserRepository
 - * +create(sub, username, email): UserEntity
 - * +findByUsername(username: string): UserEntity[0..1]
 - * +findBySub(sub: string): UserEntity[0..1]

- **Classi**

- UserController
 - * Proprietà:
 - -createUserService: ICreateUser istanza del servizio per la gestione degli utenti, iniettata tramite dependency injection
 - * Operazioni

- `+register(req)`: estrae le informazioni dell'utente dall'Id Token fornito da Amazon Cognito, e con queste chiama il metodo `createUser` del servizio per registrare l'utente nel database, ritornando una risposta un `RegisterResponseDTO` o un errore in caso di fallimento
- UserService
- * Proprietà:
 - `-userRepository`: `IUserRepository` istanza del repository per la gestione degli utenti, iniettata tramite dependency injection
 - * Operazioni
 - `+create(sub, username, email)`: `UserInfo` riceve le informazioni per creare un nuovo utente, controlla che l'utente non esista tramite `findUserBySub` e chiama il metodo `create` del repository per salvare l'utente nel database,
 - `+findBySub(sub)`: `UserInfo[0..1]` chiama il metodo `findBySub` del repository per recuperare le informazioni dell'utente corrispondente, se questo non ritorna null lo trasforma in `UserInfo` e lo restituisce, altrimenti ritorna null
 - `+findByUsername(username)`: `UserInfo[0..1]` chiama il metodo `findByUsername` del repository per recuperare le informazioni dell'utente corrispondente, se questo non ritorna null lo trasforma in `UserInfo` e lo restituisce, altrimenti ritorna null
- UserRepository
- * Proprietà:
 - `-userModel`: `Model<User>`, oggetto Mongoose per la gestione della collezione degli utenti, iniettato tramite dependency injection
 - * Operazioni
 - `+create(sub, username, email)`: `UserEntity`, crea un nuovo `userModel` e lo salva nel database, se l'operazione ha successo trasforma il risultato in `UserEntity` e lo restituisce, altrimenti ritorna un errore
 - `+findBySub(sub)`: `UserEntity[0..1]`, tramite il metodo `findOne` di Mongoose cerca un documento con il campo `sub` corrispondente, se questo non ritorna null lo trasforma in `UserEntity` e lo restituisce, altrimenti ritorna null
 - `+findByUsername(username)`: `UserEntity[0..1]`, tramite il metodo `findOne` di Mongoose cerca un documento con il campo `username` corrispondente, se questo non ritorna null lo trasforma in `UserEntity` e lo restituisce, altrimenti ritorna null

4.2.3.3 Membership

Il modulo Membership è responsabile della gestione degli inviti ai workspace; fornisce le funzionalità per inviare un invito ad un certo workspace, poter recuperare gli inviti ricevuti e poter accettare o rifiutare un invito.

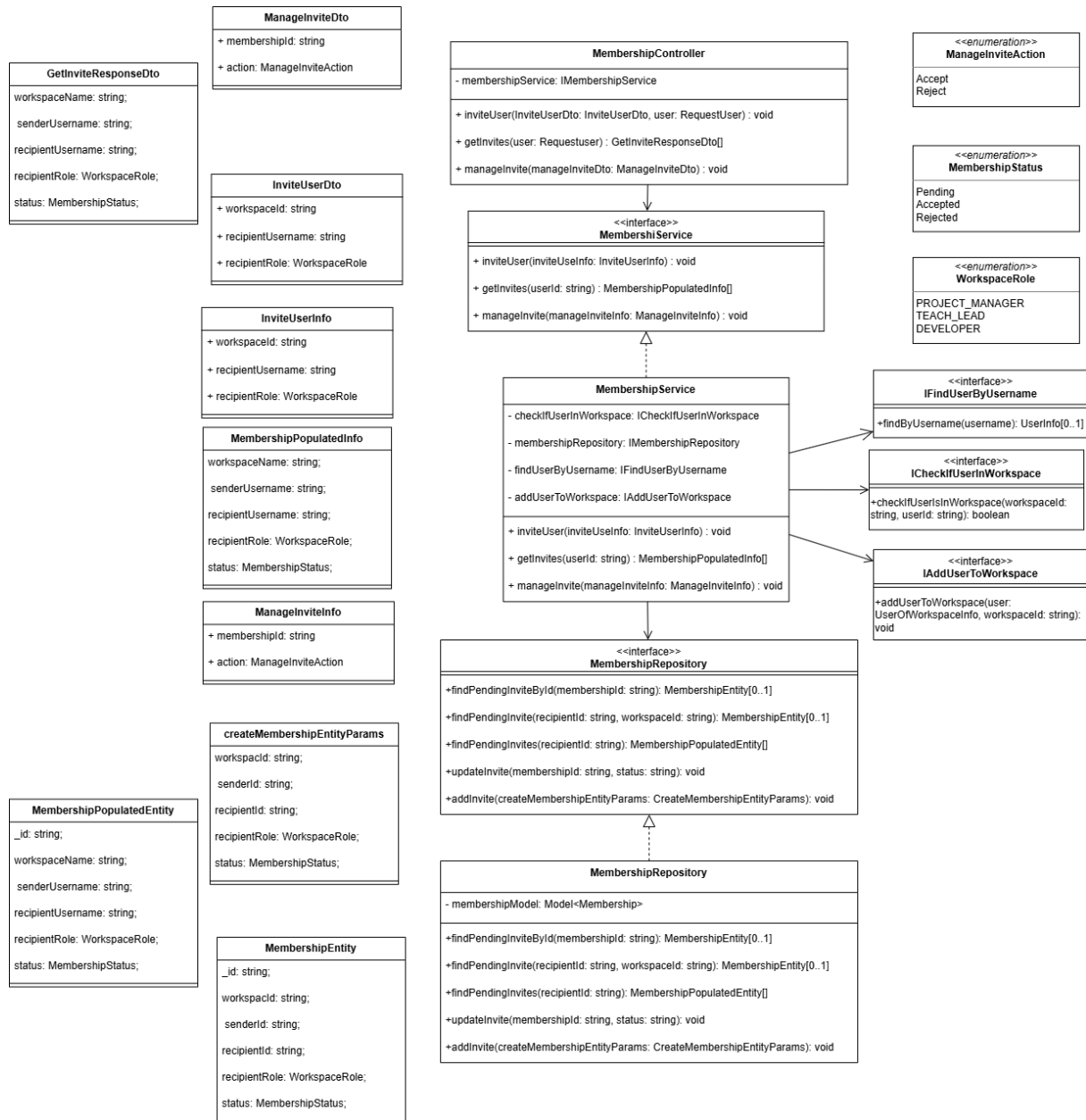


Figura 11: UML Modulo Membership

4.2.3.3.1 Tipi

- Application logic

- InviteUserDTO, rappresenta i dati per invitare un nuovo membro a un workspace:
 - * workspaceId: string
 - * recipientUsername: string
 - * recipientRole: string
- GetInviteResponseDTO, rappresenta la risposta restituita al client dopo aver recuperato un invito:
 - * workspaceName: string
 - * senderUsername: string
 - * recipientUsername: string
 - * recipientRole: WorkspaceRole
 - * status: MembershipStatus
- ManageInviteDTO, rappresenta i dati per gestire un invito (accettare o rifiutare):
 - * membershipId: string
 - * action: ManageInviteAction

- **Business logic**

- InviteUserInfo, rappresenta i dati per invitare un nuovo membro a un workspace a livello di service:
 - * workspaceId: string
 - * recipientUsername: string
 - * recipientRole: string
- membershipPopulatedInfo, rappresenta i dati di un invito popolati con le informazioni del workspace e dell'utente a livello di service:
 - * workspaceName: string
 - * senderUsername: string
 - * recipientUsername: string
 - * recipientRole: WorkspaceRole
 - * status: MembershipStatus
- ManageInviteInfo, rappresenta i dati per gestire un invito (accettare o rifiutare) a livello di service:
 - * membershipId: string
 - * action: ManageInviteAction

- **Persistence logic**

- membershipEntity, rappresenta i dati di un invito a livello di persistenza:
 - * _id: string
 - * workspaceId: string
 - * senderId: string
 - * recipientId: string
 - * recipientRole: WorkspaceRole

- * status: MembershipStatus
- createMembershipEntityParams, rappresenta i dati necessari per creare un nuovo invito a livello di persistenza:
 - * workspaceId: string
 - * senderId: string
 - * recipientId: string
 - * recipientRole: WorkspaceRole
 - * status: MembershipStatus
- membershipPopulatedEntity, rappresenta i dati di un invito popolati con le informazioni del workspace e dell'utente a livello di persistenza:
 - * _id: string
 - * workspaceName: string
 - * senderUsername: string
 - * recipientUsername: string
 - * recipientRole: WorkspaceRole
 - * status: MembershipStatus

• Enumerazioni

- WorkspaceRole, rappresenta i ruoli che un membro può avere all'interno di un workspace:
 - * PROJECT_MANAGER
 - * TECH_LEAD
 - * DEVELOPER
- MembershipStatus, rappresenta lo stato di un invito:
 - * pending
 - * accepted
 - * rejected
- ManageInviteAction, rappresenta le azioni che si possono compiere su un invito:
 - * accept
 - * reject

4.2.3.3.2 Classi e Interfacce

• Interfacce

- IMembershipService
 - * +inviteUser(inviteUserInfo: InviteUserInfo): void
 - * +getInvites(userId: string): MembershipPopulatedInfo[]
 - * +manageInvite(manageInviteInfo: ManageInviteInfo): void
- IAddUserToWorkspace

```

    * +addUserToWorkspace(user:  UserOfWorkspaceInfo, workspaceId:  string):
      void
- IcheckIfUserInWorkspace
    * +checkIfUserInWorkspace(userId:  string, workspaceId:  string):  boolean
- IfindUserByUsername
    * +findByUUsername(username):  UserInfo[0..1]
- IMembershipRepository
    * +findPendingInviteById(membershipId:  string):  MembershipEntity[0..1]
    * +findPendingInvite(recipientId:  string, workspaceId:  string):  MembershipEntity
    * +findPendingInvites(recipientId:  string):  MembershipPopulatedEntity[]
    * +updateInvite(membershipId:  string, status:  string):  void
    * +addInvite(createMembershipEntityParams:  CreateMembershipEntityParams):
      void

```

• Classi

```

- MembershipController
    * Proprietà:
      · -membershipService: IMembershipService, istanza del servizio per la gestione degli inviti, iniettata tramite dependency injection
    * Operazioni
      · + inviteUser(InviteUserDto:  InviteUserDto, user:  RequestUser) : void, riceve i dati per invitare un nuovo membro a un workspace, estrae l'id dell'utente mittente dall'oggetto RequestUser e chiama il metodo inviteUser, se l'operazione ha successo ritorna una risposta vuota, altrimenti ritorna un errore
      · + getInvites(user:  Requestuser) :  GetInviteResponseDto[], riceve l'oggetto RequestUser, estrae l'id dell'utente e chiama il metodo getInvites del servizio per recuperare la lista degli inviti ricevuti dall'utente, se l'operazione ha successo ritorna una lista di GetInviteResponseDTO, altrimenti ritorna un errore
      · + manageInvite(manageInviteDto:  ManageInviteDto) :  void, riceve i dati per gestire un invito, chiama il metodo manageInvite del servizio per accettare o rifiutare l'invito, se l'operazione ha successo ritorna una risposta vuota, altrimenti ritorna un errore
- MembershipService
    * Proprietà:
      · -membershipRepository: IMembershipRepository istanza del repository per la gestione degli inviti, iniettata tramite dependency injection
      · -findUserByUsername: IfindUserByUsername istanza del servizio per recuperare le informazioni di un utente tramite username, iniettata tramite dependency injection

```

- `-addUserToWorkspace`: `IAddUserToWorkspace` istanza del servizio per aggiungere un utente a un workspace, iniettata tramite dependency injection
- `-checkIfUserInWorkspace`: `IcheckIfUserInWorkspace` istanza del servizio per verificare se un utente è già membro di un workspace, iniettata tramite dependency injection

* Operazioni

- `+ inviteUser(inviteUserInfo: InviteUserInfo) : void`, riceve le informazioni per invitare un nuovo membro a un workspace, recupera l'utente tramite `findUserByUsername`, che l'utente non sia già membro del workspace tramite `checkIfUserInWorkspace`, e che l'utente non abbia già un invito in sospeso per quel workspace tramite `findPendingInvite`, se questi controlli hanno successo chiama il metodo `addInvite` del repository per salvare l'invito nel database, altrimenti ritorna un errore
- `+ getInvites(userId: string) : MembershipPopulatedInfo[]`, riceve l'id dell'utente, chiama il metodo `findPendingInvites` del repository per recuperare la lista degli inviti ricevuti dall'utente, se l'operazione ha successo trasforma ogni invito in un `GetInviteInfo` e ritorna la lista, altrimenti ritorna un errore
- `+ manageInvite(manageInviteInfo: ManageInviteInfo) : void`, riceve le informazioni per gestire un invito, controlla che l'invito esista tramite `findPendingInvite`, chiama il metodo `updateInvite` del repository per accettare o rifiutare l'invito, se l'operazione ha successo e l'invito è stato accettato chiama il metodo `addUserToWorkspace` per aggiungere l'utente al workspace, altrimenti ritorna un errore

– MembershipRepository

* Proprietà:

- `-userModel`: `Model<Membership>`, oggetto Mongoose per la gestione della collezione degli inviti, iniettato tramite dependency injection

* Operazioni

- `+findPendingInviteById(membershipId: string): MembershipEntity[0..1]`, tramite il metodo `findOne` di Mongoose cerca un documento con il campo `_id` corrispondente e status "pending", se questo non ritorna null lo trasforma in `MembershipEntity` e lo restituisce, altrimenti ritorna null
- `+findPendingInvite(recipientId: string, workspaceId: string): MembershipEntity`, tramite il metodo `findOne` di Mongoose cerca un documento con i campi `recipientId` e `workspaceId` corrispondenti e status "pending", se questo non ritorna null lo trasforma in `MembershipEntity` e lo restituisce, altrimenti ritorna null
- `+findPendingInvites(recipientId: string): MembershipPopulatedEntity[]`, tramite il metodo `find` di Mongoose cerca tutti i documenti con il campo `recipientId` corrispondente e status "pending", popola i campi `workspaceId` e `senderId` con le informazioni del workspace e dell'utente mittente, trasforma ogni documento in `MembershipPopulatedEntity` e ritorna la lista
- `+updateInvite(membershipId: string, status: string): void`, tra-

mite il metodo `findByIdAndUpdate` di Mongoose aggiorna il campo `status` del documento con il campo `_id` corrispondente, se l'operazione ha successo ritorna `void`, altrimenti ritorna un errore

- `+addInvite(createMembershipEntityParams: CreateMembershipEntityParams): void`, crea un nuovo documento con i dati forniti, se l'operazione ha successo ritorna `void`, altrimenti ritorna un errore

4.2.3.4 Workspace Manager

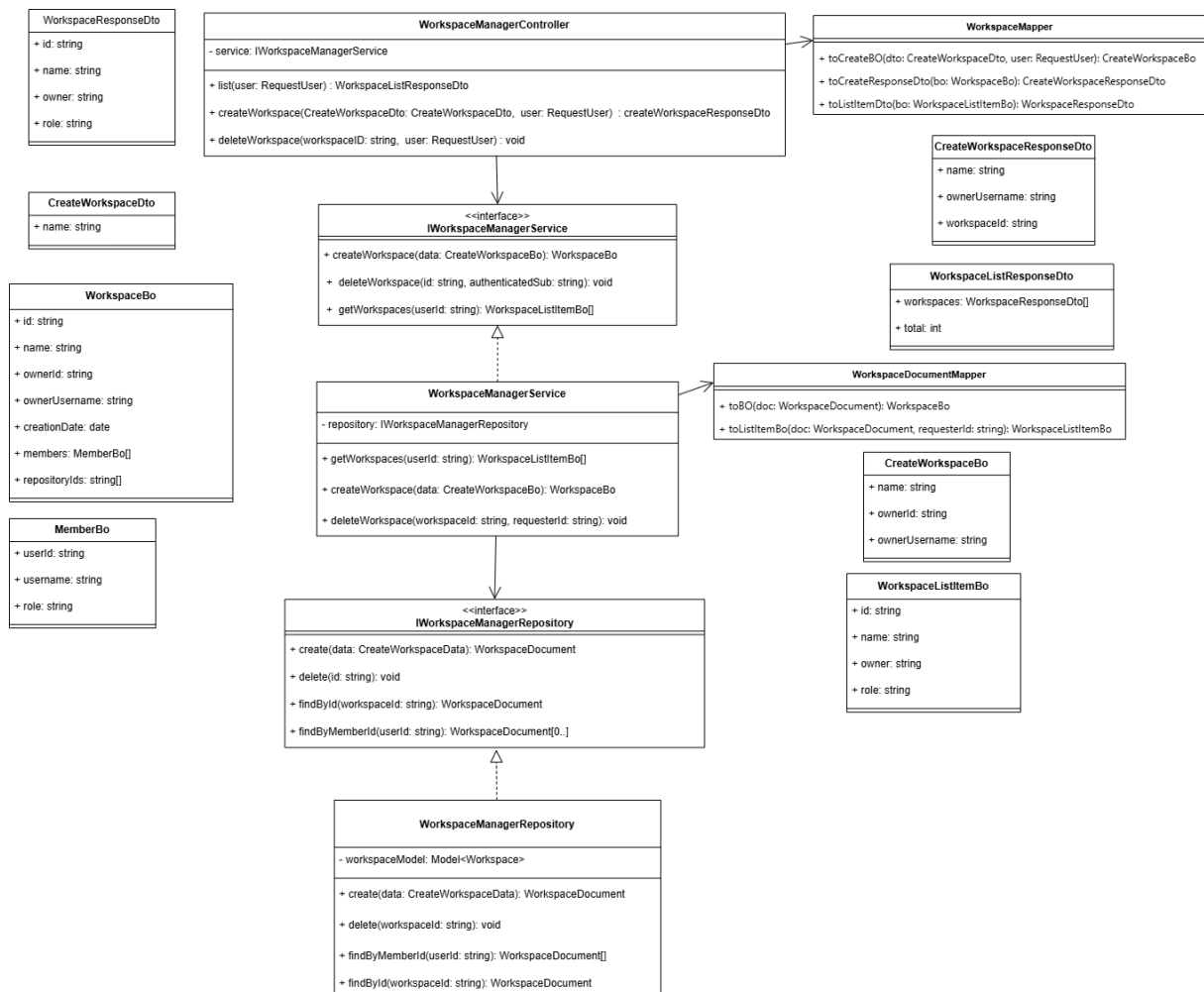


Figura 12: UML WorkspaceManager

Il modulo `WorkspaceManager` è responsabile della gestione dei workspace; fornisce le funzionalità per la creazione, eliminazione e consultazione dei workspace associati ad un utente autenticato. Il modulo garantisce il rispetto delle regole di accesso, permettendo ad esempio la cancellazione solo all'owner del workspace o la creazione di un workspace solo se non esiste già un altro workspace con lo stesso nome per lo stesso utente.

4.2.3.4.1 Tipi

- **Application logic**

- `CreateWorkspaceDto`, rappresenta i dati forniti dall'utente per la creazione di un nuovo workspace:

- * `name`: string (lunghezza tra 2 e 30 caratteri)

- `CreateWorkspaceResponseDto`, rappresenta la risposta restituita al client dopo la creazione di un workspace:
 - * `id`: string
 - * `name`: string
 - * `ownerUsername`: string
- `WorkspaceResponseDto`, rappresenta un workspace nella lista restituita al client:
 - * `id`: string
 - * `name`: string
 - * `owner`: string (username del proprietario)
 - * `role`: string (ruolo dell'utente nel workspace)
- `WorkspaceListResponseDto`, rappresenta la risposta contenente la lista dei workspace dell'utente:
 - * `workspaces`: `WorkspaceResponseDto[]`
 - * `total`: number

- **Business logic**

- `CreateWorkspaceBo`, rappresenta i dati necessari alla creazione di un workspace a livello di service:
 - * `name`: string
 - * `ownerId`: string
 - * `ownerUsername`: string
- `WorkspaceBo`, rappresenta il modello completo di workspace utilizzato nel service:
 - * `id`: string
 - * `name`: string
 - * `ownerId`: string
 - * `ownerUsername`: string
 - * `creationDate`: Date
 - * `members`: `MemberBo[]`
 - * `repositoryIds`: string[]
- `MemberBo`, rappresenta un membro del workspace:
 - * `userId`: string
 - * `username`: string
 - * `role`: string
- `WorkspaceListItemBo`, rappresenta una vista semplificata del workspace per la lista:
 - * `id`: string
 - * `name`: string
 - * `owner`: string
 - * `role`: string

- **Persistence logic**

- CreateWorkspaceData, rappresenta i dati necessari alla persistenza di un workspace:
 - * name: string
 - * ownerId: string
 - * ownerUsername: string
 - * creationDate: Date
 - * initialMembers: array
 - userId: string
 - userUsername: string
 - role: WorkspaceRole

4.2.3.4.2 Classi e Interfacce

• Interfacce

- IWorkspaceManagerService: interfaccia che definisce il contratto del service per la gestione dei workspace.
 - * createWorkspace(data: CreateWorkspaceBo): WorkspaceBo
 - * deleteWorkspace(id: string, authenticatedSub: string): void
 - * getWorkspaces(userId: string): WorkspaceListItemBo[]
- IWorkspaceManagerRepository: interfaccia che definisce le operazioni di accesso ai dati per i workspace.
 - * create(data: CreateWorkspaceData): WorkspaceDocument
 - * delete(id: string): void
 - * findById(workspaceId: string): WorkspaceDocument
 - * findByMemberId(userId: string): WorkspaceDocument[]

• Classi

- WorkspaceManagerController: classe responsabile della gestione degli endpoint REST relativi ai workspace, esposti sotto la rotta /workspaces e protetti dalla guardia JwtAuthGuard.
 - * Proprietà:
 - service: IWorkspaceManagerService, istanza del servizio per la gestione dei workspace, iniettata tramite dependency injection
 - * Operazioni:
 - create(dto: CreateWorkspaceDto, user: RequestUser): CreateWorkspaceResponse
Riceve la richiesta di creazione di un workspace da parte di un utente autenticato, delega la logica al service e restituisce i dati del workspace creato.
 - delete(workspaceId: string, user: RequestUser): void
Gestisce la richiesta di eliminazione di un workspace, delegando al service la verifica dei permessi e l'esecuzione dell'operazione.
 - list(user: RequestUser): WorkspaceResponseDto[]
Restituisce la lista dei workspace di cui l'utente è membro.

- `WorkspaceManagerService`: classe che implementa la logica di business relativa ai workspace e coordina le operazioni tra controller e repository.
 - * Proprietà:
 - `repository`: `IWorkspaceManagerRepository`, istanza del repository per la gestione dei workspace, iniettata tramite dependency injection
 - * Operazioni:
 - `createWorkspace(data: CreateWorkspaceBo): WorkspaceBo`
Crea un nuovo workspace inizializzando i membri (identifica l'owner come l'utente che ha fatto la richiesta di creazione del workspace e inserisce l'utente nella lista dei membri con il ruolo di Project Manager), impostando la data di creazione e gestendo eventuali conflitti, come la presenza di workspace con lo stesso nome per lo stesso utente.
 - `deleteWorkspace(workspaceId: string, requesterId: string): void`
Elimina un workspace dopo aver verificato che l'utente richiedente sia l'owner del workspace.
 - `getWorkspaces(userId: string): WorkspaceListItemBo[]`
Recupera tutti i workspace in cui l'utente è membro e restituisce per ciascuno le informazioni essenziali (id, nome, owner username e ruolo dell'utente).
- `WorkspaceManagerRepository`: classe che implementa la logica di persistenza dei workspace interagendo con il database.
 - * Proprietà:
 - *workspaceModel*: *Model<Workspace>*, oggetto Mongoose per la gestione della collezione dei workspace
 - * Operazioni:
 - `create(data: CreateWorkspaceData): WorkspaceDocument`
Inserisce un nuovo workspace nel database a partire dai dati strutturati ricevuti dal service.
 - `delete(workspaceId: string): void`
Elimina dal database il workspace identificativo dall'id specificato.
 - `findById(workspaceId: string): WorkspaceDocument`
Recupera un workspace dal database utilizzando il suo id.
 - `findByMemberId(userId: string): WorkspaceDocument[]`
Recupera tutti i workspace in cui l'utente è presente come membro.
- `WorkspaceMapper`: classe statica responsabile della conversione tra DTO e Business Object.
 - * Operazioni:
 - `toCreateBO(dto: CreateWorkspaceDto, user: RequestUser): CreateWorkspaceBo`
Converte i dati della richiesta e dell'utente autenticato in un Business Object utilizzato dal service.
 - `toCreateResponseDto(bo: WorkspaceBo): CreateWorkspaceResponseDto`
Converte un Business Object in un DTO di risposta per la creazione del workspace.

- `toListItemDto(bo: WorkspaceListItemBo): WorkspaceResponseDto`
Converte un Business Object semplificato in un DTO per la risposta della lista dei workspace.
- `WorkspaceDocumentMapper`: classe statica responsabile della conversione tra documenti di persistenza e Business Object.
 - * Operazioni:
 - `toBO(doc: WorkspaceDocument): WorkspaceBo`
Converte un documento completo proveniente dal database in un oggetto di dominio utilizzato dal service.
 - `toListItemBo(doc: WorkspaceDocument, requesterId: string): WorkspaceListItemBo`
Converte un documento in una rappresentazione semplificata contenente solo le informazioni necessarie per la lista dei workspace.

4.2.3.5 Workspace User

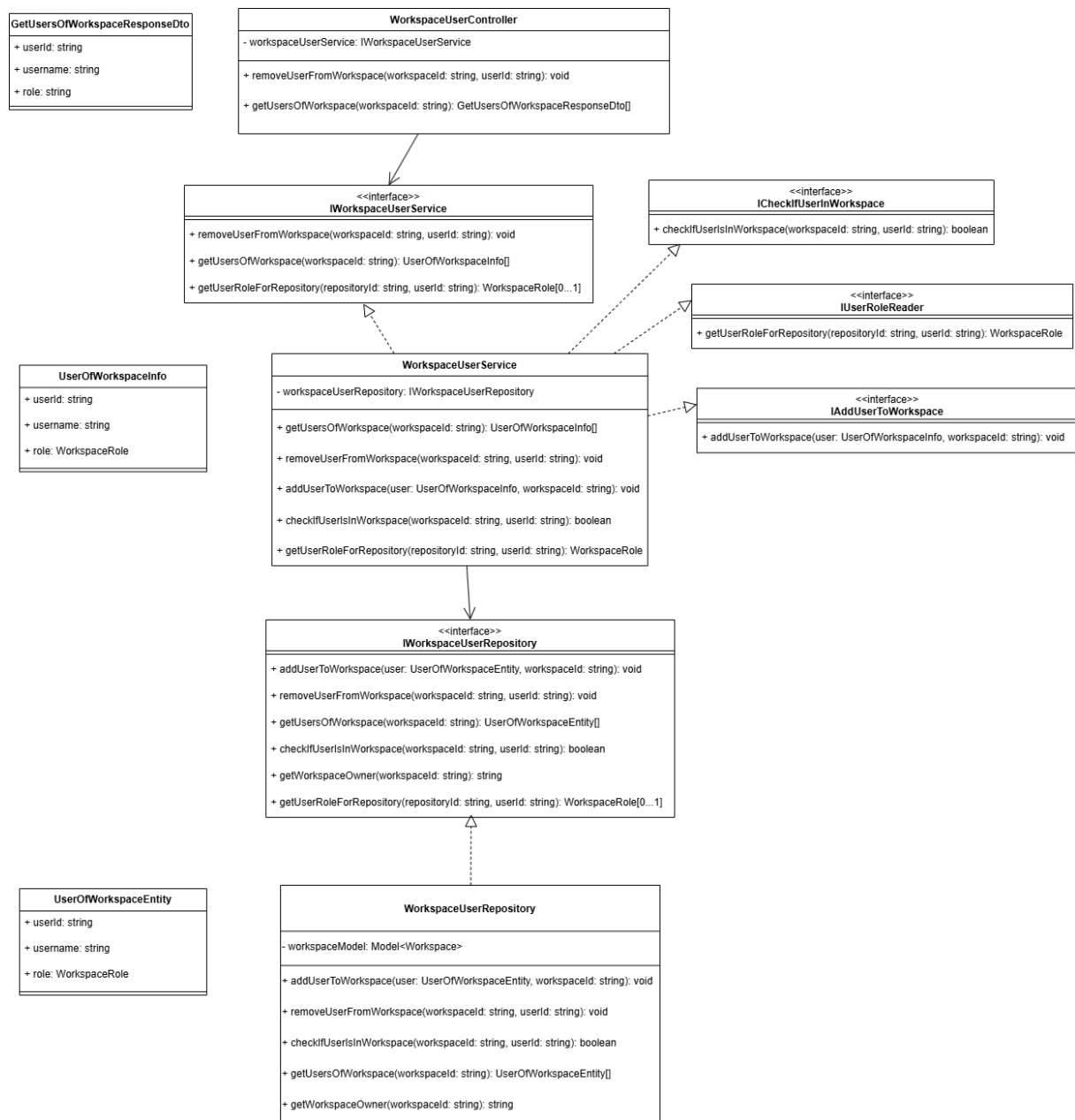


Figura 13: UML WorkspaceUser

Il modulo **WorkspaceUser** è responsabile della gestione degli utenti all'interno dei workspace. Fornisce le funzionalità per recuperare la lista degli utenti appartenenti a un workspace, aggiungere o rimuovere membri e determinare il ruolo di un utente in un workspace. Il modulo garantisce il rispetto delle regole di consistenza, impedendo ad esempio l'aggiunta duplicata di uno stesso utente o la rimozione dell'owner del workspace, e si occupa dell'accesso e della modifica delle informazioni sui membri di un workspace.

4.2.3.5.1 Tipi

- **Application logic**

- `GetUsersOfWorkspaceResponseDto`, rappresenta un utente appartenente a un workspace nella risposta restituita al client:
 - * `userId`: string
 - * `username`: string
 - * `role`: string (ruolo dell'utente nel workspace)

- **Business logic**

- `UserOfWorkspaceInfo`, rappresenta le informazioni di un utente all'interno di un workspace utilizzate a livello di service:
 - * `userId`: string
 - * `username`: string
 - * `role`: `WorkspaceRole`

- **Persistence logic**

- `UserOfWorkspaceEntity`, rappresenta un utente memorizzato all'interno del workspace nel database:
 - * `userId`: string
 - * `username`: string
 - * `role`: `WorkspaceRole`

4.2.3.5.2 Classi e Interfacce

- **Interfacce**

- `IWorkspaceUserService`: interfaccia che definisce il contratto del service per la gestione degli utenti nei workspace.
 - * `removeUserFromWorkspace(workspaceId: string, userId: string): void`
 - * `getUsersOfWorkspace(workspaceId: string): UserOfWorkspaceInfo[]`
 - * `getUserRoleForRepository(repositoryId: string, userId: string): WorkspaceRole`
- `IAddUserToWorkspace`: interfaccia che espone l'operazione di aggiunta di un utente a un workspace, utilizzata da altri moduli.
 - * `addUserToWorkspace(user: UserOfWorkspaceInfo, workspaceId: string): void`
- `ICheckIfUserInWorkspace`: interfaccia che consente di verificare l'appartenenza di un utente a un workspace.
 - * `checkIfUserIsInWorkspace(workspaceId: string, userId: string): boolean`
- `IUserRoleReader`: interfaccia che espone la lettura del ruolo di un utente rispetto a una repository.

- * getUserRoleForRepository(repositoryId: string, userId: string): WorkspaceRole
- IWorkspaceUserRepository: interfaccia che definisce le operazioni di accesso ai dati per la gestione degli utenti nei workspace.
 - * addUserToWorkspace(user: UserOfWorkspaceEntity, workspaceId: string): void
 - * removeUserFromWorkspace(workspaceId: string, userId: string): void
 - * getUsersOfWorkspace(workspaceId: string): UserOfWorkspaceEntity[]
 - * checkIfUserIsInWorkspace(workspaceId: string, userId: string): boolean
 - * getUserRoleForRepository(repositoryId: string, userId: string): WorkspaceRole
 - * getWorkspaceOwner(workspaceId: string): string

• Classi

- WorkspaceUserController: classe responsabile della gestione degli endpoint REST relativi agli utenti dei workspace, esposti sotto la rotta /workspaces.
 - * Proprietà:
 - workspaceUserService: IWorkspaceUserService, istanza del servizio per la gestione degli utenti appartenenti ad un workspace, iniettata tramite dependency injection
 - * Operazioni:
 - getUsersOfWorkspace(workspaceId: string): GetUserOfWorkspaceResponseDto[]
Restituisce la lista degli utenti appartenenti al workspace specificato.
 - removeUserFromWorkspace(workspaceId: string, userId: string): void
Gestisce la richiesta di rimozione di un utente da un workspace, delegando la logica al service.
- WorkspaceUserService: classe che implementa la logica di business relativa alla gestione degli utenti nei workspace.
 - * Proprietà:
 - workspaceUserRepository: IWorkspaceUserRepository, istanza del repository per la gestione degli utenti di un workspace, iniettata tramite dependency injection
 - * Operazioni:
 - getUsersOfWorkspace(workspaceId: string): UserOfWorkspaceInfo[]
Recupera tutti gli utenti appartenenti a un workspace, per ogni membro recupera userId, username e ruolo.
 - removeUserFromWorkspace(workspaceId: string, userId: string): void
Rimuove un utente dal workspace dopo aver verificato che sia effettivamente membro, non sia l'owner e che l'utente che lo richiede abbia il ruolo di Project Manager.
 - getUserRoleForRepository(repositoryId: string, userId: string): WorkspaceRole[0...1]
Recupera il ruolo dell'utente nella repository associata al workspace.

- `addUserToWorkspace(user: UserOfWorkspaceInfo, workspaceId: string): void`
Aggiunge un utente al workspace dopo aver verificato che non sia già presente.
- `checkIfUserIsInWorkspace(workspaceId: string, userId: string): boolean`
Verifica se un utente appartiene al workspace.
- `WorkspaceUserRepository`: classe che implementa la logica di persistenza per la gestione degli utenti nei workspace.
 - * Proprietà:
 - *workspaceModel*: *Model<Workspace>*, oggetto Mongoose per la gestione della collezione dei workspace
 - * Operazioni:
 - `getUsersOfWorkspace(workspaceId: string): UserOfWorkspaceEntity[]`
Recupera dal database tutti gli utenti appartenenti a un workspace.
 - `removeUserFromWorkspace(workspaceId: string, userId: string): void`
Rimuove un utente dal workspace nel database.
 - `addUserToWorkspace(user: UserOfWorkspaceEntity, workspaceId: string): void`
Inserisce un nuovo utente nel workspace nel database.
 - `getUserRoleForRepository(repositoryId: string, userId: string): WorkspaceRole[0...1]`
Recupera il ruolo dell'utente nella repository, se esiste.
 - `checkIfUserIsInWorkspace(workspaceId: string, userId: string): boolean`
Verifica nel database se l'utente è membro del workspace.
 - `getWorkspaceOwner(workspaceId: string): string`
Recupera l'identificativo dell'owner del workspace.

4.2.3.6 Workspace Repository

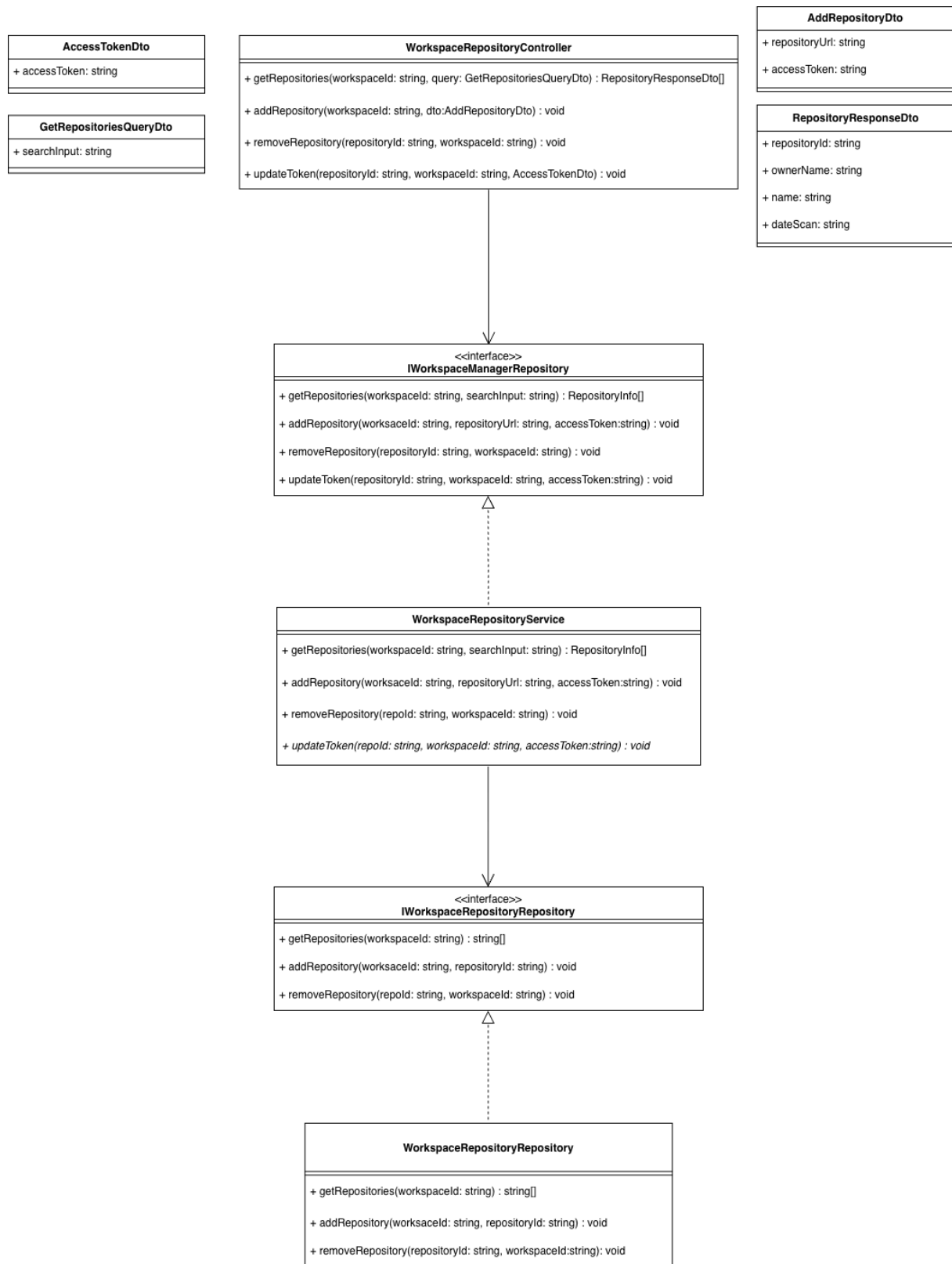


Figura 14: UML WorkspaceRepository

Il modulo `WorkspaceRepository` è responsabile della gestione delle associazioni logiche tra i workspace e le repository esterne. Non si occupa di manipolare i dati fisici del codice sorgente, ma fornisce le funzionalità per collegare, scollegare, consultare e aggiornare i riferimenti delle repository all'interno di uno specifico workspace. Garantisce inoltre che ogni operazione passi attraverso le corrette verifiche di sicurezza e delega le interazioni con i provider esterni al modulo `Repository`.

4.2.3.6.1 Tipi

- **Application logic**

- `AddRepositoryDto`, rappresenta i dati forniti dall'utente per collegare una nuova repository a un workspace:
 - * `repositoryUrl`: string (l'URL esatto della repository su GitHub)
 - * `accessToken`: string (opzionale, rappresenta il Personal Access Token necessario qualora la repository sia privata)
- `GetRepositoriesQueryDto`, rappresenta i parametri di query per filtrare la lista delle repository collegate:
 - * `searchInput`: string (opzionale, utilizzato per filtrare i risultati per nome)
- `AccessTokenDto`, rappresenta il payload per l'aggiornamento delle credenziali di una repository già associata:
 - * `accessToken`: string (il nuovo token da memorizzare)
- `RepositoryResponseDto`, rappresenta la struttura dei dati di una repository restituita al client a seguito di un'interrogazione:
 - * `repositoryId`: string (ID univoco del repository)
 - * `ownerName`: string (Username del proprietario su GitHub)
 - * `name`: string (Nome del repository)
 - * `dateScan`: string (opzionale, data dell'ultima scansione)

- **Business logic**

- `RepositoryInfo`, rappresenta il Business Object scambiato tra i livelli di servizio. Contiene le informazioni aggregate della repository elaborate dal `RepositoryReaderService` e viene utilizzato dal service prima della conversione finale in DTO. A differenza dei moduli core, il modulo `WorkspaceRepository` non definisce Business Object (BO) complessi per le operazioni di scrittura o aggiornamento, in quanto i layer di servizio scambiano direttamente tipi primitivi come identificativi (ID) e URL, delegando la costruzione degli oggetti di dominio al modulo `Repository`.

- **Persistence logic**

A differenza del modulo `WorkspaceManager`, questo modulo non definisce un oggetto di persistenza autonomo (Data) né un proprio Schema Mongoose. Le sue operazioni di persistenza si limitano all'estrazione e alla manipolazione diretta del campo relazionale `repositories` (un array di stringhe che funge da elenco di chiavi esterne) all'interno del preesistente `WorkspaceDocument`.

4.2.3.6.2 Classi e Interfacce

• Interfacce

- IWorkspaceRepositoryService: interfaccia che definisce il contratto del service per l'orchestrazione dei legami tra workspace e repository.
 - * getRepositories(workspaceId: string, searchInput: string): RepositoryInfo[]
 - * addRepository(workspaceId: string, repositoryUrl: string, accessToken: string): void
 - * removeRepository(repositoryId: string, workspaceId: string): void
 - * updateToken(repositoryId: string, workspaceId: string, accessToken: string): void
- IWorkspaceRepositoryRepository: interfaccia che definisce unicamente le operazioni di accesso e modifica dei dati relazionali sul database.
 - * getRepositories(workspaceId: string): string[]
 - * addRepository(workspaceId: string, repositoryId: string): void
 - * removeRepository(repositoryId: string, workspaceId: string): void

• Classi

- WorkspaceRepositoryController: classe responsabile della gestione degli endpoint REST per le repository di un workspace.
 - * getRepositories(workspaceId: string, query: GetRepositoriesQueryDto): RepositoryResponseDto[]
Riceve la richiesta di lettura, estrae il parametro di ricerca opzionale e restituisce la lista mappata in DTO.
 - * addRepository(workspaceId: string, dto: AddRepositoryDto): void
Estrae i parametri dal DTO e invoca l'associazione di una nuova repository al workspace.
 - * removeRepository(workspaceId: string, repositoryId: string): void
Gestisce la richiesta di eliminazione del legame tra una specifica repository e il workspace.
 - * updateToken(workspaceId: string, repositoryId: string, dto: AccessTokenDto): void
Riceve il nuovo token di accesso e ne comanda l'aggiornamento.
- WorkspaceRepositoryService: classe che implementa la logica di business e agisce da orchestratore.
 - * getRepositories(workspaceId: string, searchInput: string): RepositoryInfo[]
Recupera gli identificativi delle repositories associate al workspace e delega al modulo Repository la lettura dei dettagli completi, applicando eventuali filtri di ricerca.
 - * addRepository(workspaceId: string, repositoryUrl: string, accessToken: string): void
Delega al modulo Repository la persistenza fisica del nuovo URL e del to-

ken, ricevendo un ID univoco che viene successivamente salvato nell'array del workspace.

- * `removeRepository(repositoryId: string, workspaceId: string): void`
Rimuove l'identificativo della repository dall'array del workspace specificato, interrompendo di fatto il legame logico.
- * `updateToken(repositoryId: string, workspaceId: string, accessToken: string): void`
Inoltra la richiesta di aggiornamento delle credenziali di accesso per una specifica repository associata al workspace.

– `WorkspaceRepositoryRepository`: implementa l'accesso ai dati tramite Mongoose.

- * `getRepositories(workspaceId: string): string[]`
Interroga il database estraendo esclusivamente l'elenco degli ID delle repository collegate al workspace fornito.
- * `addRepository(workspaceId: string, repositoryId: string): void`
Esegue un aggiornamento atomico sul documento `Workspace` per aggiungere il nuovo `repositoryId` all'array `repositories`.
- * `removeRepository(repositoryId: string, workspaceId: string): void`
Esegue un aggiornamento atomico sul documento `Workspace` per espellere il `repositoryId` dall'array.

- * repositoryId: string
- * ownerName: string
- * name: string
- * dateScan: string (Opzionale)
- * documentationScore: number (opzionale)
- * codeCoverage: number (opzionale)
- * cvss: number (opzionale)
- * accessToken: string (opzionale)
- RepositoryScores: rappresenta i punteggi e le metriche di qualità elaborati e passati a livello di service per l'aggiornamento storico:
 - * dateScan: Date (Opzionale)
 - * documentationScore: number (opzionale)
 - * codeCoverage: number (opzionale)
 - * cvss: number (opzionale)

• Persistence logic

- RepositoryEntity, appresenta l'entità grezza estratta o da salvare a livello di persistenza nel database:
 - * repositoryId: string
 - * ownerName: string
 - * name: string
 - * dateScan: Date (Opzionale)
 - * documentationScore: number (opzionale)
 - * codeCoverage: number (opzionale)
 - * cvss: number (opzionale)
 - * accessToken: string (opzionale)

4.2.3.7.2 Classi e Interfacce

• Interfacce

- IRepositoryService: definisce il contratto esposto dal service principale verso il controller.
 - * getRepository(repositoryId: string): RepositoryInfo
 - * getBranches(repositoryId: string): string[]
- IRepositoryFindRepository: definisce le operazioni di lettura multipla e singola dal database.
 - * getRepositories(repositoryIds: string[], searchInput: string): RepositoryEntity[]
 - * getRepository(repositoryId: string): RepositoryEntity
- IRepositoryPersistRepository: definisce le operazioni di scrittura e aggiornamento base sul database.

- * addRepository(ownerName: string, name: string, accessToken: string): string
 - * updateToken(repositoryId: string, accessToken: string): void
- IRepositoryScoreRepository: definisce le operazioni di aggiornamento delle metriche sul database.
 - * updateScores(repositoryId: string, scores: RepositoryScores): void
- IGitHubBranchesRepository: definisce il contratto per il recupero dei branch tramite API esterne.
 - * getBranches(ownerName: string, name: string, accessToken: string): string[]
- IGitHubAccessRepository: definisce il contratto per la verifica dei permessi d'accesso su GitHub.
 - * verifyAccess(ownerName: string, name: string, accessToken: string): void
- IRepositoryReader: definisce il contratto per la lettura aggregata di repository.
 - * getRepositories(repositoryIds: string[], searchInput: string): RepositoryInfo[]
- IRepositoryWriter: definisce il contratto per la creazione e l'aggiornamento tecnico delle repository.
 - * addRepository(repositoryUrl: string, accessToken: string): string
 - * updateToken(repositoryId: string, accessToken: string): void

• Classi

- RepositoryController: è il punto di contatto con l'esterno che riceve le richieste HTTP relative alle repository.
 - * getRepository(repositoryId: string): riceve la richiesta di visualizzazione di una singola repository, chiama il servizio interno per recuperare tutti i dati e li impacchetta in un formato pronto per essere inviato al client.
 - * getBranches(repositoryId: string): gestisce la richiesta per visualizzare l'elenco dei branch di un progetto, facendosi restituire la lista aggiornata per mostrarla all'utente.
- RepositoryService: classe che implementa IRepositoryService per la consultazione base.
 - * getBranches(repositoryId): recupera le credenziali salvate nel database e le utilizza per contattare GitHub in tempo reale, ottenendo così la lista dei branch sempre aggiornata.
 - * getRepository(repositoryId: string): cerca la repository nel database e formatta i dati affinché siano utilizzabili dal resto dell'applicazione.
- RepositoryRepository: è il componente che parla direttamente col database, mascherandone la complessità al resto del sistema.

- * `getRepositories(repositoryIds: string[], searchInput: string)`: estrae un gruppo di repository, è progettato per essere "tollerante": se c'è una ricerca testuale, ignora le maiuscole/minuscole e "disinnesca" eventuali caratteri speciali inseriti dall'utente per evitare malfunzionamenti o errori nel motore di ricerca.
- * `getRepository(repositoryId: string)`: cerca una repository specifica. Se questa non esiste nel database, blocca l'operazione e segnala un errore di "Risorsa non trovata".
- * `addRepository(owner: string, name:string, accessToken:string)`: gestisce l'aggiunta in modo intelligente per evitare duplicati. Prima verifica se un progetto con quello stesso nome e proprietario esiste già: se è nuovo, lo crea; se esiste già, si limita ad aggiornare le sue credenziali, mantenendo intatti gli altri dati.
- * `updateToken(repositoryId: string, accessToken: string)`: modifica esclusivamente il dato relativo al token di accesso di un progetto, assicurandosi prima che quel progetto esista.
- * `updateScores(repositoryId: string, scores: RepositoryScores)`: aggiorna tutte le metriche e i punteggi di qualità di una repository al termine di un'analisi.
- * `toRepositoryEntity()`: è un metodo di supporto interno che serve a pulire i dati grezzi appena estratti da MongoDB per trasformarli in un formato semplice e standardizzato, in modo che il resto dell'applicazione non debba preoccuparsi di come il database salva fisicamente i documenti.
- `RepositoryScoreService`: implementa `IRepositoryScoreWriter` per la gestione dei risultati delle analisi AI.
 - * `updateScores(repositoryId: string, scores: RepositoryScores)`: riceve le metriche calcolate dopo una scansione di sicurezza e le salva permanentemente nel profilo della repository.
- `IGitHubBranchesRepository`: definisce il contratto per il recupero dei branch tramite API esterne.
 - * `getBranches(ownerName: string, name: string, accessToken: string): string[]`
- `GitHubRepository`: è il traduttore che permette all'applicazione di comunicare con i server esterni di GitHub.
 - * `getBranches(ownerName: string, name: string, accessToken: string)`: effettua la connessione a GitHub per scaricare la lista dei branch. Inoltre, cattura gli eventuali problemi di rete e li traduce in messaggi di errore chiari e gestibili dall'applicazione.
 - * `verifyAccess(ownerName: string, name: string, accessToken: string)`: esegue un "ping" di prova verso il progetto su GitHub, serve esclusivamente a confermare che la chiave fornita dall'utente abbia i permessi corretti per leggere il codice sorgente prima di procedere con le scansioni.
- `RepositoryReaderService`: implementa `IRepositoryReader` per gestire le letture massive di gruppo, utili quando altri moduli hanno bisogno di dati aggregati.

- * `getRepositories(repositoryIds: string[], searchInput: string)`: riceve un elenco di identificativi e restituisce i dettagli di tutte le repository corrispondenti. Se l'utente ha inserito del testo nella barra di ricerca, filtra i risultati per mostrare solo i progetti il cui nome corrisponde alla ricerca.
- `RepositoryWriterService`: implementa `IRepositoryWriter` per controllare la creazione e la modifica dei dati tecnici.
 - * `addRepository(repositoryUrl: string, accessToken: string)`: riceve il link GitHub inserito dall'utente, prima di salvare qualsiasi cosa contatta GitHub per verificare che il link esista e che le credenziali fornite siano corrette. Solo dopo questa validazione, salva il progetto nel database.
 - * `updateToken(repositoryId: string, accessToken: string)`: sostituisce la vecchia chiave di accesso di una repository con una nuova, validandola prima su GitHub per accertarsi che funzioni.

4.2.3.8 Report

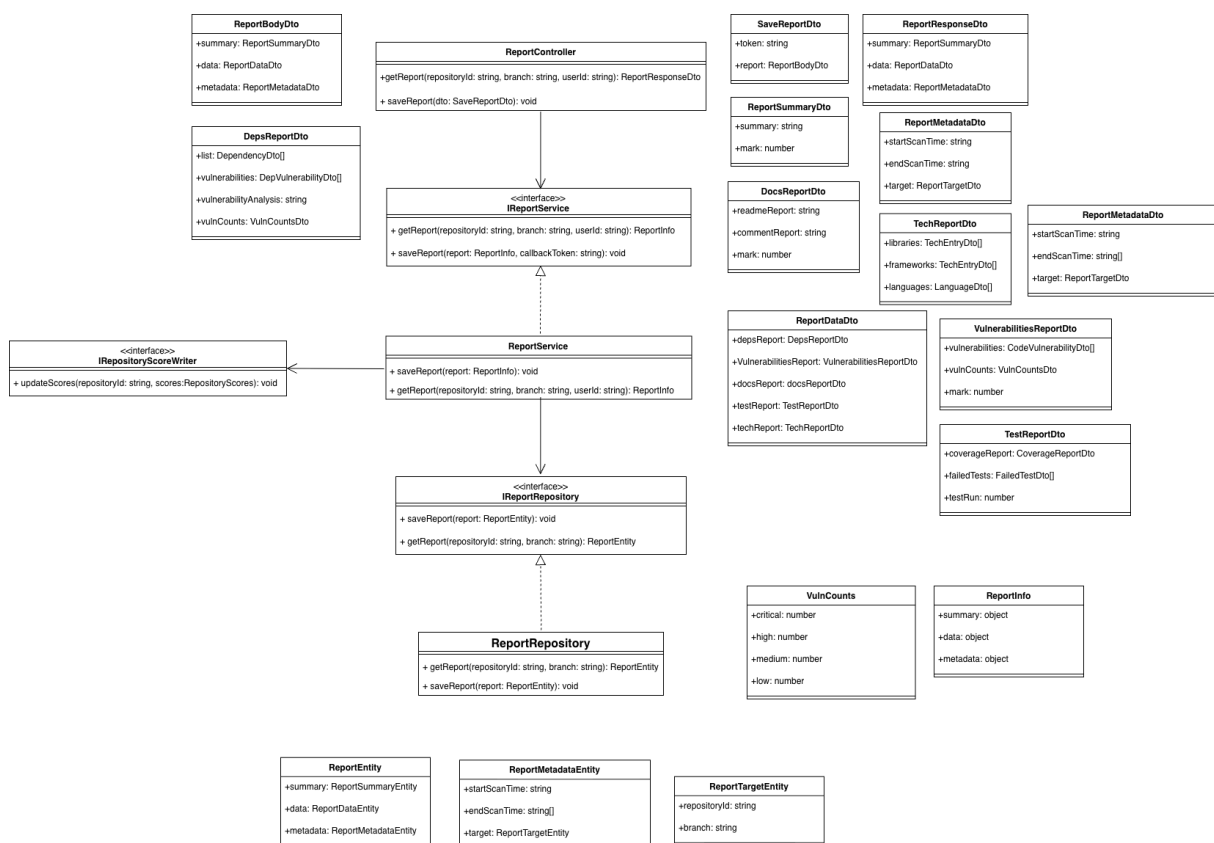


Figura 16: UML Report

Il modulo Report costituisce il punto di arrivo della pipeline di analisi di CodeGuardian. La sua responsabilità principale è la gestione del ciclo di vita dei dati risultanti dalle scansioni:

ricezione sicura dai container di analisi, persistenza strutturata su database, applicazione di logiche di aggregazione post-scansione e distribuzione controllata dei risultati agli utenti finali. Il modulo implementa un rigoroso sistema di controllo degli accessi basato sui ruoli, garantendo che la visibilità dei dettagli tecnici sia coerente con le responsabilità dell'utente all'interno del workspace.

4.2.3.8.1 Tipi

- **Application logic**

- `SaveReportDto`, rappresenta i dati inviati dal container di scansione per il salvataggio di un nuovo report.
 - * `token`: string
 - * `report`: `ReportBodyDto`
- `ReportBodyDto`, rappresenta il corpo strutturato del report inviato dall'AI.
 - * `summary`: `ReportSummaryDto`
 - * `data`: `ReportDataDto`
 - * `metadata`: `ReportMetadataDto`
- `ReportResponseDto`, rappresenta la risposta restituita al client dopo aver richiesto un report.
 - * `summary`: `ReportSummaryDto`
 - * `data`: `ReportDataDto`
 - * `metadata`: `ReportMetadataDto`
- `ReportSummaryDto`, rappresenta i dati relativi al riepilogo testuale e al voto complessivo.
 - * `summary`: string
 - * `mark`: number
- `ReportDataDto`, rappresenta i dati centrali del report suddivisi per aree di analisi.
 - * `depsReport`: `DepsReportDto`
 - * `vulnerabilitiesReport`: `VulnerabilitiesReportDto`
 - * `docsReport`: `DocsReportDto`
 - * `testReport`: `TestReportDto`
 - * `techReport`: `TechReportDto`
- `DepsReportDto`, rappresenta i dati relativi alle dipendenze esterne e alle loro vulnerabilità.
 - * `list`: `DependencyDto[]`
 - * `vulnerabilities`: `DepVulnerabilityDto[]`
 - * `vulnerabilityAnalysis`: string
 - * `vulnCounts`: `VulnCountsDto`
- `VulnerabilitiesReportDto`, rappresenta i dati relativi alle vulnerabilità individuate nel codice sorgente.

- * vulnerabilities: CodeVulnerabilityDto[]
- * mark: number
- * vulnCounts: VulnCountsDto
- DocsReportDto, rappresenta i dati relativi all'analisi della documentazione e dei commenti.
 - * readmeReport: string
 - * mark: number
 - * commentReport: string
- TestReportDto, rappresenta i dati relativi alla code coverage e all'esecuzione dei test.
 - * coverageReport: CoverageReportDto
 - * failedTests: FailedTestDto[]
 - * testsRun: number
- TechReportDto, rappresenta i dati relativi ai framework, alle librerie e ai linguaggi rilevati.
 - * libraries: TechEntryDto[]
 - * frameworks: TechEntryDto[]
 - * languages: LanguageDto[]
- ReportMetadataDto, rappresenta i dati relativi ai tempi e al bersaglio della scansione.
 - * startScanTime: string
 - * endScanTime: string
 - * target: ReportTargetDto

• Business logic

- ReportInfo, rappresenta il modello completo di un report utilizzato a livello di service
 - * summary: object
 - * data: object
 - * metadata: object
- VulnCounts: rappresenta i dati aggregati del conteggio delle vulnerabilità per gravità a livello di service.
 - * critical: number
 - * high: number
 - * medium: number
 - * low: number

• Persistence logic

- ReportEntity, rappresenta i dati completi di un report a livello di persistenza.
 - * summary: ReportSummaryEntity
 - * data: ReportDataEntity

- * metadata: ReportMetadataEntity
- ReportMetadataEntity, rappresenta i dati dei metadati associati a un report a livello di persistenza.
 - * startScanTime: string
 - * endScanTime: string
 - * target: ReportTargetEntity
- ReportTargetEntity, rappresenta i dati identificativi del bersaglio del report a livello di persistenza.
 - * repositoryId: string
 - * branch: string

4.2.3.8.2 Classi e Interfacce

• Interfacce

- IReportService: definisce il contratto per la logica di business relativa alla gestione e all'elaborazione dei report.
 - * saveReport(report: ReportInfo, callbackToken: string): void
 - * getReport(repositoryId: string, branch: string, userId: string): ReportInfo
- IReportRepository: definisce le operazioni per l'accesso ai dati e la persistenza dei report nel database.
 - * saveReport(report: ReportEntity): void
 - * getReport(repositoryId: string, branch: string): ReportEntity

• Classi

- ReportController: classe responsabile della gestione degli endpoint REST per la ricezione e la consultazione dei report.
 - * saveReport(dto: SaveReportDto): intercetta il payload al termine di una scansione, prima di procedere valida crittograficamente il token JWT per autenticare il container mittente, estrae i riferimenti della repository e inoltra i dati al layer di servizio.
 - * getReport(repositoryId: string, branch: string, userId: string): gestisce la richiesta GET dell'utente, delega il recupero al service, passandogli l'ID dell'utente per permettere l'applicazione delle regole di visibilità e restituisce il risultato formattato per il frontend.
- ReportService: implementa la logica di dominio principale, occupandosi del ciclo di vita post-analisi e dei controlli di sicurezza.
 - * saveReport(report: ReportInfo): invia il report al database per la persistenza e segnala al sistema che la scansione è "Completata". Se l'analisi riguarda il ramo principale del codice, si occupa anche di inviare i nuovi voti di sicurezza al modulo Repository per aggiornare lo storico del progetto.

- * `getReport(repositoryId: string, branch: string, userId: string)`: recupera il report grezzo dall'archivio e calcola dinamicamente i totali delle vulnerabilità per gravità; inoltre, applica le regole di censura aziendale.
- `ReportRepository`: incapsula la logica di interazione con MongoDB, garantendo l'isolamento del framework Mongoose dal resto dell'applicazione.
 - * `saveReport(report: ReportEntity)`: inserisce fisicamente il documento nel database, utilizza una logica di aggiornamento intelligente (se esiste già un report per quella specifica repository e per quel branch, lo sovrascrive; altrimenti ne crea uno nuovo, evitando così di generare doppioni).
 - * `getReport(repositoryId: string, branch: string)`: effettua una ricerca nel database filtrando i documenti per repository e branch, ordina automaticamente i risultati in modo decrescente in base alla data di creazione per garantire di estrarre e restituire sempre l'ultima scansione effettuata.

4.2.3.9 Scan

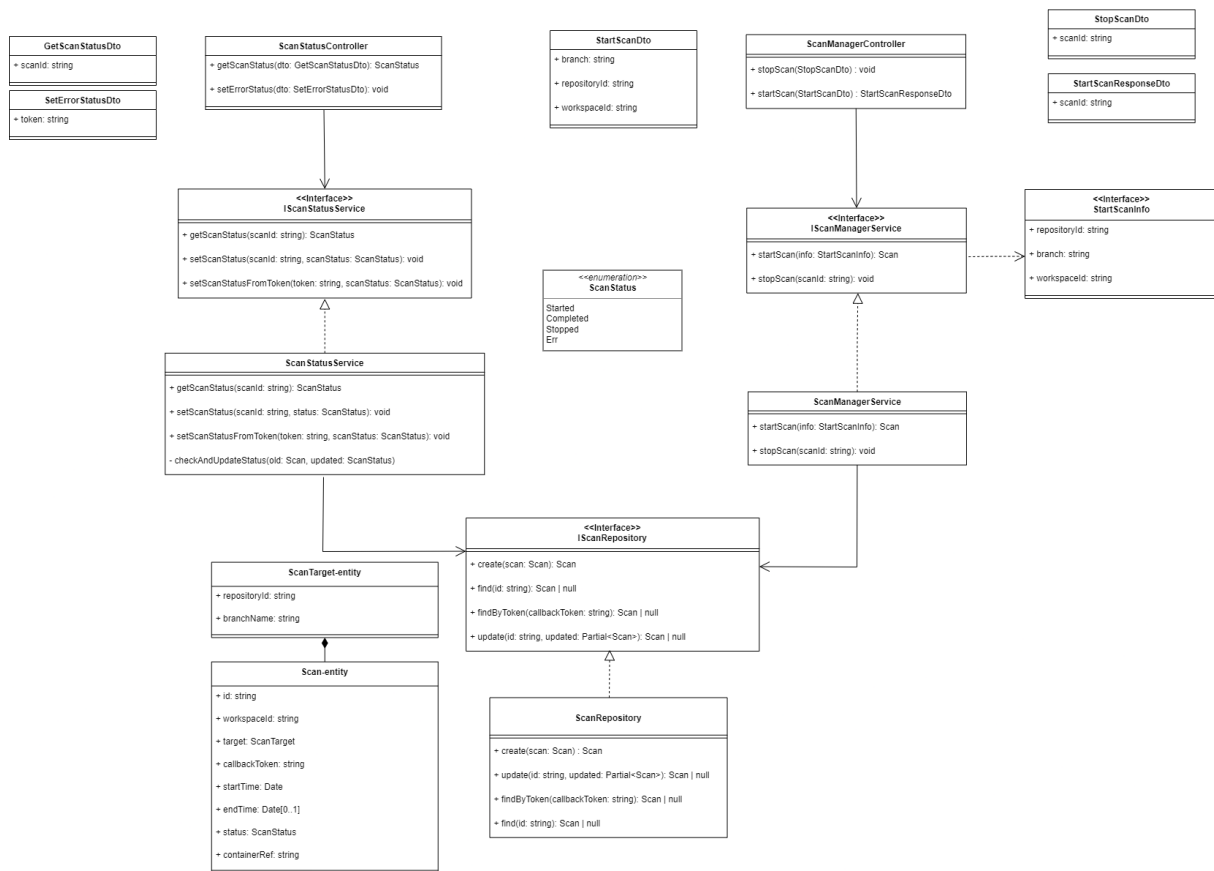


Figura 17: UML Modulo Scan

Il modulo Scan è il motore di orchestrazione centrale dell'applicazione. È responsabile dell'intero ciclo di vita delle analisi di sicurezza: dal recupero dei dati del repository target, al provisioning dinamico dei container di esecuzione su infrastruttura cloud (AWS ECS Fargate), fino all'interruzione manuale. Il modulo monitora in tempo reale lo stato di avanzamento tramite webhook sicuri (JWT) e gestisce la persistenza dei metadati a database, proteggendo gli endpoint con un rigoroso layer di autenticazione.

4.2.3.9.1 Modelli e Schemi (Database)

• Schemi Mongoose

- **ScanSchemaClass**: Definisce la struttura della collezione MongoDB utilizzando i decorator nativi di NestJS (@Schema, @Prop) per la validazione a livello di Object Document Mapper (ODM).
- * **Timestamps**: Utilizza la configurazione { timestamps: true, collection: 'scans' } per delegare a Mongoose la generazione e l'aggiornamento automatico dei campi createdAt e updatedAt.

- * **Indici:** Definisce un indice composto fondamentale per l'ottimizzazione delle performance di ricerca: `ScanSchema.index({ 'target.repositoryId': 1, 'target.branchName': 1 }, { name: 'idx_repo_branch' })`.
- `ScanTargetSchemaClass`: Rappresenta lo schema del sotto-documento incorporato in `ScanSchemaClass`, definendo le proprietà `repositoryId` e `branchName`.

4.2.3.9.2 Classi e Interfacce

• Interfacce

- `IScanRepository`
 - * + `create(scan: Scan): Promise<Scan>`
 - * + `find(id: string): Promise<Scan | null>`
 - * + `findByToken(callbackToken: string): Promise<Scan | null>`
 - * + `update(id: string, updated: Partial<Scan>): Promise<Scan | null>`

• Classi

- `ScanModule` (*Modulo di configurazione principale*)
 - * **Descrizione:** Modulo di orchestrazione che definisce il perimetro del dominio `Scan` all'interno di `NestJS`.
 - * **Configurazione Dati:** Registra i modelli `Mongoose` nel contesto dell'applicazione tramite `MongooseModule.forFeature`.
 - * **Configurazione Sicurezza:** Cabla dinamicamente il modulo `JWT` (`JwtModule.registerAsync`) iniettando il `ConfigService` per recuperare i segreti crittografici e i tempi di scadenza in modo sicuro dalle variabili d'ambiente.
 - * **Provider:** Lega le interfacce alle relative implementazioni concrete (es. `IScanRepository` a `ScanRepository`) per consentire la `Dependency Injection`.
- `ScanRepository`
 - * **Proprietà:**
 - – `scanModel: Model<Scan>`, oggetto `Mongoose` per la gestione della collezione delle scansioni, iniettato tramite `dependency injection`.
 - * **Operazioni**
 - + `create(scan: Scan): Promise<Scan>`, utilizza il metodo `create` di `Mongoose` per inserire un nuovo documento a database.
 - + `find(id: string): Promise<Scan | null>`, utilizza il metodo `findOne` di `Mongoose` concatenato a `.lean().exec()` per restituire il documento ottimizzato sotto forma di `Plain Old JavaScript Object (POJO)`, riducendo l'overhead.
 - + `findByToken(callbackToken: string): Promise<Scan | null>`, recupera il documento ottimizzato ricercando per corrispondenza esatta del token `JWT`.
 - + `update(id: string, updated: Partial<Scan>): Promise<Scan | null>`, utilizza il metodo `findOneAndUpdate` di `Mongoose` per applicare modifiche

in modo atomico, richiedendo esplicitamente di restituire il documento post-modifica tramite l'opzione `{ returnDocument: 'after' }`.

4.2.3.10 Scan Manager

4.2.3.10.1 Tipi

- **Application logic**

- StartScanDto, rappresenta i dati forniti dal client per avviare una nuova scansione:
 - * workspaceId: string
 - * repositoryId: string
 - * branch: string
- StartScanResponseDto, rappresenta la risposta restituita al client dopo l'avvio con successo di un task:
 - * scanId: string
- StopScanDto, rappresenta i dati per richiedere l'interruzione di una scansione:
 - * scanId: string

- **Business logic**

- StartScanInfo, definisce i dati passati dal Controller al Service per avviare l'orchestrazione a livello di service:
 - * workspaceId: string
 - * repositoryId: string
 - * branch: string

- **Persistence logic**

- Scan, rappresenta il modello completo della scansione a livello di persistenza:
 - * id: string
 - * workspaceId: string
 - * target: ScanTarget
 - * callbackToken: string
 - * startTime: Date
 - * endTime: Date (opzionale)
 - * status: ScanStatus
 - * containerRef: string
- ScanTarget, rappresenta l'obiettivo della scansione come sotto-documento a livello di persistenza:
 - * repositoryId: string
 - * branchName: string

4.2.3.10.2 Classi e Interfacce

- **Interfacce**

- IScanManagerService

```
* + startScan(info: StartScanInfo): Promise<Scan>
* + stopScan(scanId: string): Promise<void>
```

• Classi

– ScanManagerController

* Proprietà:

- - scanManagerService: IScanManagerService, istanza del servizio per l'orchestrazione delle scansioni, iniettata tramite dependency injection

* Operazioni

- + startScan(startScanDto: StartScanDto): Promise<StartScanResponseDto>, riceve i dati per avviare una scansione e chiama il metodo startScan del servizio. Se ha successo, ritorna un DTO contenente l'ID univoco della scansione con status HTTP 201 (CREATED).
- + stopScan(stopScanDto: StopScanDto): Promise<void>, riceve l'oggetto rotta mappandolo e validandolo sul StopScanDto, estrae l'ID della scansione e chiama il metodo stopScan del servizio per forzarne l'interruzione. Se ha successo ritorna una risposta vuota con status HTTP 204 (NO CONTENT).

– ScanManagerService

* Proprietà:

- - scanRepository: IScanRepository, istanza del repository per la persistenza delle scansioni, iniettata tramite dependency injection
- - repositoryReader: IRepositoryReader, istanza del servizio per recuperare i dettagli del repository, iniettata tramite dependency injection
- - configService: ConfigService, istanza per la lettura delle variabili d'ambiente, iniettata tramite dependency injection
- - jwtService: JwtService, istanza per la generazione e firma dei token, iniettata tramite dependency injection
- - logger: Logger, istanza nativa di NestJS utilizzata per il tracciamento interno delle operazioni (es. logging dell'ARN del task ECS)

* Operazioni

- + startScan(info: StartScanInfo): Promise<Scan>, recupera i dati del repository tramite repositoryReader, genera un ID univoco e crea un token JWT contenente le credenziali target e gli URL di callback. Successivamente, istanzia un client AWS ECS, invia un comando per avviare un task Fargate iniettando il token come variabile d'ambiente, e infine salva l'entità a database tramite scanRepository.create associandovi l'ARN del container. In caso di fallimento nell'avvio del task su ECS, solleva una InternalServerErrorException.
- + stopScan(scanId: string): Promise<void>, recupera la scansione tramite scanRepository.find (sollevando una NotFoundException se inesistente). Verifica che sia in stato "Started" (altrimenti solleva una ConflictException),

invia un comando di stop ad AWS ECS utilizzando il `containerRef`, e aggiorna lo stato a database tramite `scanRepository.update`.

4.2.3.11 Scan Status

4.2.3.11.1 Tipi

- **Application logic**

- `GetScanStatusDto`, rappresenta i dati necessari per interrogare lo stato di una scansione:
 - * `scanId`: string
- `SetErrorStatusDto`, rappresenta il payload ricevuto dal container in caso di fallimento:
 - * `token`: string

- **Enumerazioni**

- `ScanStatus`, rappresenta lo stato di avanzamento di una scansione:
 - * `Started`
 - * `Completed`
 - * `Stopped`
 - * `Err`

4.2.3.11.2 Classi e Interfacce

- **Interfacce**

- `IScanStatusService`
 - * + `getScanStatus(scanId: string): Promise<ScanStatus>`
 - * + `setScanStatus(scanId: string, scanStatus: ScanStatus): Promise<void>`
 - * + `setScanStatusFromToken(token: string, scanStatus: ScanStatus): Promise<void>`

- **Classi**

- `ScanStatusController`
 - * **Proprietà:**
 - - `scanStatusService`: `IScanStatusService`, istanza del servizio per la gestione dello stato, iniettata tramite dependency injection
 - * **Operazioni** (*Nota: Il controller è montato sulla rotta base `/scan/:scanId/status`, definendo lo scope per l'ID della scansione su tutti i metodi*)
 - + `getScanStatus(dto: GetScanStatusDto): Promise<ScanStatus>`, endpoint protetto da `JwtAuthGuard`. Riceve l'ID della scansione tramite il parametro di rotta e chiama il metodo `getScanStatus` del servizio per restituire lo stato corrente al client.
 - + `setErrorStatus(dto: SetErrorStatusDto): Promise<void>`, endpoint protetto da `ScanAuthGuard` per ricevere i webhook dai container. Riceve il token dal body e chiama `setScanStatusFromToken` forzando lo stato `Err`.

La chiamata è avvolta in un blocco `try/catch` che intercetta eventuali eccezioni del Service (`NotFoundException` o `ConflictException`) e rilancia proattivamente una `InternalServerErrorException`.

– `ScanStatusService`

* Proprietà:

- - `scanRepository`: `IScanRepository`, istanza del repository per la persistenza delle scansioni, iniettata tramite `dependency injection`

* Operazioni

- + `getScanStatus(scanId: string): Promise<ScanStatus>`, cerca la scansione tramite `scanRepository.find`. Se esiste ne restituisce lo stato, altrimenti solleva una `NotFoundException`.
- + `setScanStatus(scanId: string, status: ScanStatus): Promise<void>`, recupera la scansione tramite ID (sollevando `NotFoundException` in caso di assenza) e chiama il metodo privato `checkAndUpdateStatus` per validare e applicare la transizione di stato.
- + `setScanStatusFromToken(token: string, scanStatus: ScanStatus): Promise<void>`, recupera la scansione a database interrogando direttamente il token JWT tramite `scanRepository.findByToken`, per poi delegare l'aggiornamento a `checkAndUpdateStatus`.
- - `checkAndUpdateStatus(old: Scan, updated: ScanStatus): Promise<void>`, logica privata che assicura che lo stato possa essere modificato solo se attualmente in "Started" (in caso contrario solleva una `ConflictException`). Aggiorna il database tramite `scanRepository.update` registrando anche il timestamp di fine (`endTime`).

4.2.3.12 Scan Auth

4.2.3.12.1 Tipi

- **Application logic**

- `ContainerRequest`, rappresenta la struttura attesa della richiesta HTTP intercettata dalla guardia:
 - * `body: { token: string }`

4.2.3.12.2 Classi e Interfacce

- **Classi**

- `ScanAuthGuard` (*Implementa l'interfaccia `CanActivate` nativa di `@nestjs/common`*)
 - * Proprietà:
 - - `jwtService: JwtService`, istanza per la validazione crittografica dei token, iniettata tramite `dependency injection`
 - * Operazioni
 - + `canActivate(context: ExecutionContext): Promise<boolean>`, estrae l'oggetto `request` dal contesto di esecuzione di NestJS, recupera il `token` dal `body` e ne verifica la validità tramite `jwtService.verifyAsync()`. Se il token è valido permette l'accesso alla rotta ritornando `true`, altrimenti solleva una `UnauthorizedException`.

4.3 Container

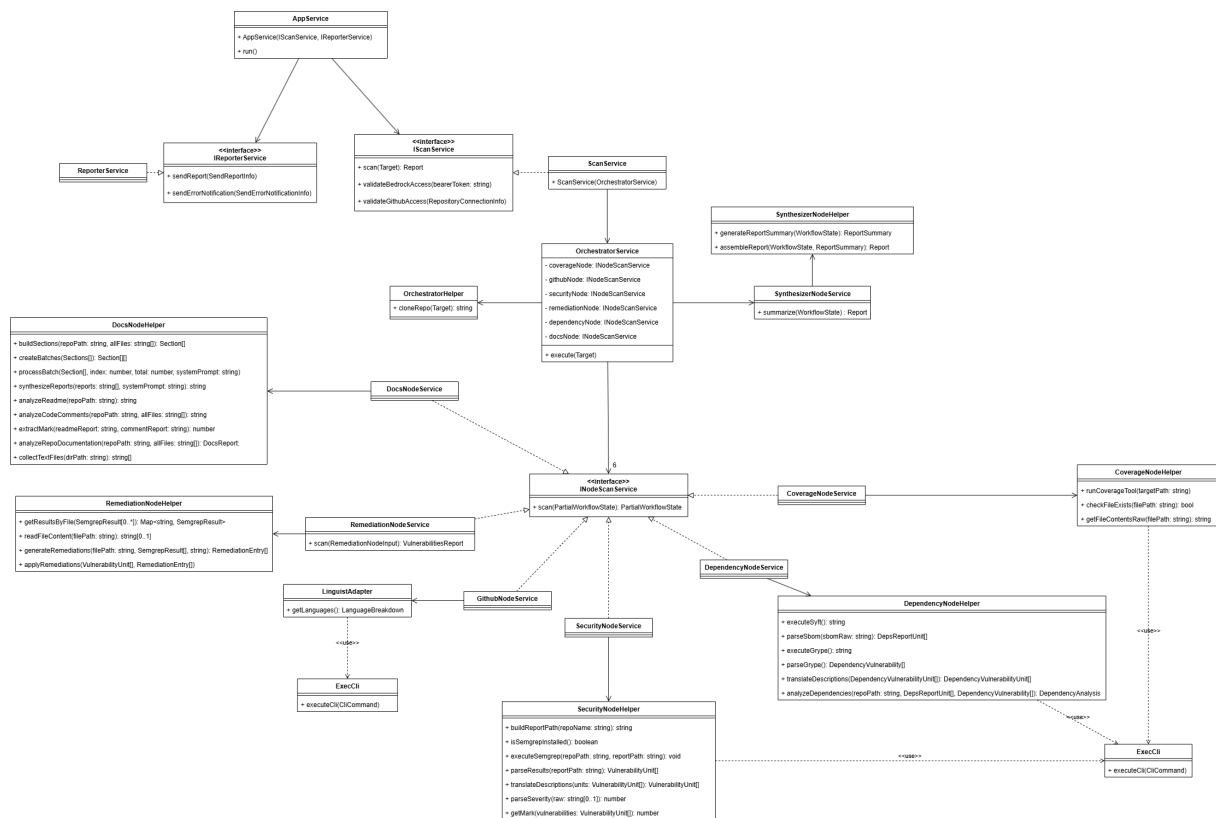


Figura 18: UML Container (Scanner)

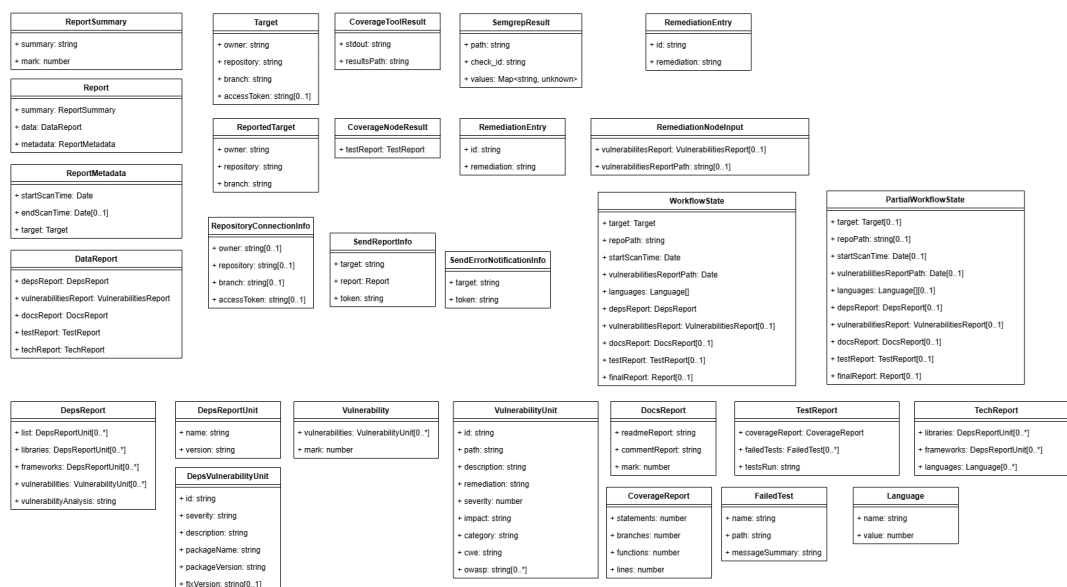


Figura 19: UML Tipi Container (Scanner)

Il **Container** è il componente autonomo del sistema CodeGuardian incaricato dell'esecuzione materiale dell'analisi di un repository. Viene istanziato dal backend come task isolato su *AWS ECS (Fargate)* ogni volta che una nuova scansione viene avviata, e comunica con il backend tramite webhook al termine dell'elaborazione.

Il Container è un'applicazione **NestJS standalone**, priva di un server HTTP in ascolto: il suo unico punto di ingresso è il metodo `AppService.run()`, invocato direttamente all'avvio del processo. Tutta la configurazione necessaria (credenziali GitHub, token Bedrock, URL di callback) viene iniettata tramite la variabile d'ambiente `REPORT_CALLBACK_TOKEN`, un JWT firmato generato dal backend nel momento in cui il task viene lanciato.

Il flusso di esecuzione del Container segue le seguenti fasi:

1. Decodifica del `REPORT_CALLBACK_TOKEN` e validazione dei dati di configurazione;
2. Validazione della raggiungibilità del repository GitHub e delle credenziali AWS Bedrock;
3. Avvio della pipeline di analisi tramite `ScanService`, che delega l'esecuzione al grafo LangGraph orchestrato da `OrchestratorService`;
4. Invio del report prodotto all'URL di successo, oppure notifica dell'errore all'URL di fallimento, tramite `ReporterService`.

L'architettura interna è organizzata in tre sezioni principali:

- **AppModule**: modulo radice che aggrega gli altri moduli e configura le dipendenze globali;
- **ScanModule**: incapsula la logica di scansione e delega l'esecuzione del grafo di analisi all'`OrchestratorModule`;
- **ReporterModule**: gestisce la comunicazione dei risultati verso il backend al termine dell'elaborazione.

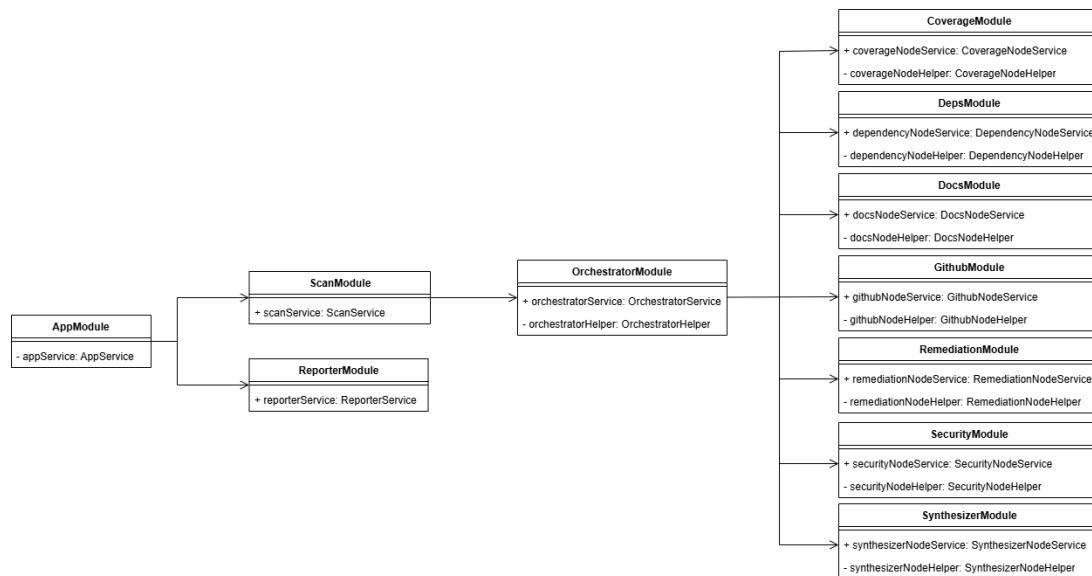


Figura 20: Moduli del container di scansione

4.3.1 Design pattern

All'interno del container non sono stati utilizzati pattern particolari, salvo il *Dependency Injection*, di cui si parlerà nella prossima sezione.

Per questioni di praticità in fase di testing è stato scelto di utilizzare classi *helper* non esportate, dunque private, per ciascun modulo, qualora necessario.

Interfacce sono state utilizzate nel riferimento ai vari nodi da parte dell'orchestratore, avendo in mente un'idea di *Strategy*. Ciò nonostante, mancando nodi alternativi a quelli attualmente implementati all'interno del grafico, tale pattern non è di fatto stato implementato, e l'interfaccia è stata resa quanto più lasca possibile seguendo le firme richieste da LangGraph.

4.3.1.1 Dependency Injection

Il pattern *Dependency Injection*, gestito direttamente da NestJS, è stato ampiamente utilizzato all'interno del container. Ciò ha permesso di diminuire il livello di dipendenza tra le classi e facilitare la fase di test.

4.3.2 AppModule

AppModule contiene come unico *provider* il AppService, che funge da *entry point* dell'applicazione. Non ci sono moduli che dipendono da AppModule, che ha dipendenze verso ReporterModule e ScanModule.

4.3.2.1 AppService

AppService espone l'metodo `run()`, il cui ruolo è verificare l'esistenza dei dati forniti dal backend, quindi delegarne la validazione e il lancio della scansione a una classe che implementa l'interfaccia `IScanService`. Al termine della scansione delega la trasmissione del report, o

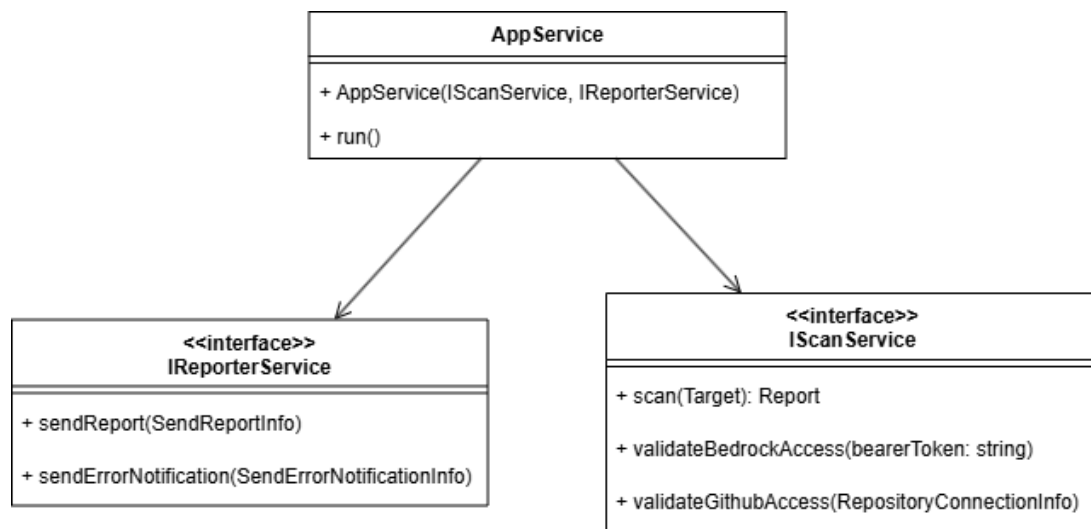


Figura 21: UML AppModule

dell'errore, in caso di errore fatale durante l'esecuzione della scansione, a una classe che implementa `IReporterService`.

4.3.3 ReporterModule

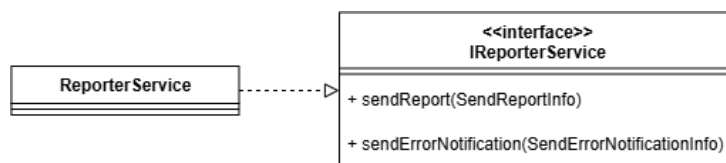


Figura 22: UML ReporterModule

`ReporterModule` contiene come unico *provider* il `ReporterService`, il cui ruolo è contattare il backend al termine della scansione.

4.3.3.1 ReporterService

`ReporterService` implementa l'interfaccia `IReporterService` ed espone due metodi:

- `sendReport(SendReportInfo): void`, che si occupa di trasmettere il report di una scansione all'indirizzo specificato nel parametro
- `sendErrorNotification(SendErrorNotificationInfo): void`, che si occupa di trasmettere una notifica di errore all'indirizzo specificato nel parametro

4.3.4 ScanModule

`ScanModule` contiene come unico *provider* lo `ScanService` ed è l'interfaccia per il lancio della scansione.

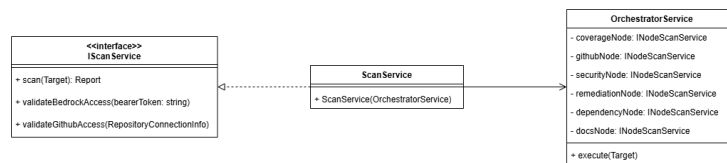


Figura 23: UML ScanModule

4.3.4.1 ScanService

ScanService implementa l'interfaccia IScanService, quindi espone 3 metodi:

- `scan(Target): Report`, che delega il lancio della scansione all'`OrchestratorService` iniettato nel costruttore, e ne restituisce il report
- `validateBedrockAccess(bearerToken: string): void`, che valida la connessione verso AWS Bedrock e lancia un'eccezione se non riesce a connettersi al servizio
- `validateGithubAccess(RepositoryConnectionInfo): void`, che valida la connessione verso il repository bersaglio della scansione e lancia un'eccezione se non riesce a connettersi ad esso

4.3.5 OrchestratorModule

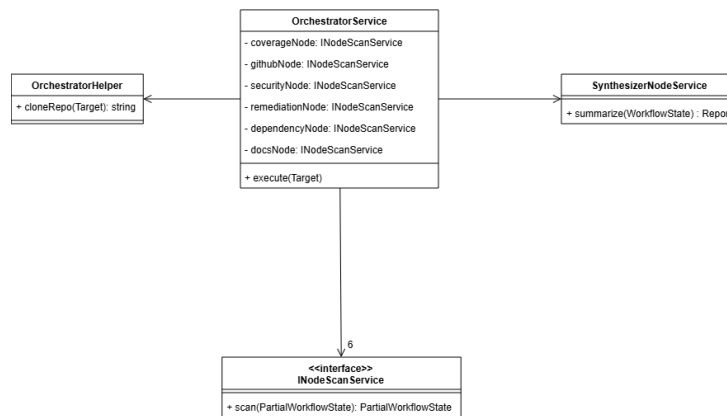


Figura 24: UML OrchestratorModule

OrchestratorModule contiene due *provider*: `OrchestratorService` e `OrchestratorHelper`. Il modulo importa i moduli di tutti i nodi di analisi e il `SynthesizerModule`, ed esporta l'`OrchestratorService`.

4.3.5.1 OrchestratorService

`OrchestratorService` è responsabile della costruzione e dell'esecuzione del grafo di scansione. Riceve le implementazioni di `INodeScanService` dei nodi di analisi tramite iniezione per token, e `SynthesizerNodeService` tramite iniezione diretta.

- `execute(target: Target): Promise<Report | undefined>`: richiede un `Target` valido; costruisce ed esegue il grafo di analisi in cui i nodi di copertura, dipendenze, linguaggi, sicurezza e documentazione operano in parallelo, seguiti dal nodo di remediation e dal

sintetizzatore; restituisce il Report finale. Se il sintetizzatore non produce un report, viene sollevata un'eccezione di tipo `Error`.

4.3.5.2 OrchestratorHelper

`OrchestratorHelper` fornisce all'`OrchestratorService` l'operazione di acquisizione locale del repository da analizzare.

- `cloneRepo(target: Target): Promise<string>`: richiede un `Target` valido con owner, repository, branch e token di accesso; restituisce il percorso locale al repository. Se il repository è già presente localmente, restituisce il percorso senza eseguire nuovamente il clone. In caso di errore durante il clone, viene sollevata un'eccezione di tipo `Error`.

4.3.6 Nodo sintetizzatore

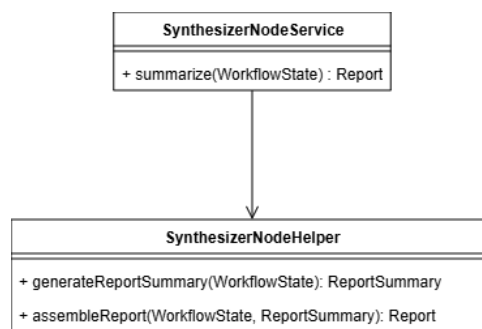


Figura 25: UML SynthesizerModule

`SynthesizerModule` contiene due *provider*: `SynthesizerNodeService` e `SynthesizerNodeHelper`. Il modulo esporta il `SynthesizerNodeService`, iniettato nel nodo orchestratore del grafo.

4.3.6.1 SynthesizerNodeService

`SynthesizerNodeService` è responsabile dell'aggregazione dei risultati prodotti dai nodi di analisi e della generazione del report finale.

- `summarize(state: WorkflowState): Promise<Report>`: richiede uno stato del grafo contenente i risultati dei nodi di copertura, sicurezza, dipendenze, documentazione e linguaggi; restituisce un `Report` comprensivo di riassunto, punteggio complessivo, dati aggregati e metadati della scansione; non solleva eccezioni.

4.3.6.2 SynthesizerNodeHelper

`SynthesizerNodeHelper` fornisce al `SynthesizerNodeService` le operazioni necessarie alla produzione del sommario e all'assemblaggio del report finale.

- `generateReportSummary(state: WorkflowState): Promise<ReportSummary>`: richiede uno stato del grafo contenente i risultati dell'analisi; restituisce un `ReportSummary` riassunto e punteggio. In caso di errore durante la generazione, restituisce un `ReportSummary` di fallback con punteggio minimo, senza propagare eccezioni di tipo `Error`.

- `assembleReport(state: WorkflowState, summary: ReportSummary): Report`: richiede lo stato del grafo e un `ReportSummary` già prodotto; restituisce il `Report` finale con dati aggregati e metadati della scansione; non solleva eccezioni.

4.3.7 Nodo analisi dei test

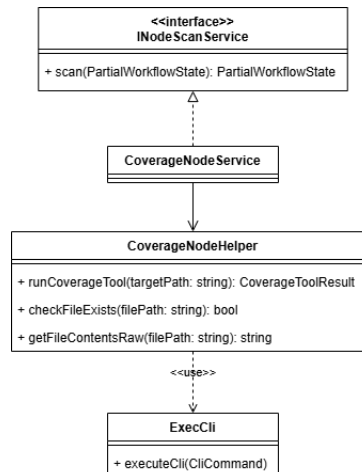


Figura 26: UML CoverageModule

`CoverageModule` contiene due *provider*: `CoverageNodeService`, registrato tramite il token `COVERAGE_NODE_SERVICE_TOKEN`, e `CoverageNodeHelper`. Il modulo esporta il token del `CoverageNodeService` per poter permettere l'iniezione di interfacce implementate da tale service.

4.3.7.1 CoverageNodeService

`CoverageNodeService` implementa l'interfaccia `INodeScanService` ed è responsabile dell'analisi della copertura dei test del repository analizzato.

- `scan({ repoPath }: { repoPath: string }): Promise<CoverageNodeResult>`: richiede che lo stato condiviso del grafo contenga un percorso valido al repository locale; restituisce un `CoverageNodeResult` contenente le percentuali di copertura per istruzioni, rami, funzioni e righe, l'elenco dei test falliti e il numero totale di test eseguiti. In caso di errore in qualsiasi fase dell'analisi, restituisce un report azzerato senza propagare eccezioni.

4.3.7.2 CoverageNodeHelper

`CoverageNodeHelper` fornisce al `CoverageNodeService` le operazioni necessarie all'esecuzione della suite di test e alla lettura dei risultati.

- `runCoverageTool(targetPath: string): Promise<CoverageToolResult>`: richiede un percorso valido al repository; restituisce l'output testuale del test runner e il percorso al file dei risultati generato. Se l'esecuzione non va a buon fine, propaga un'eccezione di tipo `Error` al chiamante.

- `checkFileExists(filePath: string): boolean`: dato un percorso, restituisce un valore booleano che indica se il file esiste; non solleva eccezioni.
- `getFileContentsRaw(resultsPath: string): string`: dato un percorso valido, restituisce il contenuto testuale del file; solleva un'eccezione di tipo `Error` se il file non esiste o non è leggibile.

4.3.8 Nodo GitHub

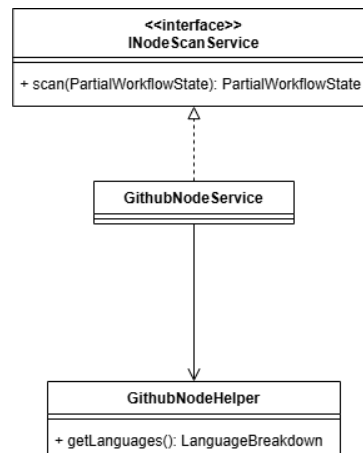


Figura 27: UML GithubModule

GithubModule contiene due *provider*: GithubNodeService, registrato tramite il token `GITHUB_NODE_SERVICE_TOKEN`, e GithubNodeHelper. Il modulo esporta il token del GithubNodeService per poter permettere l'iniezione di interfacce implementate da tale service.

4.3.8.1 GithubNodeService

GithubNodeService implementa l'interfaccia INodeScanService ed è responsabile del rilevamento dei linguaggi di programmazione presenti nel repository analizzato.

- `scan({ repoPath }: { repoPath: string }): Promise<GithubNodeResult>`: richiede che lo stato condiviso del grafo contenga un percorso valido al repository locale; restituisce un GithubNodeResult con l'elenco dei linguaggi rilevati, ciascuno associato alla propria percentuale di utilizzo. In caso di errore durante il rilevamento, restituisce un GithubNodeResult con un elenco vuoto senza propagare eccezioni.

4.3.8.2 GithubNodeHelper

GithubNodeHelper fornisce al GithubNodeService l'operazione di rilevamento dei linguaggi di programmazione.

- `getLanguages(repoPath: string): Promise<LanguageBreakdown>`: richiede un percorso valido al repository; restituisce la distribuzione percentuale dei linguaggi rilevati come mappa nome - percentuale. Eventuali eccezioni di tipo `Error` vengono propagate al chiamante.

4.3.9 Nodo analisi della documentazione

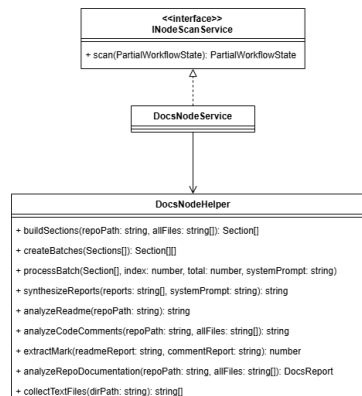


Figura 28: UML DocsModule

DocsModule contiene due *provider*: DocsNodeService, registrato tramite il token DOCS_NODE_SERVICE_TOKEN e DocsNodeHelper. Il modulo esporta il token del DocsNodeService per consentire l'iniezione di interfacce implementate da tale service.

4.3.9.1 DocsNodeService

DocsNodeService implementa l'interfaccia INodeScanService ed è responsabile della valutazione della qualità della documentazione presente nel repository analizzato, considerando sia il file README.md sia i commenti nel codice sorgente.

- `scan({ repoPath }: { repoPath: string }): Promise<Partial<WorkflowState>`: richiede un percorso valido al repository locale; delega all'helper la raccolta dei file di testo rilevanti e l'intera analisi documentale, restituendo un aggiornamento parziale del WorkflowState contenente il DocsReport prodotto. Non solleva eccezioni.

4.3.9.2 DocsNodeHelper

DocsNodeHelper fornisce al DocsNodeService le operazioni necessarie alla raccolta dei file, all'analisi tramite modello LLM e al calcolo del voto finale. Il modello utilizzato è configurabile tramite variabili d'ambiente e opera con temperatura zero per garantire output deterministici.

- `collectTextFiles(dirPath: string): string[]`: richiede un percorso valido alla directory radice; restituisce la lista dei percorsi assoluti di tutti i file di testo analizzabili, escludendo le directory irrilevanti (quali node_modules e dist) e i file di dimensione superiore a 100 KB. Non solleva eccezioni.
- `analyzeRepoDocumentation(repoPath: string, files: string[]): Promise<DocsReport>`: richiede un percorso valido al repository e la lista dei file raccolti; avvia in parallelo l'analisi del README e quella dei commenti nel codice, poi ne combina i risultati invocando il calcolo del voto finale. Restituisce il DocsReport completo. Non solleva eccezioni.
- `analyzeReadme(repoPath: string): Promise<string>`: richiede un percorso valido al repository; legge il file README.md e lo sottopone al modello LLM, che valuta panoramica e struttura del progetto (fino a 5 punti), completezza delle istruzioni di build ed esempi di

utilizzo (fino a 3 punti) e leggibilità del testo tramite l'indice di Gulpease per l'italiano o la formula di Flesch per l'inglese (fino a 2 punti). In assenza del README, restituisce una stringa di avviso predefinita senza interpellare il modello. Non solleva eccezioni.

- `analyzeCodeComments(repoPath: string, files: string[]): Promise<string>`: richiede un percorso valido al repository e la lista dei file da analizzare; suddivide il contenuto in batch da 512 KB e li invia al modello in parallelo per valutare presenza di commenti sulle funzioni pubbliche (fino a 2 punti), coerenza tra commenti e codice (fino a 4 punti) e completezza della documentazione (fino a 4 punti). Se i batch sono più di uno, i report parziali vengono sintetizzati in un'ulteriore chiamata al modello. Restituisce il report testuale in Markdown. Non solleva eccezioni.
- `extractMark(readmeReport: string, commentReport: string): Promise<number>`: richiede i due report testuali prodotti dalle analisi precedenti; chiede al modello di estrarne i punteggi numerici e di calcolare il voto finale come media delle due somme parziali, ciascuna su base 10. Restituisce un numero decimale con al più una cifra dopo la virgola. In caso di errore nella chiamata al modello, restituisce 0 senza propagare eccezioni.

4.3.10 Nodo analisi della sicurezza del codice

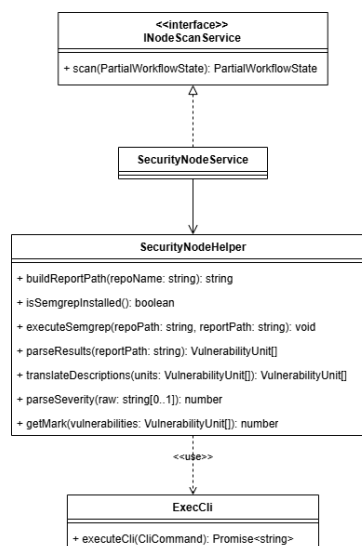


Figura 29: UML SecurityModule

SecurityModule contiene due *provider*: SecurityNodeService, registrato tramite il token SECURITY_NODE_SERVICE_TOKEN, e SecurityNodeHelper. Il modulo esporta il token del SecurityNodeService per consentire l'iniezione di interfacce implementate da tale service. L'output di questo nodo alimenta direttamente il nodo RemediationNode.

4.3.10.1 SecurityNodeService

SecurityNodeService implementa l'interfaccia INodeScanService ed è responsabile dell'identificazione delle vulnerabilità presenti nel codice sorgente tramite analisi statica con *Semgrep*.

- `scan({ repoPath }: { repoPath: string }): Promise<Partial<WorkflowState>`: richiede un percorso valido al repository locale; verifica la disponibilità di *Semgrep*, esegue la scansione, traduce le descrizioni delle vulnerabilità in italiano tramite il modello LLM e calcola il voto di sicurezza. Restituisce un aggiornamento parziale del `WorkflowState` contenente il `VulnerabilitiesReport` e il percorso del file JSON grezzo prodotto da *Semgrep*, necessario al nodo di remediation. In caso di errore in qualsiasi fase, restituisce un oggetto vuoto senza propagare eccezioni.

4.3.10.2 SecurityNodeHelper

`SecurityNodeHelper` fornisce al `SecurityNodeService` le operazioni necessarie all'esecuzione di *Semgrep*, all'interpretazione dei risultati e al calcolo del voto finale.

- `buildReportPath(repoName: string): string`: richiede il nome del repository; restituisce il percorso del file JSON in cui *Semgrep* salverà i risultati, includendo un timestamp per garantire l'unicità del nome. Non solleva eccezioni.
- `isSemgrepInstalled(): Promise<boolean>`: verifica che *Semgrep* sia presente e raggiungibile nel PATH di sistema; restituisce `true` se lo strumento è disponibile, `false` altrimenti. Non solleva eccezioni.
- `executeSemgrep(repoPath: string, reportPath: string): Promise<void>`: richiede un percorso valido al repository e il percorso di destinazione del report; esegue *Semgrep* con rilevamento automatico del ruleset, escludendo le directory di build e le dipendenze esterne, e salva i risultati in formato JSON. Propaga un'eccezione di tipo `Error` al chiamante in caso di fallimento.
- `parseResults(reportPath: string): Promise<VulnerabilityUnit[]>`: richiede il percorso del file JSON prodotto da *Semgrep*; lo legge e trasforma ogni risultato in un oggetto `VulnerabilityUnit` con identificatore, percorso del file, descrizione, severità, categoria e riferimenti agli standard CWE e OWASP. Il campo `remediation` viene inizializzato vuoto. Propaga un'eccezione di tipo `Error` in caso di file non leggibile o JSON non valido.
- `translateDescriptions(units: VulnerabilityUnit[]): Promise<VulnerabilityUnit[]>`: richiede la lista delle vulnerabilità rilevate; invia le descrizioni originali in inglese al modello LLM, che le restituisce tradotte in italiano. Restituisce la lista aggiornata con le traduzioni rimappate. In caso di errore nella chiamata al modello, restituisce la lista originale invariata senza propagare eccezioni.
- `getMark(vulnerabilities: VulnerabilityUnit[]): number`: richiede la lista delle vulnerabilità; calcola il voto come media delle severity, dove `INFO` vale 10, `WARNING` vale 5 e `ERROR` vale 0. Restituisce 10 in assenza di vulnerabilità. Non solleva eccezioni.

4.3.11 Nodo creazione delle remediation

`RemediationModule` contiene due *provider*: `RemediationNodeService`, registrato tramite il token `REMEDIATION_NODE_SERVICE_TOKEN`, e `RemediationNodeHelper`. Il modulo esporta il token del `RemediationNodeService` per consentire l'iniezione di interfacce implementate da tale service. Questo nodo è il successore diretto di `SecurityNode` nel grafo e si occupa di arricchire le vulnerabilità già rilevate con le relative azioni correttive.

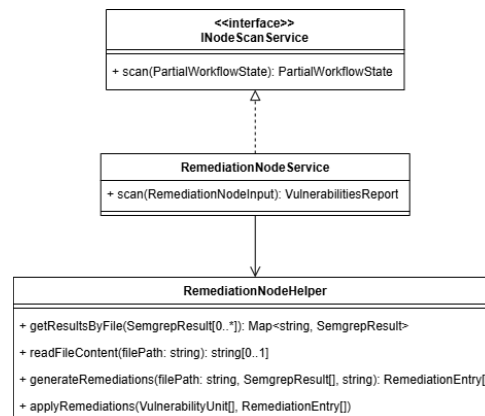


Figura 30: UML RemediationModule

4.3.11.1 RemediationNodeService

`RemediationNodeService` implementa l'interfaccia `INodeScanService` ed è responsabile della generazione delle remediation per le vulnerabilità identificate da `SecurityNode`. A differenza degli altri nodi, il metodo `scan` riceve in input gli output del nodo precedente anziché il percorso della repository.

- `scan({ vulnerabilitiesReport, vulnerabilitiesReportPath }): Promise<Partial<WorkflowState>>`: richiede il report delle vulnerabilità già tradotte e il percorso del file grezzo prodotto da *Semgrep*; verifica la presenza e l'accessibilità del file, poi delega all'helper il raggruppamento dei risultati per file sorgente e l'elaborazione in parallelo di ciascun gruppo. Per ogni file, applica le remediation generate sulle corrispondenti `VulnerabilityUnit`. Restituisce il `VulnerabilitiesReport` aggiornato con le remediation compilate. In assenza di vulnerabilità o in caso di errore nell'accesso al file, restituisce il report originale invariato o un oggetto vuoto senza propagare eccezioni. Gli errori relativi a singoli file vengono gestiti individualmente, senza interrompere l'elaborazione degli altri.

4.3.11.2 RemediationNodeHelper

`RemediationNodeHelper` fornisce al `RemediationNodeService` le operazioni necessarie al raggruppamento delle vulnerabilità, alla lettura dei file sorgente, alla generazione delle remediation tramite modello LLM e alla loro applicazione sul report. Il limite di token per risposta è impostato a 15 000, valore superiore rispetto agli altri nodi, per accomodare la generazione di remediation articolate su file con molte vulnerabilità.

- `groupResultsByFile(results: SemgrepResult[]): Map<string, SemgrepResult[]>`: richiede la lista dei risultati grezzi prodotti da *Semgrep*; li raggruppa per percorso del file affetto e restituisce una mappa che associa ciascun percorso alla lista delle vulnerabilità rilevate in quel file, garantendo che ogni file venga elaborato in un'unica chiamata al modello. Non solleva eccezioni.
- `readFileContent(filePath: string): Promise<string | null>`: richiede il percorso di un file sorgente; restituisce il contenuto testuale del file, oppure `null` se il file non è leggibile. Non propaga eccezioni.

- `generateRemediations(filePath, results, fileContent, model): Promise<RemediationEntry[]>`: richiede il percorso del file, le vulnerabilità rilevate al suo interno, il contenuto sorgente e un'istanza del modello LLM; invia tutto al modello, che restituisce un array JSON con una remediation per ciascuna vulnerabilità. Ogni remediation è una stringa Markdown strutturata in tre sezioni: un esempio della vulnerabilità, le azioni correttive o di mitigazione e le conseguenze di una mancata risoluzione. Propaga un'eccezione di tipo `Error` in caso di errore nella chiamata al modello o di risposta non parsificabile.
- `applyRemediations(vulnerabilities, remediations): void`: richiede la lista delle `VulnerabilityUnit` e le remediation generate; aggiorna in-place il campo `remediation` di ciascuna unità abbinandola tramite identificatore. Non solleva eccezioni.

4.3.12 Nodo analisi della dipendenze

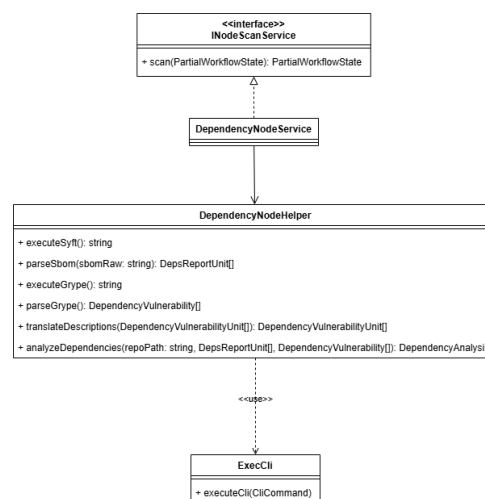


Figura 31: UML DependencyModule

DepsModule contiene due *provider*: `DependencyNodeService`, registrato tramite il token `DEPENDENCY_NODE_SERVICE_TOKEN`, e `DependencyNodeHelper`. Il modulo esporta il token del `DependencyNodeService` per consentire l'iniezione di interfacce implementate da tale service.

4.3.12.1 DependencyNodeService

`DependencyNodeService` implementa l'interfaccia `INodeScanService` ed è responsabile della produzione di un inventario completo delle dipendenze della repository e dell'identificazione delle vulnerabilità note al loro interno, combinando i risultati di *Syft* e *Grype* con un'elaborazione tramite modello LLM.

- `scan({ repoPath }: { repoPath: string }): Promise<Partial<WorkflowState>`: richiede un percorso valido al repository locale; genera il SBOM tramite *Syft*, ricerca le vulnerabilità note con *Grype*, traduce le descrizioni in italiano e chiede al modello LLM di classificare le dipendenze e produrre un'analisi discorsiva delle criticità. Restituisce un aggiornamento parziale del `WorkflowState` contenente il `DepsReport` completo. Un eventuale fallimento di *Grype* viene gestito in modo isolato, producendo un elenco di

vulnerabilità vuoto senza compromettere il resto dell'analisi. In caso di errore generale, restituisce un report con tutti i campi vuoti senza propagare eccezioni.

4.3.12.2 DependencyNodeHelper

DependencyNodeHelper fornisce al DependencyNodeService le operazioni necessarie all'invocazione degli strumenti di analisi, all'interpretazione dei loro output e alla classificazione delle dipendenze tramite modello LLM.

- `executeSyft(repoPath: string): Promise<string>`: richiede un percorso valido al repository; invoca *Syft* sulla directory e restituisce il SBOM (*Software Bill of Materials*) in formato JSON, ovvero l'inventario completo di tutte le dipendenze rilevate. Propaga un'eccezione di tipo `Error` al chiamante in caso di fallimento.
- `parseSbom(sbomRaw: string): DepsReportUnit[]`: richiede il contenuto JSON del SBOM prodotto da *Syft*; restituisce la lista delle dipendenze, ciascuna identificata da nome e versione. Propaga un'eccezione di tipo `Error` in caso di JSON non valido.
- `executeGrype(sbomRaw: string): Promise<string>`: richiede il contenuto JSON del SBOM; lo salva temporaneamente su disco, invoca *Grype* per la ricerca di vulnerabilità note e ne restituisce il report in formato JSON. Il file temporaneo viene rimosso al termine dell'operazione, indipendentemente dall'esito. Propaga un'eccezione di tipo `Error` al chiamante in caso di fallimento.
- `parseGrype(grypeRaw: string): DependencyVulnerability[]`: richiede il contenuto JSON del report prodotto da *Grype*; restituisce la lista delle vulnerabilità rilevate, ciascuna con identificatore CVE, severità, descrizione, pacchetto interessato, versione affetta e versione corretta disponibile, se presente. Propaga un'eccezione di tipo `Error` in caso di JSON non valido.
- `translateDescriptions(vulnerabilities: DependencyVulnerability[]): Promise<DependencyVulnerability[]>`: richiede la lista delle vulnerabilità rilevate; invia le descrizioni originali in inglese al modello LLM, che le restituisce tradotte in italiano in forma leggermente più discorsiva. Restituisce la lista aggiornata con le traduzioni rimappate. In caso di errore nella chiamata al modello, restituisce la lista originale invariata senza propagare eccezioni.
- `analyzeDependencies(repoPath: string, list: DepsReportUnit[], vulnerabilities: DependencyVulnerability[]): Promise<DependencyAnalysis>`: richiede il percorso del repository, la lista delle dipendenze e le vulnerabilità tradotte; legge il `package.json` e invia tutto al modello LLM, che restituisce la separazione delle dipendenze in framework e librerie con le rispettive versioni installate, e un'analisi discorsiva in Markdown delle vulnerabilità emerse entro un limite di 1 000 caratteri. In caso di errore, restituisce una `DependencyAnalysis` con tutti i campi vuoti senza propagare eccezioni.

4.3.13 Tipi

4.3.13.1 DepsReportUnit

La classe `DepsReportUnit` rappresenta il tipo di trasferimento dati relativo a una singola dipendenza rilevata durante l'analisi del progetto. Essa espone i seguenti attributi:

- `name: string`: stringa che indica il nome della dipendenza;
- `version: string`: stringa che indica la versione della dipendenza rilevata all'interno del progetto analizzato.

4.3.13.2 DepsVulnerabilityUnit

La classe `DepsVulnerabilityUnit` rappresenta il tipo di trasferimento dati relativo a una singola vulnerabilità associata a una dipendenza del progetto. Essa espone i seguenti attributi:

- `id: string`: stringa che identifica univocamente la vulnerabilità all'interno del sistema di scansione;
- `severity: string`: stringa che indica il livello di severità della vulnerabilità (ad esempio *low*, *medium*, *high* o *critical*);
- `description: string`: stringa contenente una descrizione testuale della vulnerabilità rilevata;
- `packageName: string`: stringa che indica il nome del pacchetto affetto dalla vulnerabilità;
- `packageVersion: string`: stringa che indica la versione del pacchetto affetto dalla vulnerabilità;
- `fixVersion: string[0..1]`: stringa opzionale che indica la versione del pacchetto nella quale la vulnerabilità risulta corretta. Il campo può essere assente qualora non sia ancora disponibile una versione risolutiva.

4.3.13.3 SemgrepResult

La classe `SemgrepResult` rappresenta il tipo di trasferimento dati relativo a un singolo risultato prodotto dallo strumento di analisi statica Semgrep durante la scansione del codice sorgente. Ogni istanza corrisponde a un'occorrenza di una regola violata all'interno del progetto. Essa espone i seguenti attributi:

- `path: string`: stringa che indica il percorso del file in cui è stata rilevata la violazione della regola di analisi statica;
- `check_id: string`: stringa che identifica univocamente la regola Semgrep che ha prodotto il risultato, tipicamente nella forma `<ruleset>.<rule-name>`;
- `values: Map<string, unknown>`: mappa che associa a ciascuna chiave stringa un valore di tipo non determinato a priori, contenente i metadati aggiuntivi restituiti da Semgrep per il risultato corrente, quali la posizione nel file, il messaggio descrittivo e la severità.

4.3.13.4 RemediationEntry

La classe `RemediationEntry` rappresenta il tipo di trasferimento dati relativo a una singola indicazione di rimedio associata a una vulnerabilità identificata nel progetto. Essa viene utilizzata per trasportare le istruzioni di correzione generate per ciascuna vulnerabilità nel contesto del workflow di analisi. Essa espone i seguenti attributi:

- `id: string`: stringa che identifica univocamente la vulnerabilità a cui l'indicazione di rimedio si riferisce, in corrispondenza con l'identificativo presente nel report delle vulnerabilità;
- `remediation: string`: stringa contenente le istruzioni testuali di correzione o mitigazione della vulnerabilità identificata.

4.3.13.5 RemediationNodeInput

La classe `RemediationNodeInput` rappresenta il tipo di trasferimento dati relativo all'input fornito al nodo del workflow deputato alla generazione delle indicazioni di rimedio per le vulnerabilità rilevate. Essa espone i seguenti attributi:

- `vulnerabilitiesReport: VulnerabilitiesReport[0..1]`: oggetto opzionale di tipo `VulnerabilitiesReport` che contiene il report strutturato delle vulnerabilità da elaborare. Il campo può essere assente qualora il report non sia ancora disponibile al momento dell'invocazione del nodo;
- `vulnerabilitiesReportPath: string[0..1]`: stringa opzionale che indica il percorso del file contenente il report grezzo delle vulnerabilità prodotto dallo strumento di scansione esterno. Il campo può essere assente qualora il percorso non sia ancora stato determinato o qualora il report sia già disponibile in forma strutturata tramite l'attributo precedente.

4.3.13.6 VulnerabilityUnit

La classe `VulnerabilityUnit` rappresenta il tipo di trasferimento dati relativo al dettaglio di una singola vulnerabilità restituita dal sistema di scansione. Essa espone i seguenti attributi:

- `id: string`: stringa che identifica univocamente la vulnerabilità
- `path: string`: stringa che indica il percorso del file o del componente in cui la vulnerabilità è stata rilevata
- `description: string`: stringa contenente una descrizione estesa della natura e delle implicazioni della vulnerabilità
- `remediation: string`: stringa contenente le indicazioni per la risoluzione o la mitigazione della vulnerabilità
- `severity: number`: valore numerico che rappresenta il livello di severità della vulnerabilità
- `impact: string`: stringa che descrive l'impatto potenziale della vulnerabilità sul sistema qualora venisse sfruttata

- `category: string`: stringa che indica la categoria di appartenenza della vulnerabilità (ad esempio *injection*, *broken access control* e simili)
- `cwe: string`: stringa che riporta l'identificativo CWE (*Common Weakness Enumeration*) associato alla vulnerabilità
- `owasp: string[0..*]`: lista di stringhe che riportano i riferimenti alla classificazione OWASP associati alla vulnerabilità. Il campo può essere vuoto qualora la vulnerabilità non rientri in nessuna categoria OWASP

4.3.13.7 Vulnerability

La classe `Vulnerability` rappresenta il tipo di trasferimento dati relativo al risultato complessivo dell'analisi delle vulnerabilità associate a una dipendenza. Essa espone i seguenti attributi:

- `vulnerabilities: VulnerabilityUnit[0..*]`: lista di oggetti di tipo `VulnerabilityUnit` che raccoglie il dettaglio di ciascuna vulnerabilità individuata;
- `mark: number`: valore numerico che rappresenta il punteggio di rischio complessivo calcolato a partire dall'insieme delle vulnerabilità rilevate.

4.3.13.8 DepsReport

La classe `DepsReport` rappresenta il tipo di trasferimento dati relativo al report completo delle dipendenze analizzate. Essa espone i seguenti attributi:

- `list: DepsReportUnit[0..*]`: lista di oggetti di tipo `DepsReportUnit` che raccoglie l'insieme completo delle dipendenze rilevate
- `libraries: DepsReportUnit[0..*]`: lista di oggetti di tipo `DepsReportUnit` che raccoglie le sole librerie che sono state trovate durante lo scan del repository
- `frameworks: DepsReportUnit[0..*]`: lista di oggetti di tipo `DepsReportUnit` che raccoglie i framework che sono stati identificati durante lo scan
- `vulnerabilities: VulnerabilityUnit[0..*]`: lista di oggetti di tipo `VulnerabilityUnit` che raccoglie le vulnerabilità individuate nell'insieme delle dipendenze analizzate
- `vulnerabilityAnalysis: string`: stringa contenente l'analisi testuale complessiva delle vulnerabilità rilevate, generata a partire dai dati raccolti durante la scansione

4.3.13.9 DocsReport

La classe `DocsReport` rappresenta il tipo di trasferimento dati relativo al report della qualità della documentazione del progetto analizzato. Essa espone i seguenti attributi:

- `readmeReport: string`: stringa contenente l'analisi testuale del file README del progetto, comprensiva di una valutazione della sua completezza e leggibilità;
- `commentReport: string`: stringa contenente l'analisi testuale della qualità e della copertura dei commenti presenti nel codice sorgente del progetto;

- `mark: number`: valore numerico che rappresenta il punteggio complessivo assegnato alla documentazione del progetto a partire dai dati raccolti durante l'analisi.

4.3.13.10 CoverageToolResult

La classe `CoverageToolResult` contiene i risultati del lancio del coverage, limitatamente all'output verso `stdout` del tool e il percorso al file generato. Essa espone i seguenti attributi:

- `stdout: string`: contenuto dello `stdout` a seguito dell'esecuzione del tool
- `resultsPath: string`: percorso al file generato

4.3.13.11 CoverageNodeResult

La classe `CoverageNodeResult` rappresenta il tipo di trasferimento dati relativo al risultato dell'analisi di copertura del codice elaborato e strutturato dal nodo corrispondente del workflow. A differenza di `CoverageToolResult`, che contiene l'output grezzo dello strumento esterno, questa classe espone i dati già interpretati e convertiti nel formato di business. Essa espone il seguente attributo:

- `testReport: TestReport`: oggetto di tipo `TestReport` che contiene il report dei test prodotto a partire dall'elaborazione dell'output dello strumento di analisi della copertura.

4.3.13.12 CoverageReport

La classe `CoverageReport` rappresenta il tipo di trasferimento dati relativo alle metriche di copertura del codice sorgente rilevate durante l'esecuzione dei test. Essa espone i seguenti attributi:

- `statements: number`: valore numerico che indica la percentuale di istruzioni del codice sorgente coperte dai test;
- `branches: number`: valore numerico che indica la percentuale di rami decisionali del codice sorgente coperti dai test;
- `functions: number`: valore numerico che indica la percentuale di funzioni del codice sorgente coperte dai test;
- `lines: number`: valore numerico che indica la percentuale di righe del codice sorgente coperte dai test.

4.3.13.13 TestReport

La classe `TestReport` rappresenta il tipo di trasferimento dati relativo al report dei test del progetto analizzato. Essa espone i seguenti attributi:

- `coverageReport: CoverageReport`: oggetto di tipo `CoverageReport` che raccoglie le metriche di copertura del codice rilevate durante l'esecuzione della suite di test;
- `failedTests: FailedTest[0..*]`: lista di oggetti di tipo `FailedTest` che raccoglie il dettaglio di ciascun test terminato con esito negativo. Il campo può essere vuoto qualora tutti i test abbiano avuto esito positivo;

- `testsRun: string`: stringa che riporta il numero totale di test eseguiti durante l'analisi del progetto.

4.3.13.14 FailedTest

La classe `FailedTest` rappresenta il tipo di trasferimento dati relativo a un singolo test che ha prodotto un esito negativo durante l'esecuzione della suite di test del progetto. Essa espone i seguenti attributi:

- `name: string`: stringa che indica il nome del test che ha prodotto un esito negativo;
- `path: string`: stringa che indica il percorso del file in cui il test fallito è definito all'interno del progetto;
- `messageSummary: string`: stringa contenente un riepilogo del messaggio di errore prodotto dal test durante la sua esecuzione.

4.3.13.15 Language

La classe `Language` rappresenta il tipo di trasferimento dati relativo a un singolo linguaggio di programmazione rilevato nel progetto analizzato. Essa espone i seguenti attributi:

- `name: string`: stringa che indica il nome del linguaggio di programmazione;
- `value: number`: valore numerico che indica la percentuale di codice sorgente del progetto scritta nel linguaggio corrispondente.

4.3.13.16 TechReport

La classe `TechReport` rappresenta il tipo di trasferimento dati relativo al report dello stack tecnologico rilevato nel progetto analizzato. Essa espone i seguenti attributi:

- `libraries: DepsReportUnit[0..*]`: lista di oggetti di tipo `DepsReportUnit` che raccoglie le librerie esterne individuate nel progetto analizzato;
- `frameworks: DepsReportUnit[0..*]`: lista di oggetti di tipo `DepsReportUnit` che raccoglie i framework individuati nel progetto analizzato;
- `languages: Language[0..*]`: lista di oggetti di tipo `Language` che raccoglie i linguaggi di programmazione rilevati nel progetto analizzato, corredati della rispettiva percentuale di utilizzo.

4.3.13.17 ReportSummary

La classe `ReportSummary` rappresenta il tipo di trasferimento dati relativo al riepilogo sintetico del report generato a partire dall'analisi del progetto. Essa espone i seguenti attributi:

- `summary: string`: stringa contenente una descrizione testuale complessiva dei risultati dell'analisi, ottenuta aggregando le informazioni provenienti dalle diverse sezioni del report;
- `mark: number`: valore numerico che rappresenta il punteggio globale assegnato al progetto analizzato, calcolato a partire dai punteggi delle singole sezioni del report.

4.3.13.18 ReportMetadata

La classe `ReportMetadata` rappresenta il tipo di trasferimento dati relativo ai metadati associati all'esecuzione di una scansione. Essa espone i seguenti attributi:

- `startScanTime`: `Date`: valore di tipo `Date` che indica il momento di avvio della scansione del progetto;
- `endScanTime`: `Date[0..1]`: valore opzionale di tipo `Date` che indica il momento di completamento della scansione. Il campo può essere assente qualora la scansione non sia ancora terminata al momento della restituzione del dato;
- `target`: `Target`: oggetto di tipo `Target` che identifica il repository GitHub oggetto della scansione, comprensivo delle credenziali di accesso.

4.3.13.19 DataReport

La classe `DataReport` rappresenta il tipo di trasferimento dati che aggrega i report dettagliati prodotti da ciascuna delle sezioni di analisi del progetto. Essa espone i seguenti attributi:

- `depsReport`: `DepsReport`: oggetto di tipo `DepsReport` che contiene il report relativo alle dipendenze del progetto analizzato;
- `vulnerabilitiesReport`: `VulnerabilitiesReport`: oggetto di tipo `VulnerabilitiesReport` che contiene il report relativo alle vulnerabilità individuate nelle dipendenze del progetto;
- `docsReport`: `DocsReport`: oggetto di tipo `DocsReport` che contiene il report relativo alla qualità della documentazione del progetto;
- `testReport`: `TestReport`: oggetto di tipo `TestReport` che contiene il report relativo ai test del progetto;
- `techReport`: `TechReport`: oggetto di tipo `TechReport` che contiene il report relativo allo stack tecnologico rilevato nel progetto.

4.3.13.20 Report

La classe `Report` rappresenta il tipo di trasferimento dati relativo al report completo prodotto al termine dell'analisi di un progetto. Essa costituisce il tipo radice della gerarchia dei DTO del sistema e aggrega tutte le informazioni raccolte durante la scansione. Essa espone i seguenti attributi:

- `summary`: `ReportSummary`: oggetto di tipo `ReportSummary` che contiene il riepilogo sintetico e il punteggio globale del report;
- `data`: `DataReport`: oggetto di tipo `DataReport` che raggruppa i report dettagliati prodotti da ciascuna delle sezioni di analisi del progetto;
- `metadata`: `ReportMetadata`: oggetto di tipo `ReportMetadata` che contiene le informazioni contestuali relative all'esecuzione della scansione, quali i tempi di inizio e fine e il riferimento al repository analizzato.

4.3.13.21 Target

La classe `Target` rappresenta il tipo di trasferimento dati relativo al repository GitHub da sottoporre ad analisi, comprensivo delle credenziali necessarie per l'accesso. Essa espone i seguenti attributi:

- `owner: string`: stringa che indica il nome dell'utente o dell'organizzazione proprietaria del repository GitHub da analizzare;
- `repository: string`: stringa che indica il nome del repository GitHub da sottoporre ad analisi;
- `branch: string`: stringa che indica il nome del branch del repository da analizzare;
- `accessToken: string[0..1]`: stringa opzionale che contiene il token di accesso necessario per l'autenticazione verso repository privati. Il campo può essere assente qualora il repository sia pubblico e non richieda autenticazione.

4.3.13.22 ReportedTarget

La classe `ReportedTarget` rappresenta il tipo di trasferimento dati relativo al riferimento al repository GitHub associato a un report già prodotto. A differenza di `Target`, non include il token di accesso, in quanto utilizzata esclusivamente in contesti di consultazione di report preesistenti e non di avvio di nuove scansioni. Essa espone i seguenti attributi:

- `owner: string`: stringa che indica il nome dell'utente o dell'organizzazione proprietaria del repository GitHub a cui il report si riferisce;
- `repository: string`: stringa che indica il nome del repository GitHub a cui il report si riferisce;
- `branch: string`: stringa che indica il nome del branch del repository a cui il report si riferisce.

4.3.13.23 WorkflowState

La classe `WorkflowState` rappresenta il tipo di trasferimento dati relativo allo stato completo di un'istanza di workflow di analisi in corso. Essa raccoglie tutte le informazioni accumulate durante le varie fasi della scansione, dalle informazioni iniziali sul repository fino ai report parziali e finale prodotti. Essa espone i seguenti attributi:

- `target: Target`: oggetto di tipo `Target` che identifica il repository GitHub oggetto della scansione in corso;
- `repoPath: string`: stringa che indica il percorso locale in cui il repository è stato clonato per essere sottoposto ad analisi;
- `startScanTime: Date`: valore di tipo `Date` che indica il momento di avvio della scansione;
- `vulnerabilitiesReportPath: Date`: valore di tipo `Date` che indica il percorso del file contenente il report grezzo delle vulnerabilità prodotto dallo strumento di scansione esterno;

- `languages: Language[]`: lista di oggetti di tipo `Language` che raccoglie i linguaggi di programmazione rilevati nel repository durante la fase iniziale di analisi;
- `depsReport: DepsReport`: oggetto di tipo `DepsReport` che contiene il report delle dipendenze prodotto al termine della relativa fase di analisi;
- `vulnerabilitiesReport: VulnerabilitiesReport[0..1]`: oggetto opzionale di tipo `VulnerabilitiesReport` che contiene il report delle vulnerabilità. Il campo può essere assente qualora la relativa fase di analisi non sia ancora completata;
- `docsReport: DocsReport[0..1]`: oggetto opzionale di tipo `DocsReport` che contiene il report della documentazione. Il campo può essere assente qualora la relativa fase di analisi non sia ancora completata;
- `testReport: TestReport[0..1]`: oggetto opzionale di tipo `TestReport` che contiene il report dei test. Il campo può essere assente qualora la relativa fase di analisi non sia ancora completata;
- `finalReport: Report[0..1]`: oggetto opzionale di tipo `Report` che contiene il report finale aggregato. Il campo può essere assente qualora la fase di aggregazione e generazione del report conclusivo non sia ancora terminata.

4.3.13.24 PartialWorkflowState

La classe `PartialWorkflowState` rappresenta il tipo di trasferimento dati relativo a un aggiornamento parziale dello stato di un workflow di analisi. A differenza di `WorkflowState`, tutti i suoi attributi sono opzionali, in quanto essa viene utilizzata per trasmettere esclusivamente i campi dello stato che hanno subito una modifica in seguito al completamento di una singola fase della scansione. Essa espone i seguenti attributi:

- `target: Target[0..1]`: oggetto opzionale di tipo `Target` che, se presente, aggiorna il riferimento al repository oggetto della scansione;
- `repoPath: string[0..1]`: stringa opzionale che, se presente, aggiorna il percorso locale del repository clonato;
- `startScanTime: Date[0..1]`: valore opzionale di tipo `Date` che, se presente, aggiorna il momento di avvio della scansione;
- `vulnerabilitiesReportPath: Date[0..1]`: valore opzionale che, se presente, aggiorna il percorso del file contenente il report grezzo delle vulnerabilità;
- `languages: Language[] [0..1]`: lista opzionale di oggetti di tipo `Language` che, se presente, aggiorna l'insieme dei linguaggi di programmazione rilevati nel repository;
- `depsReport: DepsReport[0..1]`: oggetto opzionale di tipo `DepsReport` che, se presente, aggiorna il report delle dipendenze del progetto;
- `vulnerabilitiesReport: VulnerabilitiesReport[0..1]`: oggetto opzionale di tipo `VulnerabilitiesReport` che, se presente, aggiorna il report delle vulnerabilità;
- `docsReport: DocsReport[0..1]`: oggetto opzionale di tipo `DocsReport` che, se presente, aggiorna il report della documentazione;

- `testReport: TestReport[0..1]`: oggetto opzionale di tipo `TestReport` che, se presente, aggiorna il report dei test;
- `finalReport: Report[0..1]`: oggetto opzionale di tipo `Report` che, se presente, aggiorna il report finale aggregato del progetto analizzato.

4.3.13.25 RepositoryConnectionInfo

La classe `RepositoryConnectionInfo` rappresenta il tipo di trasferimento dati relativo alle informazioni di connessione a un repository GitHub, utilizzato tipicamente in contesti di verifica parziale della configurazione. Tutti i suoi attributi sono opzionali, in quanto la classe può essere impiegata per trasmettere anche solo un sottoinsieme delle credenziali disponibili. Essa espone i seguenti attributi:

- `owner: string[0..1]`: stringa opzionale che indica il nome dell'utente o dell'organizzazione proprietaria del repository GitHub;
- `repository: string[0..1]`: stringa opzionale che indica il nome del repository GitHub;
- `branch: string[0..1]`: stringa opzionale che indica il nome del branch del repository;
- `accessToken: string[0..1]`: stringa opzionale che contiene il token di accesso per l'autenticazione verso repository privati.

4.3.13.26 SendReportInfo

La classe `SendReportInfo` rappresenta il tipo di trasferimento dati relativo alle informazioni necessarie per l'invio di un report generato a un destinatario esterno. Essa espone i seguenti attributi:

- `target: string`: stringa che identifica il destinatario a cui il report deve essere inviato, tipicamente un indirizzo email o un identificativo di canale;
- `report: Report`: oggetto di tipo `Report` che contiene il report completo da trasmettere al destinatario;
- `token: string`: stringa contenente il token di autenticazione necessario per autorizzare l'operazione di invio verso il servizio di notifica esterno.

4.3.13.27 SendErrorNotificationInfo

La classe `SendErrorNotificationInfo` rappresenta il tipo di trasferimento dati relativo alle informazioni necessarie per l'invio di una notifica di errore a un destinatario esterno, in caso di fallimento del workflow di analisi. Essa espone i seguenti attributi:

- `target: string`: stringa che identifica il destinatario a cui la notifica di errore deve essere inviata;
- `token: string`: stringa contenente il token di autenticazione necessario per autorizzare l'operazione di invio della notifica verso il servizio esterno.

5 Stato dei requisiti funzionali

5.1 Requisiti Funzionali

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-1-F-Ob	L'utente non autenticato deve poter registrarsi al sistema	Soddisfatto
R-2-F-Ob	Mentre un utente non autenticato si sta registrando al sistema, deve specificare lo username con cui vuole registrarsi	Soddisfatto
R-3-F-Ob	Mentre un utente non autenticato si sta registrando al sistema, deve specificare l'indirizzo email con cui vuole registrarsi	Soddisfatto
R-4-F-Ob	Mentre un utente non autenticato si sta registrando al sistema, deve specificare la password che dovrà inserire in fase di autenticazione	Soddisfatto
R-5-F-Ob	Al momento della registrazione, il sistema deve rifiutare la registrazione se l'username è già stato utilizzato da un altro utente	Soddisfatto
R-6-F-Ob	Al momento della registrazione, il sistema deve rifiutare la registrazione se l'indirizzo email è già stato utilizzato da un altro utente	Soddisfatto
R-7-F-Ob	Il sistema deve inviare un codice OTP via email per la conferma della registrazione	Soddisfatto
R-8-F-Ob	In fase di verifica del codice OTP, il sistema deve mostrare un messaggio di errore se il codice OTP è errato	Soddisfatto
R-9-F-Ob	In fase di verifica del codice OTP, il sistema deve mostrare un messaggio di errore se il codice OTP è scaduto	Soddisfatto
R-10-F-Ob	Un utente non autenticato deve poter effettuare il login tramite username e password	Soddisfatto
R-11-F-Ob	Il sistema deve mostrare un messaggio di errore in caso di credenziali errate	Soddisfatto
R-12-F-Ob	L'utente deve poter recuperare la password inviando un codice OTP all'indirizzo di posta elettronica con cui l'utente si è registrato	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-13-F-Ob	L'utente deve visualizzare un messaggio di errore se il codice OTP è errato o scaduto durante il procedimento di recupero password	Soddisfatto
R-14-F-Ob	Un utente autenticato che ha il ruolo di Project Manager all'interno di un workspace deve poter invitare a quel workspace un utente registrato tramite username	Soddisfatto
R-15-F-Ob	Un utente autenticato che ha il ruolo di Project Manager all'interno di un workspace deve poter invitare a quel workspace un utente registrato tramite username, e assegnargli il ruolo di Project Manager	Soddisfatto
R-16-F-Ob	Un utente autenticato che ha il ruolo di Project Manager all'interno di un workspace deve poter invitare a quel workspace un utente registrato tramite username, e assegnargli il ruolo di Tech Lead	Soddisfatto
R-17-F-Ob	Un utente autenticato che ha il ruolo di Project Manager all'interno di un workspace deve poter invitare a quel workspace un utente registrato tramite username, e assegnargli il ruolo di Developer	Soddisfatto
R-18-F-Ob	Al momento di invito dell'utente, il sistema non deve inviare l'invito se tale utente appartiene già al workspace	Soddisfatto
R-19-F-Ob	Al momento di invito dell'utente, il sistema deve mostrare un messaggio di errore se l'utente invitato è già presente nel workspace	Soddisfatto
R-20-F-Ob	Un utente deve poter visualizzare una lista di inviti, per ogni invito ricevuto non accettato né rifiutato	Soddisfatto
R-21-F-Ob	Quando un utente autenticato visualizza un invito deve poter visualizzare il nome del workspace a cui è stato invitato	Soddisfatto
R-22-F-Ob	Quando un utente autenticato visualizza un invito deve poter visualizzare lo username del mittente	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-23-F-Ob	Quando un utente autenticato visualizza un invito deve poter visualizzare il ruolo che assumerebbe una volta accettato l'invito	Soddisfatto
R-24-F-Ob	Quando un utente autenticato visualizza un invito deve poter accettare tale invito	Soddisfatto
R-25-F-Ob	Quando un utente autenticato visualizza un invito deve poter rifiutare tale invito	Soddisfatto
R-26-F-Ob	Un utente autenticato deve poter visualizzare la lista dei workspace a cui appartiene	Soddisfatto
R-27-F-Ob	Per ogni workspace a cui appartiene, un utente autenticato deve poterne visualizzare il nome	Soddisfatto
R-28-F-Ob	Per ogni workspace a cui appartiene, un utente autenticato deve poter visualizzare lo username dell'owner del workspace	Soddisfatto
R-29-F-De	Se un utente autenticato sta visualizzando la lista dei workspace a cui appartiene, deve anche poter effettuare una ricerca per nome tramite barra di ricerca	Soddisfatto
R-30-F-De	Se un utente autenticato sta visualizzando la lista dei workspace a cui appartiene, deve anche poter effettuare una ricerca per nome in tempo reale tramite barra di ricerca	Soddisfatto
R-31-F-De	Se un utente autenticato sta visualizzando la lista dei workspace, ma tale lista è vuota, deve visualizzare un messaggio dedicato	Soddisfatto
R-32-F-Ob	Un utente autenticato deve poter creare un nuovo workspace	Soddisfatto
R-33-F-Ob	Mentre un utente autenticato sta creando un workspace, deve specificarne il nome prima della sua creazione	Soddisfatto
R-34-F-Ob	Quando un utente autenticato crea un workspace gli deve essere assegnato il ruolo di owner	Soddisfatto
R-35-F-Ob	Al momento della creazione del workspace, il sistema deve rifiutarne la creazione se esiste già uno workspace con lo stesso nome creato dallo stesso utente	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-36-F-Ob	Al momento della creazione del workspace, l'utente creatore deve visualizzare un messaggio di errore se il sistema ne rifiuta la creazione	Soddisfatto
R-37-F-De	Al momento della creazione del workspace, l'utente creatore deve visualizzare un messaggio di errore dedicato se il sistema ne rifiuta la creazione	Soddisfatto
R-38-F-Ob	Per ogni workspace a cui appartiene, un utente autenticato deve poter visualizzare lo username di un utente che appartiene a quel workspace, per ogni utente che appartiene a quel workspace	Soddisfatto
R-39-F-Ob	Per ogni workspace a cui appartiene, un utente autenticato deve poter visualizzare il ruolo di un utente che appartiene a quel workspace, per ogni utente che appartiene a quel workspace	Soddisfatto
R-40-F-Ob	Un utente autenticato che appartiene a un certo workspace deve poter rimuovere qualunque utente da quel workspace	Soddisfatto
R-41-F-De	Solamente gli utenti autenticati che hanno il ruolo di Project Manager all'interno di un workspace devono poter rimuovere qualunque utente da quel workspace, ammesso che l'utente rimosso non sia owner di tale workspace	Non soddisfatto
R-42-F-De	Quando viene effettuata una scansione, il sistema deve memorizzare in memoria persistente la data di inizio della scansione	Soddisfatto
R-43-F-De	Quando viene effettuata una scansione, il sistema deve memorizzare in memoria persistente la data di fine della scansione	Soddisfatto
R-44-F-De	Il sistema deve poter reperire la data dell'ultimo push verso qualunque repository inserito all'interno di qualunque workspace	Non soddisfatto
R-45-F-De	Quando il sistema reperisce il momento dell'ultimo push verso un repository, deve aggiornare lo stato di aggiornamento di tale repository a 'non aggiornata' qualora il momento di inizio dell'ultima scansione effettuata contro quel repository preceda quello dell'ultimo push verso lo stesso repository.	Non soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-46-F-De	Qualora il sistema tenti di reperire il momento dell'ultimo push verso un repository ma incontri problemi durante la connessione con GitHub, il sistema deve aggiornare lo stato del repository a 'non disponibile'	Non soddisfatto
R-47-F-De	Un utente autenticato che ha accesso a un workspace a cui appartiene un certo repository deve poter richiedere al sistema l'aggiornamento dello stato del repository	Non soddisfatto
R-48-F-De	Un utente autenticato che ha accesso a un workspace a cui appartiene un certo repository deve poter visualizzare lo stato di aggiornamento del repository	Soddisfatto
R-49-F-De	Un utente autenticato che ha accesso a un workspace a cui appartiene un certo repository deve poter visualizzare se è in corso una scansione verso il repository	Soddisfatto
R-50-F-Ob	Un utente autenticato che ha accesso a un workspace a cui appartiene un certo repository deve poter lanciare una scansione contro quel repository	Soddisfatto
R-51-F-Ob	Se, al momento del lancio della scansione, il sistema non riesce a connettersi al repository da scansionare, deve annullare la scansione e deve mostrare un errore all'utente	Soddisfatto
R-52-F-Ob	Se, al momento del lancio della scansione, il sistema non riesce ad accedere a un qualunque tool di scansione che richiede autenticazione, deve annullare la scansione e deve mostrare un errore all'utente	Soddisfatto
R-53-F-De	Un utente autenticato che ha accesso a un workspace che contiene uno o più repository deve poter lanciare, con un unico comando, una scansione contro ogni repository del workspace	Non soddisfatto
R-54-F-Ob	Un utente autenticato che ha accesso a un workspace a cui appartiene un certo repository deve poter visualizzare i branch attuali di tale repository	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-55-F-Ob	Un utente autenticato che ha accesso a un workspace a cui appartiene un certo repository deve poter lanciare una scansione su un branch di tale repository	Soddisfatto
R-56-F-De	Un utente autenticato che ha accesso a un workspace che contiene uno o più repository deve poter lanciare, previa selezione multipla di repository all'interno del workspace, una scansione contro ogni repository selezionato	Non soddisfatto
R-57-F-Ob	Un utente autenticato che visualizza che è in corso una scansione contro un repository deve poter anche interrompere tale scansione	Soddisfatto
R-58-F-Ob	Prima di effettuare una scansione, il sistema deve clonare localmente il repository bersaglio	Soddisfatto
R-59-F-Ob	Dopo il lancio del container di scansione deve essere avviato un container per l'esecuzione della stessa	Soddisfatto
R-60-F-Ob	Dopo il lancio del container di scansione, deve essere effettuata un'analisi statica di sicurezza del codice del progetto bersaglio della scansione, e generato un report	Soddisfatto
R-61-F-Ob	Dopo il lancio del container di scansione, devono essere generate remediation di una vulnerabilità, per ogni vulnerabilità individuata in fase di analisi statica di sicurezza del codice, e generato un report	Soddisfatto
R-62-F-Ob	Dopo il lancio del container di scansione, deve essere calcolata la code coverage del progetto bersaglio della scansione, e generato un report che ne includa i valori	Soddisfatto
R-63-F-Ob	Dopo il lancio del container di scansione, devono essere individuati i test eseguiti dal bersaglio della scansione, e falliti, e generato un report che li includa	Soddisfatto
R-64-F-De	Dopo il lancio del container di scansione, deve essere analizzata la correttezza logica dei test codificati del progetto bersaglio della scansione, e generato un report	Non soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-65-F-De	Dopo il lancio del container di scansione, deve essere analizzato l'utilizzo di best practice all'interno del codice del progetto bersaglio della scansione, e generato un report	Non soddisfatto
R-66-F-De	Dopo il lancio del container di scansione, deve essere analizzata la correttezza logica a livello di codice del progetto bersaglio della scansione, e generato un report	Non soddisfatto
R-67-F-Ob	Dopo il lancio del container di scansione, devono essere analizzate le librerie utilizzate dal progetto bersaglio della scansione, e generarne un report che consista dell'elenco delle librerie e le versioni utilizzate	Soddisfatto
R-68-F-De	Dopo il lancio del container di scansione, devono essere individuate le dipendenze vulnerabili utilizzate dal progetto bersaglio della scansione, e generato un report	Soddisfatto
R-69-F-Ob	Dopo il lancio del container di scansione, deve essere analizzata la qualità di un file README in root del progetto bersaglio della scansione, e generato un report	Soddisfatto
R-70-F-Ob	Dopo il lancio del container di scansione, deve essere analizzata la qualità dei commenti nel codice del progetto bersaglio della scansione, e generato un report	Soddisfatto
R-71-F-Ob	Dopo il lancio del container di scansione, un agente sintetizzatore deve unire i report precedentemente generati, quindi creare un unico riassunto a partire dagli stessi. Il riassunto deve contenere un voto finale compreso inclusivamente tra 1 e 10, ove 1 indica una pessima qualità del progetto relativamente alle scansioni effettuate, e 10 un'ottima qualità secondo gli stessi criteri.	Soddisfatto
R-72-F-Ob	Alla fine della scansione, il container generato deve trasmettere il report e delegare il salvataggio sul database dello stesso	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-73-F-De	Un utente autenticato che appartiene a un workspace deve poter visualizzare la lista dei tag raccolta definiti all'interno di tale workspace	Non soddisfatto
R-74-F-De	Un utente autenticato che appartiene a un workspace deve poter definire un tag raccolta unico all'interno di tale workspace	Non soddisfatto
R-75-F-De	Qualora un utente autenticato appartenente a un workspace tenti di definire un tag con lo stesso nome di un altro tag già definito all'interno dello workspace, il sistema non deve definire tale tag	Non soddisfatto
R-76-F-De	Qualora un utente autenticato appartenente a un workspace tenti di definire un tag con lo stesso nome di un altro tag già definito all'interno dello workspace, l'utente deve visualizzare un messaggio di errore e il sistema non deve definire tale tag	Non soddisfatto
R-77-F-De	Un utente autenticato che appartiene a un workspace deve poter eliminare un tag raccolta definito all'interno di tale workspace	Non soddisfatto
R-78-F-De	Un utente autenticato che appartiene a un workspace deve poter assegnare un tag raccolta a un qualunque numero di repository all'interno dello stesso workspace	Non soddisfatto
R-79-F-De	Un utente autenticato che appartiene a un workspace deve poter rimuovere un tag raccolta da un qualunque numero di repository all'interno dello stesso workspace	Non soddisfatto
R-80-F-Ob	Un utente autenticato che appartiene a un workspace deve poter visualizzare la lista di repository appartenenti a tale workspace	Soddisfatto
R-81-F-Ob	Un utente autenticato che appartiene a un workspace deve poter visualizzare la lista di repository appartenenti a tale workspace indicandone il nome	Soddisfatto
R-82-F-Ob	Un utente autenticato che appartiene a un workspace deve poter visualizzare la lista di repository appartenenti a tale workspace indicandone la data dell'ultima scansione nel branch develop	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-83-F-Ob	Un utente autenticato che appartiene a un workspace deve poter visualizzare la lista di repository appartenenti a tale workspace indicandone il voto di sicurezza del codice rilevato dall'ultima scansione nel branch develop	Soddisfatto
R-84-F-Ob	Un utente autenticato che appartiene a un workspace deve poter visualizzare la lista di repository appartenenti a tale workspace indicandone il voto della documentazione calcolato durante l'ultima scansione nel branch develop	Soddisfatto
R-85-F-Ob	Un utente autenticato che appartiene a un workspace deve poter visualizzare la lista di repository appartenenti a tale workspace indicandone la percentuale delle righe di codice testate durante l'ultima scansione nel branch develop	Soddisfatto
R-86-F-Ob	Un utente autenticato che appartiene a un workspace deve poter aggiungere un repository GitHub a tale workspace	Soddisfatto
R-87-F-Ob	Un utente autenticato che appartiene a un workspace deve poter aggiungere un repository GitHub a tale workspace indicandone opzionalmente una Access Token	Soddisfatto
R-88-F-Ob	Un utente autenticato che appartiene a un workspace deve poter aggiungere una Access Token a un repository GitHub appartenente a tale workspace	Soddisfatto
R-89-F-Ob	Un utente autenticato che appartiene a un workspace deve poter aggiornare l'Access Token a un repository GitHub appartenente a tale workspace, inserendone una nuova	Soddisfatto
R-90-F-Ob	Al momento di aggiunta a un workspace di un repository GitHub, il sistema deve controllare la raggiungibilità di tale repository	Soddisfatto
R-91-F-Ob	Un utente autenticato che appartiene a un workspace deve poter rimuovere un repository registrato a tale workspace	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-92-F-De	Prima della effettiva rimozione di un repository da un workspace a opera di un utente, il sistema deve richiederne la conferma o annullamento. In caso di mancata conferma, la procedura di rimozione del repository deve essere annullata	Non soddisfatto
R-93-F-De	Un utente autenticato che appartiene a un workspace deve poter effettuare una ricerca avanzata che permetta di restringere i risultati a tutti i repository appartenenti al workspace che utilizzano tutti i linguaggi di programmazione specificati dall'utente che effettua la ricerca	Non soddisfatto
R-94-F-De	Un utente autenticato che appartiene a un workspace deve poter effettuare una ricerca avanzata che permetta di restringere i risultati a tutti i repository a cui sono stati assegnati tutti i tag specificati dall'utente che effettua la ricerca	Non soddisfatto
R-95-F-De	Un utente autenticato che appartiene a un workspace deve poter effettuare una ricerca avanzata che permetta di restringere i risultati a tutti i repository la cui ultima scansione su develop ha registrato una percentuale di code coverage su righe di codice incluso all'interno di un intervallo specificato dall'utente che effettua la ricerca	Non soddisfatto
R-96-F-De	Un utente autenticato che appartiene a un workspace deve poter effettuare una ricerca avanzata che permetta di restringere i risultati a tutti i repository la cui ultima scansione su develop ha registrato un voto sulla documentazione incluso all'interno di un intervallo specificato dall'utente che effettua la ricerca	Non soddisfatto
R-97-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base al nome della repository, in ordine alfabetico crescente	Non soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-98-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base al nome della repository, in ordine alfabetico decrescente	Non soddisfatto
R-99-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base alla data di aggiunta al workspace dalla più recente alla meno recente	Non soddisfatto
R-100-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base alla data di aggiunta al workspace dalla meno recente alla più recente	Non soddisfatto
R-101-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base alla data dell'ultima scansione sul branch develop, dalla più recente alla meno recente	Non soddisfatto
R-102-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base alla data dell'ultima scansione sul branch develop, dalla meno recente alla più recente	Non soddisfatto
R-103-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base alla percentuale di code coverage sulle righe di codice registrata durante l'ultima scansione sul branch develop, in ordine crescente	Non soddisfatto
R-104-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base alla percentuale di code coverage sulle righe di codice registrata durante l'ultima scansione sul branch develop, in ordine decrescente	Non soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-105-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base al voto della documentazione registrato durante l'ultima scansione sul branch develop, in ordine crescente	Non soddisfatto
R-106-F-De	Un utente autenticato che appartiene a un workspace deve poter ordinare la lista dei repository appartenenti a tale workspace in base al voto della documentazione registrato durante l'ultima scansione sul branch develop, in ordine decrescente	Non soddisfatto
R-107-F-Ob	Un utente autenticato che appartiene a un workspace deve poter visualizzare nel dettaglio il report generato dall'ultima scansione di un repository, per ogni repository appartenente a tale workspace	Soddisfatto
R-108-F-Ob	Un report, relativo a un certo branch di un certo repository, visualizzato dall'utente deve contenere una sezione dedicata all'analisi dei test	Soddisfatto
R-109-F-Ob	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi dei test deve rappresentare le 4 metriche di code coverage (righe di codice, statement, branch, funzioni) registrate dalla scansione associata al report	Soddisfatto
R-110-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi dei test deve rappresentare, tramite un grafico a barre, le 4 metriche di code coverage (righe di codice, statement, branch, funzioni) registrate dalla scansione associata al report	Soddisfatto
R-111-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi dei test deve mostrare la lista dei test falliti registrati dalla scansione associata al report	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-112-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi dei test deve mostrare la lista dei test di qualità insufficiente	Non soddisfatto
R-113-F-Ob	Un report, relativo a un certo branch di un certo repository, visualizzato dall'utente deve contenere una sezione dedicata alle informazioni tecniche sul progetto del repository analizzato	Soddisfatto
R-114-F-Ob	La sezione di un report, relativo a un certo branch di un certo repository, dedicata alle informazioni tecniche sul progetto del repository analizzato, deve indicare il nome dei linguaggi registrati dalla scansione associata al report	Soddisfatto
R-115-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata alle informazioni tecniche sul progetto del repository analizzato, deve indicare il nome e la percentuale d'uso dei linguaggi registrati dalla scansione associata al report	Soddisfatto
R-116-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata alle informazioni tecniche sul progetto del repository analizzato, deve rappresentare con un grafico a torta l'utilizzo dei linguaggi registrati dalla scansione associata al report	Soddisfatto
R-117-F-Ob	La sezione di un report, relativo a un certo branch di un certo repository, dedicata alle informazioni tecniche sul progetto del repository analizzato, deve indicare il nome delle librerie registrate dalla scansione associata al report	Soddisfatto
R-118-F-Ob	La sezione di un report, relativo a un certo branch di un certo repository, dedicata alle informazioni tecniche sul progetto del repository analizzato, deve indicare il nome dei framework registrati dalla scansione associata al report	Soddisfatto
R-119-F-Ob	Un report, relativo a un certo branch di un certo repository, visualizzato dall'utente deve contenere una sezione dedicata all'analisi statica della sicurezza del codice	Soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-120-F-Ob	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi statica della sicurezza del codice deve indicare la lista delle vulnerabilità registrate dalla scansione associata al report	Soddisfatto
R-121-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi statica della sicurezza del codice deve indicare, per ogni vulnerabilità, le remediation registrate durante la scansione associata al report	Soddisfatto
R-122-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi statica della sicurezza del codice deve indicare, per ogni vulnerabilità, le conseguenze di una mancata sanificazione della vulnerabilità, se registrata durante la scansione associata al report	Soddisfatto
R-123-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi statica della sicurezza del codice deve mostrare un istogramma che rappresenta la distribuzione dei livelli di criticità delle vulnerabilità rilevate	Soddisfatto
R-124-F-Ob	Un report, relativo a un certo branch di un certo repository, visualizzato dall'utente deve contenere una sezione dedicata all'analisi della documentazione	Soddisfatto
R-125-F-Ob	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi della documentazione, deve includere un'analisi del file README in root del bersaglio della scansione associata al report	Soddisfatto
R-126-F-De	La sezione di un report, relativo a un certo branch di un certo repository, dedicata all'analisi della documentazione, deve includere un'analisi dei commenti nel codice del bersaglio della scansione associata al report	Soddisfatto
R-127-F-De	Un report, relativo a un certo branch di un certo repository, visualizzato dall'utente deve contenere una sezione dedicata all'analisi della qualità del codice	Non soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-128-F-Ob	Qualora un utente autenticato richieda la visualizzazione dell'ultimo report relativo a un certo branch di un certo repository appartenente a un workspace a cui l'utente ha accesso, ma a tale branch non sia associato alcun report, l'utente deve visualizzare un messaggio opportuno	Soddisfatto
R-129-F-Ob	Un utente autenticato che appartiene a un certo workspace e sta visualizzando un repository deve poter accedere alla lista di branch presenti su Github	Soddisfatto
R-130-F-Ob	Un utente autenticato che appartiene a un certo workspace e sta visualizzando un repository deve poter selezionare uno qualunque dei branch presenti su Github e visualizzarne il report associato terminato più di recente	Soddisfatto
R-131-F-De	Un utente autenticato che appartiene a un certo workspace deve poter visualizzare una visione aggregata di quel workspace	Non soddisfatto
R-132-F-De	Un utente autenticato che sta visualizzando la visione aggregata di uno workspace a cui appartiene deve poter visualizzare la media delle percentuali di righe di codice ricoperte da test, secondo l'ultima scansione terminata su develop, se presente. Se non sono mai state lanciate scansioni contro un repository, la sua percentuale di righe di codice ricoperte da test equivale a 0	Non soddisfatto
R-133-F-De	Un utente autenticato che sta visualizzando la visione aggregata di uno workspace a cui appartiene deve poter visualizzare la percentuale di repository all'interno del workspace con code coverage sulle righe di codice uguale o maggiore della soglia minima (70%), secondo l'ultima scansione terminata su develop, se presente. Se non sono mai state lanciate scansioni contro un repository, la sua percentuale di righe di codice ricoperte da test equivale a 0	Non soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-134-F-De	Un utente autenticato che sta visualizzando la visione aggregata di uno workspace a cui appartiene deve poter visualizzare la percentuale di code coverage minima tra i repository appartenenti a un workspace, secondo l'ultima scansione terminata su develop, se presente. Se non sono mai state lanciate scansioni contro un repository, la sua percentuale di righe di codice ricoperte da test equivale a 0	Non soddisfatto
R-135-F-De	Un utente autenticato che sta visualizzando la visione aggregata di uno workspace a cui appartiene deve poter visualizzare la distribuzione di utilizzo dei linguaggi di programmazione all'interno del workspace tramite un grafico a torta. I linguaggi fanno riferimento all'ultima scansione terminata su develop, se presente. L'utilizzo di un linguaggio fa riferimento alle percentuali calcolate in questa scansione, e non alle righe di codice effettive	Non soddisfatto
R-136-F-De	Un utente autenticato che sta visualizzando la visione aggregata di uno workspace a cui appartiene deve poter visualizzare la distribuzione della gravità delle vulnerabilità all'interno dei repository del workspace tramite un istogramma. Le vulnerabilità fanno riferimento all'ultima scansione terminata su develop, se presente	Non soddisfatto
R-137-F-De	Un utente autenticato che sta visualizzando la visione aggregata di uno workspace a cui appartiene deve poter visualizzare il voto minimo della documentazione tra i repository all'interno del workspace, secondo l'ultima scansione terminata sul branch develop, se effettuata	Non soddisfatto
R-138-F-De	Un utente autenticato che sta visualizzando l'ultimo report relativo a un certo branch di un certo repository deve poter aggiornare la visualizzazione del report a quello successivo, qualora un'altra scansione sia terminata dall'inizio della visione del report	Non soddisfatto
R-139-F-Op	Integrazione con CI/CD (GitHub Actions)	Non soddisfatto

Tabella 2: Tabella dei Requisiti Funzionali

Codice	Descrizione	Stato
R-140-F-Op	Analisi storica e confronto tra versioni	Non soddisfatto
R-141-F-Op	Ranking dei progetti per qualità complessiva	Non soddisfatto
R-142-F-Op	Remediation interattiva con anteprima	Non soddisfatto
R-143-F-Op	Notifiche integrate (Slack/Teams/Email)	Non soddisfatto
R-144-F-Op	Plugin system per nuovi agenti di analisi	Non soddisfatto

5.2 Requisiti di Qualità

Tabella 3: Tabella dei Requisiti di Qualità

Codice	Descrizione	Stato
R-1-Q-Ob	Coverage minimo dei test di unità del 70% tramite test automatizzati	Soddisfatto
R-2-Q-Ob	Presenza di documentazione ^G tecnica completa (manuale utente, documentazione API ^G)	Soddisfatto
R-3-Q-Ob	Le API ^G devono essere documentate tramite Swagger ^G /OpenAPI	Soddisfatto
R-4-Q-Ob	Deve essere presente un documento di Bug Reporting	Soddisfatto
R-5-Q-Ob	L'applicazione deve raggiungere un indice di manutenibilità ≥ 60 secondo le metriche di complessità ciclomatica	Soddisfatto
R-6-Q-De	Il sistema deve completare una scansione di un repository di dimensione media (< 50.000 LOC) entro 15 minuti	Soddisfatto
R-7-Q-Ob	L'analisi di sicurezza deve basarsi sullo standard OWASP Top 10 ^G	Soddisfatto
R-8-Q-De	L'interfaccia utente deve essere responsive e fruibile da dispositivi con risoluzioni da 360x640 (mobile) a 1920x1080 (desktop)	Non soddisfatto

5.3 Requisiti di Vincolo

Tabella 4: Tabella dei Requisiti di Vincolo

Codice	Descrizione	Stato
R-1-V-Ob	Backend e Orchestratore sviluppati in Node.js ($\geq 18.x$) o Python (≥ 3.10)	Soddisfatto
R-2-V-Ob	Frontend sviluppato utilizzando la libreria React ^G .js ($\geq 18.x$)	Soddisfatto
R-3-V-Ob	Database supportati: MongoDB ^G (≥ 6.0) o PostgreSQL (≥ 15)	Soddisfatto
R-4-V-Ob	L'infrastruttura cloud deve essere ospitata su Amazon Web Services ^G (AWS ^G)	Soddisfatto
R-5-V-Ob	L'autenticazione deve essere gestita tramite Amazon Cognito ^G	Soddisfatto
R-6-V-Ob	Il codice sorgente deve essere versionato tramite Git ^G	Soddisfatto